

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
Актуальность темы	6
Цель работы.....	7
Задачи работы	7
Объект и предмет исследования	8
Методы и средства	8
Практическая значимость.....	8
1 АНАЛИТИЧЕСКАЯ ЧАСТЬ	9
1.1 Обзор предметной области.....	9
1.2 Анализ существующих аналогов	10
1.3 Сравнительный анализ аналогов.....	12
1.4 Выводы по аналитической части.....	13
2 ПРОЕКТНАЯ ЧАСТЬ	16
2.1 Общая архитектура приложения	16
2.2 Модель С4: контекст и контейнеры	17
2.3 Диаграмма вариантов использования	20
2.4 Проектирование базы данных (ER-диаграмма).....	20
2.5 Диаграмма классов.....	21
2.6 Диаграмма компонентов.....	22
2.7 Интерфейсы взаимодействия модулей	22
2.8 Выводы по проектной части.....	23
3 ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ	24
3.1 Обоснование выбора языка программирования.....	24

3.2	Обоснование выбора фреймворка пользовательского интерфейса..	25
3.3	Обоснование выбора системы управления базой данных	26
3.4	Обоснование средств защиты данных	26
3.5	Вспомогательные библиотеки	27
3.6	Инструменты обеспечения качества и процесса разработки.....	28
3.7	Выводы по технологической части.....	29
4	РЕАЛИЗАЦИЯ	30
4.1	Структура проекта.....	30
4.2	Модели предметной области	30
4.3	Модуль безопасности.....	31
4.4	Хранение данных и шифрование «в покое»	32
4.5	Слой доступа к данным (Repository).....	33
4.6	Слой бизнес-логики	34
4.7	Сборка компонентов	36
4.8	Пользовательский интерфейс.....	36
4.9	Организация кода и контроль версий	37
4.10	Сборка и развёртывание под Android.....	37
4.11	Объём реализации	39
4.12	Выводы по разделу.....	40
5	ТЕСТИРОВАНИЕ	41
5.1	Методика тестирования	41
5.2	Модульное и интеграционное тестирование	41
5.3	Покрытие кода.....	42
5.4	Статический анализ безопасности (SAST)	43
5.5	Проверка зависимостей (SCA)	44

5.6 Непрерывная интеграция (Docker)	44
5.7 Анализ производительности.....	48
5.8 Исправленные дефекты.....	48
5.9 Выводы по разделу.....	49
ЗАКЛЮЧЕНИЕ	50
Выводы	50
Перспективы развития	51
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....	53
ПРИЛОЖЕНИЯ.....	55
Приложение А. Исходный код	55
Приложение Б. Снимки экрана приложения	56
Приложение В. Листинги кода приложения	67
Приложение Г. Листинги тестов	132

ВВЕДЕНИЕ

Актуальность темы

В современных условиях рост числа безналичных операций, многообразие платёжных инструментов (банковские карты, электронные кошельки, рассрочки, подписочные сервисы) и общая динамика повседневных расходов существенно усложняют контроль над личным бюджетом. Значительная часть граждан не ведёт систематического учёта доходов и расходов, что приводит к импульсивным тратам, отсутствию финансовой «подушки безопасности» и трудностям при достижении долгосрочных финансовых целей.

Повышение уровня финансовой грамотности населения является признанной социально значимой задачей, а одним из практических инструментов её решения выступают приложения для управления личными финансами. Подобные программные продукты позволяют фиксировать операции, классифицировать их по категориям, наглядно представлять структуру расходов и планировать накопления.

Вместе с тем многие существующие решения обладают рядом недостатков: обязательная привязка к облачным сервисам и передача чувствительных финансовых данных на сторонние серверы, навязчивая монетизация (реклама, платные подписки), избыточная сложность интерфейса. Особую значимость приобретает вопрос защиты персональных финансовых данных, поскольку информация о доходах и расходах относится к категории конфиденциальной.

В связи с этим актуальной является разработка кроссплатформенного мобильного приложения для учёта личных финансов, которое обеспечивает локальное хранение данных с их шифрованием, удобную категоризацию операций, наглядную визуализацию и поддержку финансового планирования без обязательной передачи данных третьим лицам.

Цель работы

Целью курсовой работы является демонстрация способности создавать сложный программный продукт с использованием современного языка программирования и актуальных технологий, пройдя полный цикл разработки — от идеи до развёртывания. В качестве предметной реализации цели разрабатывается кроссплатформенное мобильное приложение для учёта личных доходов и расходов с категоризацией операций, визуализацией данных, постановкой финансовых целей, напоминаниями о платежах и защитой локальной базы данных средствами шифрования.

Задачи работы

Для достижения поставленной цели необходимо решить следующие задачи:

1. провести анализ предметной области «персональные финансы», выполнить обзор существующих приложений-аналогов и сформулировать функциональные и нефункциональные требования к разрабатываемому программному продукту;
2. спроектировать архитектуру приложения с использованием современных методологий проектирования, разработать структуру базы данных и пользовательский интерфейс;
3. реализовать программный продукт средствами языка Python с применением фреймворка Kivy и СУБД SQLite, включая учёт операций, их категоризацию, визуализацию в виде диаграмм, финансовые цели, напоминания о платежах и шифрование локальной базы данных;
4. обеспечить качество кода посредством тестирования и статического анализа;
5. подготовить документацию по проекту и провести презентацию результатов разработки.

Объект и предмет исследования

Объект исследования — процесс учёта и управления личными финансами пользователя.

Предмет исследования — программные средства и методы автоматизации учёта личных доходов и расходов в кроссплатформенном мобильном приложении.

Методы и средства

При выполнении работы использовались методы анализа предметной области и сравнительного анализа аналогов, методы объектно-ориентированного проектирования, методы проектирования реляционных баз данных. В качестве средств реализации применялись язык программирования Python, фреймворк для разработки кроссплатформенных приложений Kivy и встраиваемая реляционная СУБД SQLite.

Практическая значимость

Практическая значимость работы заключается в создании готового к применению приложения для учёта личных финансов, которое может использоваться частными пользователями для контроля бюджета, планирования накоплений и повышения личной финансовой дисциплины. Особенностью продукта является локальное хранение данных с шифрованием, что обеспечивает конфиденциальность финансовой информации.

1 АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1 Обзор предметной области

Предметной областью разрабатываемого программного продукта являются **персональные (личные) финансы** — совокупность денежных средств физического лица и операций, связанных с их получением, расходованием, накоплением и планированием.

Управление личными финансами представляет собой процесс, включающий следующие основные действия:

- **учёт доходов** — фиксация поступлений денежных средств (зарботная плата, переводы, проценты по вкладам и т. п.);
- **учёт расходов** — регистрация трат с указанием суммы, даты и назначения;
- **категоризация операций** — отнесение каждой транзакции к определённой категории (продукты, транспорт, жильё, развлечения и др.), что позволяет анализировать структуру бюджета;
- **анализ и визуализация** — представление накопленных данных в наглядной форме (диаграммы, графики, сводки) для оценки финансового состояния;
- **планирование** — постановка финансовых целей (накопление на крупную покупку, формирование резерва) и контроль их достижения;
- **контроль обязательных платежей** — напоминания о регулярных и предстоящих платежах (аренда, кредиты, подписки) во избежание просрочек.

Ключевыми сущностями предметной области выступают: **пользователь**, **счёт** (наличные, карта, электронный кошелёк), **транзакция** (доход или расход), **категория**, **финансовая цель** и **напоминание**. Транзакция связывает счёт и

категорию, фиксируя движение средств; финансовые цели и напоминания обеспечивают функции планирования.

Особенностью предметной области является **высокая чувствительность обрабатываемых данных**. Сведения о доходах, расходах и финансовом положении лица относятся к конфиденциальной информации, поэтому к программным продуктам данного класса предъявляются повышенные требования в части защиты данных. Существуют два основных подхода к хранению информации:

1. **облачное хранение** — данные размещаются на серверах разработчика, что обеспечивает синхронизацию между устройствами, но предполагает передачу конфиденциальной информации третьей стороне;
2. **локальное хранение** — данные хранятся непосредственно на устройстве пользователя, что повышает конфиденциальность, но требует применения средств шифрования для защиты от несанкционированного доступа.

В рамках настоящей работы выбран подход с **локальным хранением данных и шифрованием базы данных**, обеспечивающий конфиденциальность финансовой информации без обязательной передачи её на внешние серверы.

Анализ предметной области позволяет выделить базовый набор функций, востребованных в приложениях для управления личными финансами: ведение учёта доходов и расходов, категоризация транзакций, визуализация данных, постановка финансовых целей, напоминания о платежах и обеспечение безопасности хранимых данных.

1.2 Анализ существующих аналогов

Для определения требований к разрабатываемому продукту и выявления его конкурентных преимуществ выполнен обзор пяти распространённых

приложений для учёта личных финансов: CoinKeeper, Zenmoney (Дзен-мани), Money Manager, Wallet by BudgetBakers и Spendee.

1.2.1 CoinKeeper

CoinKeeper — популярное приложение для учёта личных и семейных финансов с наглядным интерфейсом, основанным на метафоре «перетаскивания монеток» между категориями. Поддерживает планирование бюджета, постановку целей и напоминания о платежах. Реализована синхронизация с банками и облачное хранение данных. Распространяется по подписочной модели; в бесплатной версии функциональность существенно ограничена.

Достоинства: интуитивный интерфейс, развитые средства бюджетирования, синхронизация между устройствами. **Недостатки:** платная подписка, облачное хранение данных, ограниченная бесплатная версия.

1.2.2 Zenmoney (Дзен-мани)

Zenmoney — приложение с акцентом на автоматический импорт операций из банков посредством подключения к счетам и обработки СМС-уведомлений. Обеспечивает детальную аналитику и планирование бюджета, доступно на нескольких платформах с облачной синхронизацией.

Достоинства: автоматический импорт транзакций, развитая аналитика, кроссплатформенность. **Недостатки:** хранение данных в облаке, зависимость от подключения к банкам, платная подписка для полного функционала.

1.2.3 Money Manager

Money Manager — приложение для учёта доходов и расходов с поддержкой двойной записи, бюджетов и подробной отчётности в виде диаграмм. Преимущественно использует локальное хранение данных с возможностью резервного копирования.

Достоинства: локальное хранение данных, развитая визуализация и отчётность, работа без подключения к сети. **Недостатки:** отсутствие встроенного шифрования базы данных, перегруженный интерфейс, часть функций доступна только в платной версии.

1.2.4 Wallet by BudgetBakers

Wallet by BudgetBakers — комплексное приложение для управления личным бюджетом с синхронизацией банковских счетов, совместным ведением бюджета и подробной аналитикой. Ориентировано на облачную модель работы.

Достоинства: широкий функционал, синхронизация с банками, совместное использование бюджета. **Недостатки:** облачное хранение, платная подписка, избыточная сложность для базового учёта.

1.2.5 Spendee

Spendee — приложение с современным дизайном, поддержкой нескольких кошельков (в том числе совместных и криптовалютных), визуализацией расходов и подключением банковских счетов. Работает на основе облачной синхронизации.

Достоинства: привлекательный интерфейс, наглядная визуализация, поддержка нескольких валют. **Недостатки:** облачное хранение данных, ограниченная бесплатная версия, платные тарифы для расширенных возможностей.

1.3 Сравнительный анализ аналогов

Результаты сравнения рассмотренных приложений по основным критериям приведены в таблице 1.1.

Таблица 1.1 — Сравнительный анализ приложений-аналогов

Критерий	CoinKeeper	Zenmoney	Money Manager	Wallet	Spendee	Разрабатываемое приложение
Учёт доходов и расходов	+	+	+	+	+	+
Категоризация транзакций	+	+	+	+	+	+
Визуализация (диаграммы)	+	+	+	+	+	+
Финансовые цели	+	+	±	+	±	+
Напоминания о платежах	+	+	±	+	±	+
Локальное хранение данных	—	—	+	—	—	+
Шифрование базы данных	—	—	—	—	—	+
Работа без подключения к сети	—	±	+	±	—	+
Бесплатное использование	—	—	±	—	±	+
Кроссплатформенность	+	+	+	+	+	+

Условные обозначения: «+» — функция реализована; «±» — реализована частично или с ограничениями; «—» — функция отсутствует.

1.4 Выводы по аналитической части

Проведённый анализ предметной области и существующих приложений-аналогов позволяет сделать следующие выводы.

Все рассмотренные приложения обеспечивают базовый набор функций: учёт доходов и расходов, категоризацию транзакций и визуализацию данных. Большинство решений предоставляют средства финансового планирования и напоминания о платежах. Вместе с тем выявлен ряд общих недостатков:

- преобладание **облачной модели хранения данных**, предполагающей передачу конфиденциальной финансовой информации на серверы разработчиков;
- **отсутствие встроенного шифрования** локальной базы данных даже у приложений с локальным хранением;
- **ограниченность бесплатных версий** и распространение по подписочной модели.

Таким образом, ниша приложения с **локальным хранением данных и обязательным шифрованием базы данных**, обеспечивающего конфиденциальность без обязательной передачи данных третьим лицам и при этом доступного для свободного использования, остаётся недостаточно заполненной. Это подтверждает целесообразность разработки предлагаемого продукта.

На основании анализа сформулированы **требования к разрабатываемому программному продукту**.

Функциональные требования:

1. ведение учёта доходов и расходов с указанием суммы, даты, категории и комментария;
2. категоризация транзакций с возможностью использования предопределённых и пользовательских категорий;
3. визуализация финансовых данных в виде диаграмм (структура расходов, динамика по периодам);
4. постановка финансовых целей и отслеживание прогресса их достижения;
5. формирование напоминаний о предстоящих платежах;
6. просмотр истории операций с фильтрацией по периоду и категориям.

Нефункциональные требования:

1. **безопасность** — хранение данных в локальной базе данных с шифрованием;
2. **кроссплатформенность** — возможность работы приложения на разных платформах;
3. **производительность** — отзывчивость интерфейса при типовых объёмах данных;
4. **удобство использования** — простой и понятный интерфейс;
5. **автономность** — работа приложения без обязательного подключения к сети Интернет.

2 ПРОЕКТНАЯ ЧАСТЬ

2.1 Общая архитектура приложения

Для разрабатываемого приложения выбрана **многослойная (layered) архитектура**, обеспечивающая разделение ответственности между уровнями и слабую связанность модулей. Выделены четыре слоя:

1. **Слой представления (Presentation)** — пользовательский интерфейс на базе Kivy и библиотеки Material-компонентов KivyMD. Визуальная часть описана на декларативном языке разметки **KV** (роль представления), а обработка действий пользователя и связывание с сервисами сосредоточены в классе приложения **WalletApp** (роль контроллера). Прямого доступа к базе данных слой представления не имеет.
2. **Слой бизнес-логики (Service)** — сервисы, реализующие прикладные сценарии: учёт транзакций, управление категориями, счетами, финансовыми целями, напоминаниями, аналитика и построение диаграмм.
3. **Слой доступа к данным (Data Access)** — реализован по паттерну **Repository**: репозитории инкапсулируют операции чтения и записи сущностей в базу данных и скрывают детали работы с SQL.
4. **Слой хранения данных (Database)** — локальная реляционная база данных SQLite, файл которой шифруется алгоритмом AES-256-GCM (библиотека `pycryptodome`).

Отдельно выделены **кросс-срезовой модуль безопасности** (модуль `vault`: вывод ключа из пароля по PBKDF2, шифрование/расшифрование файла базы данных, открытие хранилища) и **модуль уведомлений** (отправка локальных уведомлений о наступивших платежах).

Аутентификация реализована по **мастер-паролю**: при входе из пароля пользователя выводится ключ шифрования и расшифровывается база данных;

без верного пароля данные недоступны. Отдельная регистрация нескольких пользователей в текущей версии не используется — сущность `User` и поля `user_id` заложены в модель данных как задел для будущей многопользовательности.

Преимущества выбранного подхода: изоляция бизнес-логики от деталей интерфейса и хранения, упрощение тестирования (сервисы и репозитории тестируются независимо от GUI), возможность замены слоя хранения без изменения вышележащих слоёв.

2.2 Модель C4: контекст и контейнеры

Для описания архитектуры на верхнем уровне использована нотация **C4** (диаграммы Context и Container), реализованная средствами C4-PlantUML.

2.2.1 Контекстная диаграмма (C4 — Level 1)

Контекстная диаграмма (C4 Level 1) — Приложение учёта личных финансов

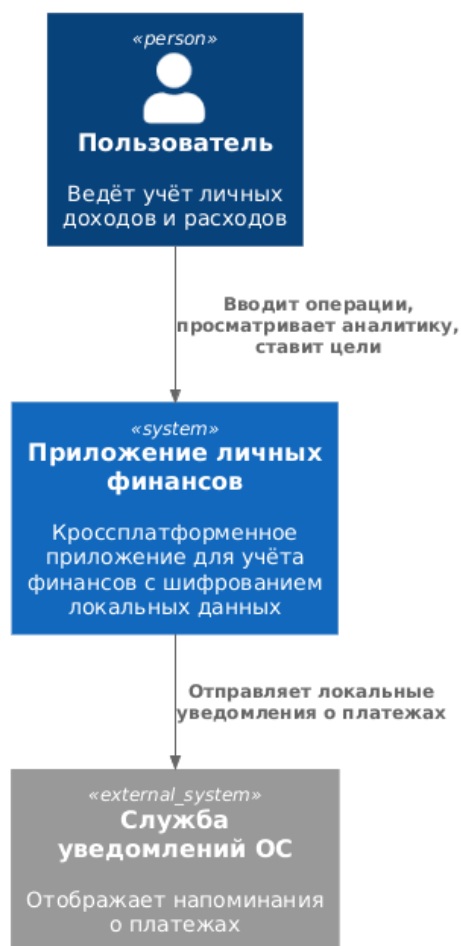


Рисунок 2.1 — Контекстная диаграмма (C4, уровень 1)

Приложение является **автономным**: внешние сетевые сервисы и серверы не используются, что соответствует требованию конфиденциальности (данные не покидают устройство). Единственное внешнее взаимодействие — с локальной службой уведомлений операционной системы для показа напоминаний.

2.2.2 Контейнерная диаграмма (C4 — Level 2)

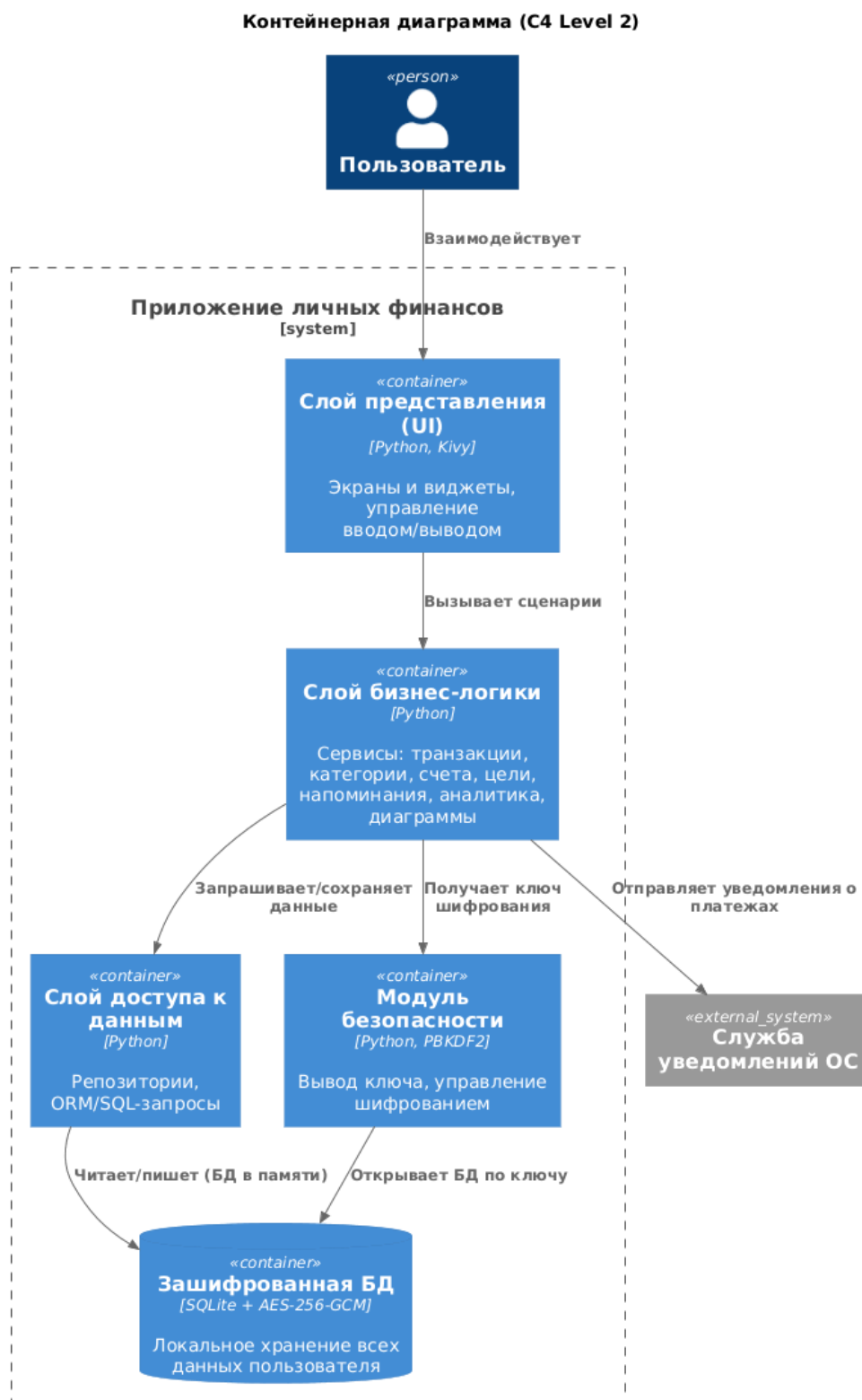


Рисунок 2.2 — Контейнерная диаграмма (C4, уровень 2)

2.3 Диаграмма вариантов использования

Основные варианты использования приложения представлены на диаграмме (PlantUML).

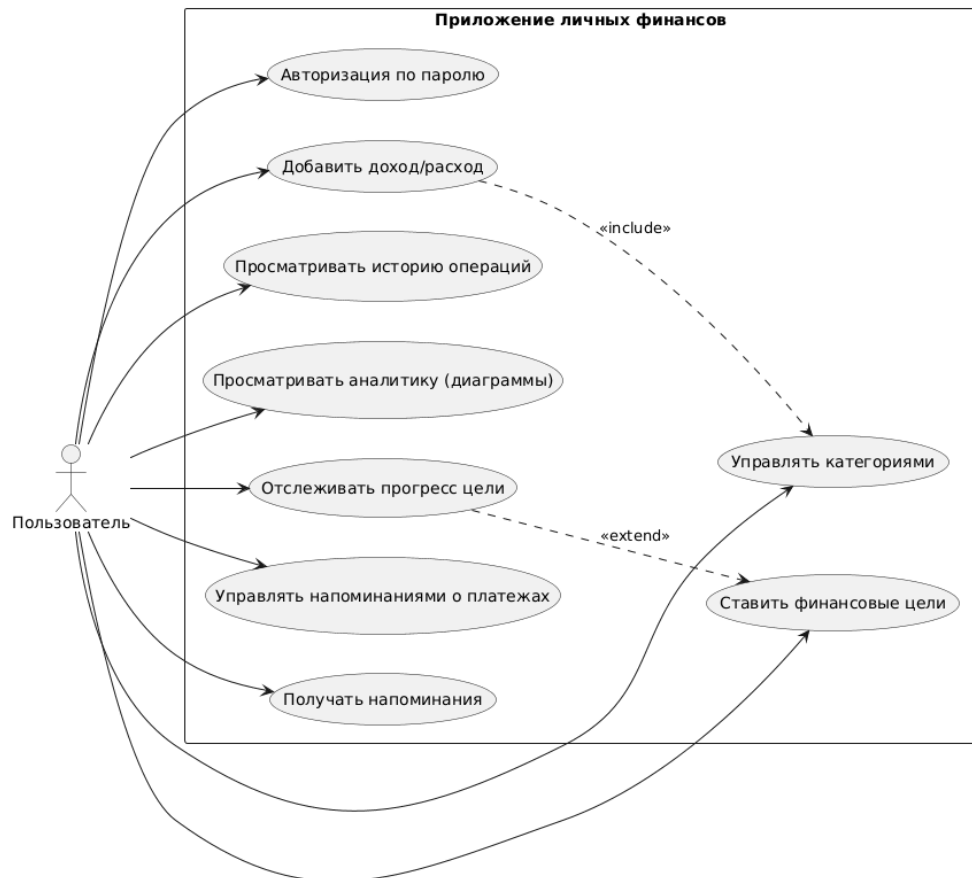


Рисунок 2.3 — Диаграмма вариантов использования

2.4 Проектирование базы данных (ER-диаграмма)

Структура базы данных спроектирована на основе сущностей предметной области. ER-диаграмма представлена средствами Mermaid.

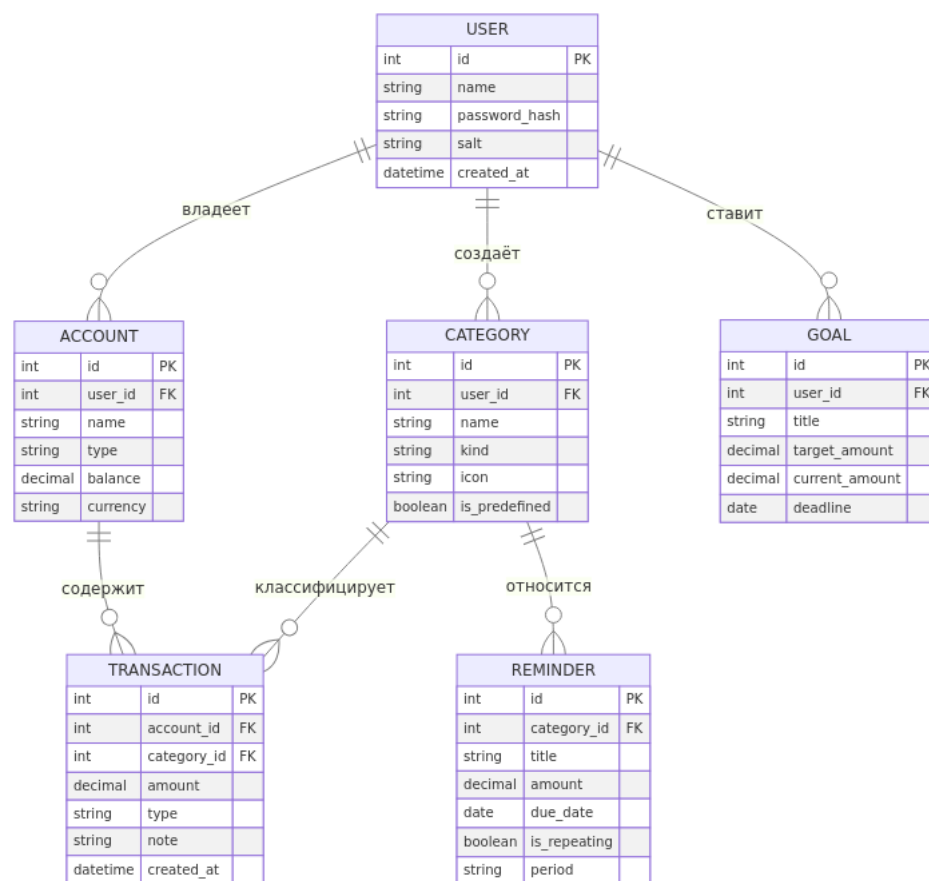


Рисунок 2.4 — ER-диаграмма базы данных

Тип транзакции (**type**) принимает значения «доход»/«расход»; поле **kind** категории определяет её принадлежность к доходам или расходам. Все таблицы хранятся в едином зашифрованном файле базы данных.

2.5 Диаграмма классов

Ключевые классы слоёв бизнес-логики и доступа к данным представлены на диаграмме классов (PlantUML).

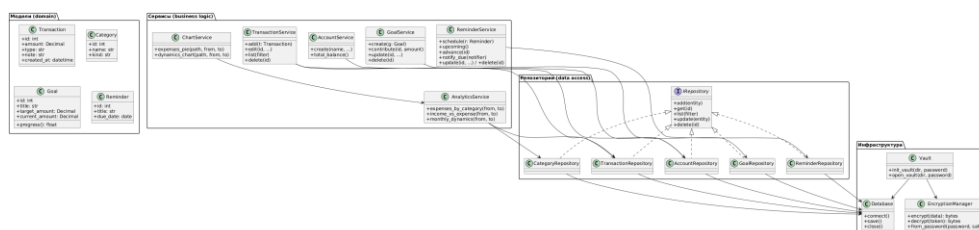


Рисунок 2.5 — Диаграмма классов

2.6 Диаграмма компонентов

Модульная структура приложения представлена на диаграмме компонентов (PlantUML).

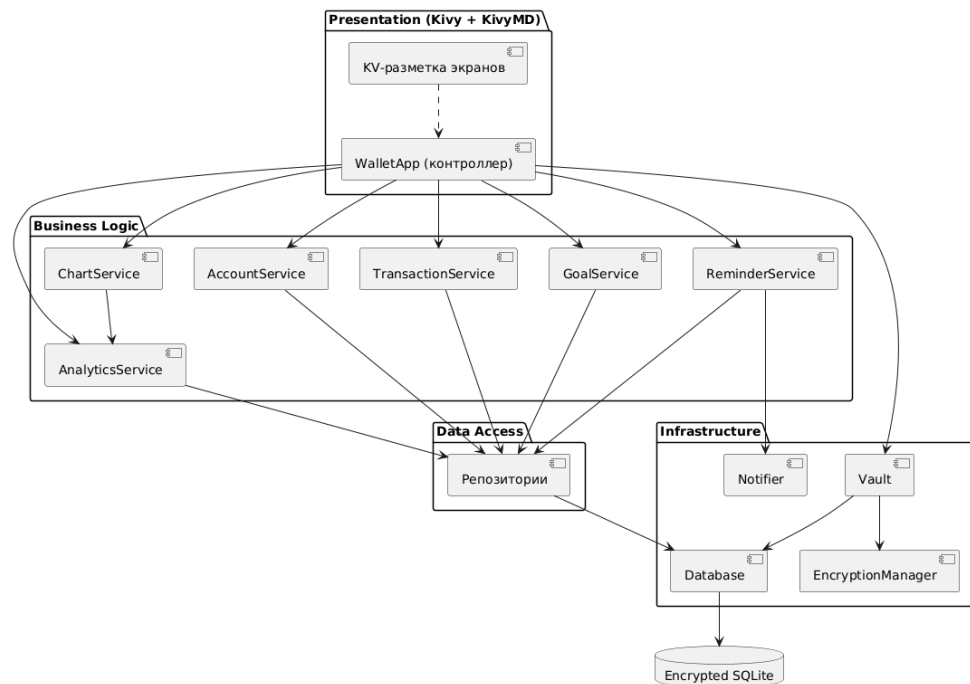


Рисунок 2.6 — Диаграмма компонентов

2.7 Интерфейсы взаимодействия модулей

Взаимодействие между слоями организовано через чётко определённые программные интерфейсы:

- **UI ↔ Бизнес-логика.** Класс приложения `WalletApp` вызывает методы сервисов (например, `TransactionService.add()`, `AnalyticsService.expenses_by_category()`, `ChartService.expenses_pie()`). UI не обращается к БД напрямую.
- **Бизнес-логика ↔ Доступ к данным.** Сервисы работают с репозиториями через общий интерфейс `IRepository` (методы `add`, `get`, `list`, `update`, `delete`), что позволяет заменять реализацию хранения, не меняя сервисы.

- **Доступ к данным ↔ БД.** Репозитории обращаются к классу Database, который инкапсулирует выполнение SQL-запросов к базе, загруженной в оперативную память.
- **Модуль безопасности ↔ БД.** При входе ключ выводится из пароля пользователя (PBKDF2), модуль Vault расшифровывает файл базы данных и загружает его в память; при сохранении база сериализуется и шифруется (AES-256-GCM). Без корректного пароля расшифровка невозможна.

Пример сценария «Добавление расхода и обновление аналитики» представлен на диаграмме последовательности (Mermaid).

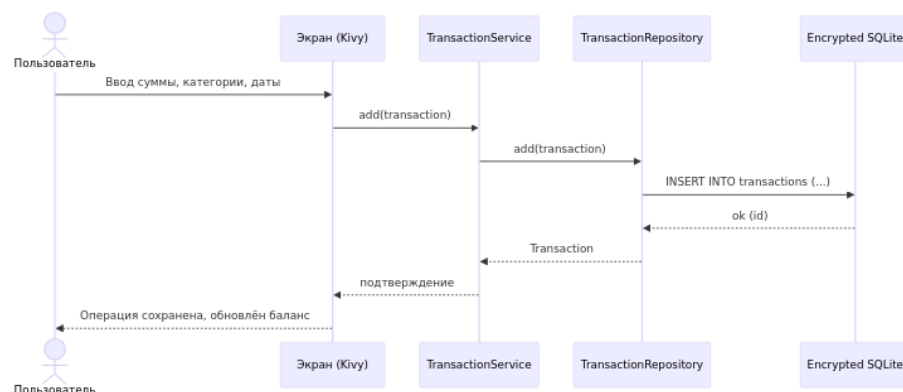


Рисунок 2.7 — Диаграмма последовательности

2.8 Выводы по проектной части

В проектной части определена многослойная архитектура приложения с применением паттерна Repository, разработаны диаграммы в нотациях C4, UML (диаграммы вариантов использования, классов, компонентов, последовательности) и ER-модель базы данных. Определены интерфейсы взаимодействия между слоями. Спроектированная архитектура обеспечивает разделение ответственности, тестируемость компонентов и изоляцию механизма шифрования данных, что соответствует сформулированным в аналитической части требованиям.

3 ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ

В данном разделе обосновывается выбор языка программирования, фреймворка пользовательского интерфейса, системы управления базой данных и вспомогательных инструментов разработки, обеспечивающих качество и безопасность программного продукта.

3.1 Обоснование выбора языка программирования

Для разработки рассматривались современные языки программирования, рекомендованные заданием: Python, C++, Go, Rust. Их сравнение применительно к задаче создания кроссплатформенного GUI-приложения с локальной базой данных приведено в таблице 3.1.

Таблица 3.1 — Сравнение языков программирования

Критерий	Python	C++	Go	Rust
Скорость разработки	высокая	низкая	средняя	средняя
Зрелые средства кроссплатформенного GUI	да (Kivy)	частично (Qt)	слабо	слабо
Порог входа	низкий	высокий	средний	высокий
Экосистема для работы с SQLite/шифрованием	богатая	богатая	хорошая	хорошая
Подходит для прикладного GUI-приложения	оптимально	избыточно	не профильно	избыточно

Выбран язык **Python**. Он обеспечивает высокую скорость разработки, имеет обширную экосистему библиотек (в том числе для работы с SQLite, шифрования, построения диаграмм) и зрелые средства создания кроссплатформенного интерфейса. Языки C++ и Rust ориентированы прежде всего на системное и высокопроизводительное программирование, а их применение для данного класса приложений приводит к неоправданному усложнению. Go не имеет зрелых кроссплатформенных GUI-решений, в особенности для мобильных платформ. Для прикладной задачи учёта личных

финансов производительности Python достаточно, а его преимущества в скорости и простоте разработки являются решающими.

3.2 Обоснование выбора фреймворка пользовательского интерфейса

Поскольку требуется кроссплатформенность (в том числе работа на мобильных устройствах), рассмотрены фреймворки для создания кроссплатформенного интерфейса на Python (таблица 3.2).

Таблица 3.2 — Сравнение средств построения интерфейса

Критерий	Kivy	BeeWare (Toga)	PyQt / PySide	Tkinter
Поддержка мобильных платформ (Android)	да	да	ограниченно	нет
Поддержка desktop	да	да	да	да
Зрелость и сообщество	высокая	развивающаяся	высокая	высокая
Гибкость оформления интерфейса	высокая	средняя	высокая	низкая
Лицензия	свободная (MIT)	свободная	GPL/коммерч.	свободная

Выбран фреймворк **Kivy**. Он специально предназначен для создания кроссплатформенных приложений с единой кодовой базой (Android, Windows, Linux, macOS), обладает собственным языком разметки интерфейса (KV), поддерживает аппаратное ускорение графики и распространяется по свободной лицензии. Tkinter не поддерживает мобильные платформы, PyQt/PySide имеют ограничения по лицензии и мобильной сборке, BeeWare на момент разработки менее зрелый. Для сборки приложения под Android применяется инструмент **Buildozer**. Поверх Kivy используется библиотека **KivyMD**, предоставляющая готовые компоненты в стиле Material Design (панель навигации, карточки, поля ввода, кнопки), что повышает качество и единообразие интерфейса.

3.3 Обоснование выбора системы управления базой данных

Приложение реализует концепцию локального хранения данных без обязательного использования сети, поэтому требуется встраиваемая СУБД, не требующая отдельного сервера. Сравнение вариантов приведено в таблице 3.3.

Таблица 3.3 — Сравнение систем хранения данных

Критерий	SQLite	PostgreSQL / MySQL	Файлы (JSON/CSV)
Встраиваемость (без сервера)	да	нет	да
Поддержка SQL и связей	да	да	нет
Поддержка шифрования	да (на уровне файла)	да (серверная)	вручную
Пригодность для мобильного приложения	оптимально	избыточно	ограниченно
Транзакционность (ACID)	да	да	нет

Выбрана СУБД **SQLite**. Это встраиваемая реляционная СУБД, не требующая отдельного серверного процесса, что идеально подходит для локального мобильного приложения. SQLite поддерживает язык SQL, транзакции (ACID) и связи между таблицами. Серверные СУБД (PostgreSQL, MySQL) избыточны и противоречат концепции автономности, а хранение в обычных файлах не обеспечивает целостности и удобства запросов.

3.4 Обоснование средств защиты данных

Ключевое требование к приложению — конфиденциальность финансовых данных. Для шифрования локальной базы данных применяется библиотека **pyscryptodome** с алгоритмом **AES-256** в режиме **GCM**. Шифруется **весь файл базы данных «в покое»**: во время работы база находится в оперативной памяти, при сохранении её содержимое сериализуется и шифруется, а при открытии — расшифровывается и загружается обратно.

Режим GCM обеспечивает не только конфиденциальность, но и контроль целостности данных: при неверном ключе расшифровка завершается ошибкой, что используется для проверки правильности пароля.

Ключ шифрования не хранится в открытом виде, а **выводится из пароля пользователя** функцией формирования ключа **PBKDF2-HMAC-SHA256** (с числом итераций 200 000) с использованием случайной соли. Соль не является секретом и хранится в отдельном файле рядом с зашифрованной базой, что позволяет вывести ключ при следующем входе. Таким образом, при получении злоумышленником файла БД без знания пароля данные остаются недоступными.

В качестве производственной альтернативы рассматривается расширение **SQLCipher** (привязка `pysqlcipher3`), обеспечивающее прозрачное постраничное шифрование файла БД тем же алгоритмом AES-256 и той же схемой вывода ключа (PBKDF2). Оно требует нативной сборки под целевую платформу, поэтому в рамках работы выбрано решение на основе библиотеки `ruscryptodome`, которая собирается под Android без дополнительных нативных зависимостей (в отличие от библиотеки `cryptography`, требующей компилятора Rust).

3.5 Вспомогательные библиотеки

Для реализации отдельных функций применяются следующие библиотеки:

- **KivyMD (нативные компоненты)** — визуализация структуры расходов и динамики бюджета непосредственно в интерфейсе (горизонтальные диаграммы), кроссплатформенно и без тяжёлых зависимостей; **Matplotlib** дополнительно используется для построения изображений диаграмм (модуль `ChartService`) в тестах и настольной версии;

- **Локальные уведомления** о платежах: на Android — нативно через системный API (pyjnius: NotificationManager с созданием канала уведомлений, обязательного на Android 8+); на настольных платформах — через библиотеку **Plyer**;
- стандартные модули Python (datetime, decimal, sqlite3) — для работы с датами, точных денежных вычислений и доступа к БД.

3.6 Инструменты обеспечения качества и процесса разработки

В соответствии с обязательными требованиями к проекту применяется набор инструментов, обеспечивающих качество кода, безопасность и воспроизводимость процесса разработки (таблица 3.4). Инструменты выбраны по принципу необходимости и достаточности — без избыточных компонентов.

Таблица 3.4 — Инструменты разработки и обеспечения качества

Назначение	Инструмент	Комментарий
Контроль версий	Git	Семантические коммиты (Conventional Commits), ветвление: основная ветка и ветки функциональности
Модульное тестирование	pytest	Тестирование сервисов и репозиторий независимо от интерфейса
Статический анализ безопасности (SAST)	Bandit	Поиск типовых уязвимостей в коде на Python
Проверка зависимостей	pip-audit	Выявление известных уязвимостей в используемых библиотеках
Контейнеризация	Docker	Контейнер для запуска тестов и анализа в CI (без Docker Compose, так как отсутствуют дополнительные сервисы)
Документирование	README, docstrings, диаграммы	Документация кода и архитектуры
ИИ-ассистенты	AI-инструменты разработки	Применяются для ускорения написания и анализа кода

Контейнеризация средствами Docker применяется в ограниченном виде — для воспроизводимого запуска тестов и статического анализа в среде непрерывной интеграции. Docker Compose и контейнеризация самого GUI-приложения не используются, поскольку приложение является монолитным, не содержит дополнительных сервисов и работает с встроенной базой данных; их добавление было бы избыточным.

3.7 Выводы по технологической части

В технологической части обоснован выбор технологического стека: язык программирования **Python**, фреймворк кроссплатформенного интерфейса **Kivy** с библиотекой Material-компонентов **KivyMD**, встраиваемая СУБД **SQLite** с шифрованием файла базы данных средствами библиотеки **pycryptodome** (AES-256-GCM, ключ на основе PBKDF2). Определены вспомогательные библиотеки (Matplotlib для тестов/десктопа, Plyer и нативный API Android через **rujnius** для уведомлений) и инструменты обеспечения качества и безопасности (pytest, Bandit, pip-audit, Docker, Git). Выбранные технологии соответствуют сформулированным требованиям, обеспечивают кроссплатформенность, конфиденциальность данных и приемлемую трудоёмкость разработки.

4 РЕАЛИЗАЦИЯ

В данном разделе описаны ключевые компоненты разработанного приложения и приведены фрагменты программного кода, иллюстрирующие реализацию спроектированной в разделе 2 архитектуры. Полный исходный код приведён в приложении.

4.1 Структура проекта

Приложение реализовано на языке Python и организовано по слоям в соответствии с принятой архитектурой:

```
wallet/  
├─ models/          # модели предметной области (entities.py)  
├─ core/            # инфраструктура: security.py, db.py, vault.py,  
notifications.py  
├─ repositories/    # слой доступа к данным (паттерн Repository)  
├─ services/        # слой бизнес-логики  
├─ ui/              # слой представления (app.py, KivyMD)  
└─ app_context.py   # сборка компонентов (внедрение зависимостей)  
tests/              # модульные тесты (pytest)
```

Каждый слой зависит только от нижележащего: представление обращается к сервисам, сервисы — к репозиториям, репозитории — к базе данных. Прямого доступа интерфейса к БД нет.

4.2 Модели предметной области

Сущности реализованы как классы-датаклассы. Денежные суммы хранятся в типе `Decimal` во избежание ошибок округления чисел с плавающей точкой. Типы операций и категорий описаны перечислениями.

Листинг 4.1 — Модель транзакции и вычисление прогресса цели

```
class TransactionType(str, Enum):  
    INCOME = "income"  
    EXPENSE = "expense"
```

```

@dataclass
class Transaction:
    amount: Decimal
    type: TransactionType
    account_id: int
    category_id: Optional[int] = None
    note: str = ""
    created_at: datetime = field(default_factory=datetime.now)
    id: Optional[int] = None

@dataclass
class Goal:
    title: str
    target_amount: Decimal
    current_amount: Decimal = Decimal("0")
    deadline: Optional[date] = None
    id: Optional[int] = None

    @property
    def progress(self) -> float:
        if self.target_amount <= 0:
            return 0.0
        return min(float(self.current_amount / self.target_amount), 1.0)

```

4.3 Модуль безопасности

Ключ шифрования не хранится в открытом виде, а выводится из пароля пользователя функцией PBKDF2-HMAC-SHA256 с солью. Данные шифруются алгоритмом AES-256 в режиме GCM, обеспечивающем конфиденциальность и контроль целостности.

Листинг 4.2 — Вывод ключа и шифрование (core/security.py)

```

PBKDF2_ITERATIONS = 200_000
KEY_LENGTH = 32 # 256 бит -> AES-256
NONCE_LENGTH = 12
TAG_LENGTH = 16

def derive_key(password: str, salt: bytes,
               iterations: int = PBKDF2_ITERATIONS) -> bytes:
    return hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), salt, iterations,
        dklen=KEY_LENGTH)

```

```

class EncryptionManager:
    def __init__(self, key: bytes):
        if len(key) != KEY_LENGTH:
            raise ValueError(f"Ключ должен быть длиной {KEY_LENGTH}
байт")
        self._key = key

    def encrypt(self, data: bytes) -> bytes:
        nonce = os.urandom(NONCE_LENGTH)
        cipher = AES.new(self._key, AES.MODE_GCM, nonce=nonce)
        ciphertext, tag = cipher.encrypt_and_digest(data)
        return nonce + tag + ciphertext

    def decrypt(self, token: bytes) -> bytes:
        nonce = token[:NONCE_LENGTH]
        tag = token[NONCE_LENGTH:NONCE_LENGTH + TAG_LENGTH]
        ciphertext = token[NONCE_LENGTH + TAG_LENGTH:]
        cipher = AES.new(self._key, AES.MODE_GCM, nonce=nonce)
        return cipher.decrypt_and_verify(ciphertext, tag)

```

4.4 Хранение данных и шифрование «в покое»

Во время работы база данных находится в оперативной памяти. При сохранении её содержимое сериализуется (`sqlite3.serialize`), шифруется и записывается в файл `wallet.db.enc`; при открытии файл расшифровывается и загружается обратно (`deserialize`). Без верного пароля расшифровка завершается ошибкой проверки целостности GCM.

Листинг 4.3 — Открытие и сохранение зашифрованной БД (core/db.py)

```

def connect(self) -> sqlite3.Connection:
    self._conn = sqlite3.connect(":memory:")
    self._conn.row_factory = sqlite3.Row
    if self.path and self.path.exists():
        plaintext = self.encryption.decrypt(self.path.read_bytes())
        self._conn.deserialize(plaintext)
    self._conn.execute("PRAGMA foreign_keys = ON")
    self._conn.executescript(SCHEMA)
    self._conn.commit()
    return self._conn

def save(self) -> None:
    if not self.path:
        return
    data = self.connection.serialize()
    # атомарная запись: временный файл + os.replace, чтобы сбой
    # в процессе записи не повредил существующий файл БД

```

```
tmp_path = self.path.with_name(self.path.name + ".tmp")
tmp_path.write_bytes(self.encryption.encrypt(data))
os.replace(tmp_path, self.path)
```

Модуль `vault` управляет хранилищем: соль (не является секретом) хранится в отдельном файле, что позволяет вывести ключ при следующем входе.

Листинг 4.4 — Открытие хранилища по паролю (`core/vault.py`)

```
def open_vault(data_dir: str | Path, password: str) -> Database:
    salt = _salt_path(data_dir).read_bytes()
    key = derive_key(password, salt)
    db = Database(path=_db_path(data_dir),
    encryption=EncryptionManager(key))
    try:
        db.connect()
    except Exception as exc:                               # неверный ключ -> ошибка
расшифровки
        db.close()
        raise InvalidPasswordError("Неверный пароль") from exc
    return db
```

4.5 Слой доступа к данным (Repository)

Репозитории инкапсулируют SQL-операции. Общий интерфейс `IRepository` задаёт единый контракт CRUD, базовый класс `BaseRepository` реализует общие методы.

Листинг 4.5 — Базовый репозиторий (`repositories/base.py`)

```
class BaseRepository(IRepository[T]):
    table: str = ""

    def __init__(self, conn: sqlite3.Connection):
        self.conn = conn

    def get(self, entity_id: int) -> Optional[T]:
        cur = self.conn.execute(
            f"SELECT * FROM {self.table} WHERE id = ?", (entity_id,))
        row = cur.fetchone()
        return self._row_to_entity(row) if row else None

    def list(self) -> list[T]:
        cur = self.conn.execute(f"SELECT * FROM {self.table} ORDER BY
id")
        return [self._row_to_entity(r) for r in cur.fetchall()]
```

Репозиторий транзакций дополнительно поддерживает фильтрацию по счёту, категории, типу и периоду (используется в истории операций и аналитике).

4.6 Слой бизнес-логики

Сервисы реализуют прикладные сценарии. Например, при добавлении операции `TransactionService` проверяет сумму и пересчитывает баланс счёта: доход увеличивает баланс, расход уменьшает.

Листинг 4.6 — Учёт операции и пересчёт баланса
(*services/transaction_service.py*)

```
def add(self, transaction: Transaction) -> Transaction:
    if transaction.amount <= 0:
        raise ValueError("Сумма операции должна быть положительной")
    saved = self.transactions.add(transaction)
    self._apply_to_balance(saved, sign=1)
    return saved

def _apply_to_balance(self, tx: Transaction, sign: int) -> None:
    account = self.accounts.get(tx.account_id)
    if account is None:
        return
    delta = tx.amount if tx.type == TransactionType.INCOME else -
tx.amount
    account.balance += Decimal(sign) * delta
    self.accounts.update(account)
```

`AnalyticsService` агрегирует данные для диаграмм. В пользовательском интерфейсе диаграммы отображаются нативными виджетами KivyMD (горизонтальные полосы), что работает на всех платформах без тяжёлых зависимостей. Дополнительно модуль `ChartService` строит изображения диаграмм средствами Matplotlib (неинтерактивный backend Agg) — для настольной версии и тестов.

Листинг 4.7 — Расходы по категориям и построение круговой диаграммы

```
def expenses_by_category(self, date_from=None, date_to=None) ->
dict[str, Decimal]:
    cat_names = {c.id: c.name for c in self.categories.list()}
    result: dict[str, Decimal] = {}
    for tx in self.transactions.list_filtered(
        tx_type=TransactionType.EXPENSE, date_from=date_from,
date_to=date_to):
        name = cat_names.get(tx.category_id, "Без категории")
        result[name] = result.get(name, Decimal("0")) + tx.amount
    return result

def expenses_pie(self, path, date_from=None, date_to=None) -> Path:
    data = self.analytics.expenses_by_category(date_from, date_to)
    fig, ax = plt.subplots()
    if data:
        ax.pie([float(v) for v in data.values()],
            labels=list(data.keys()), autopct="%1.1f%%")
    ax.set_title("Структура расходов по категориям")
    fig.savefig(path)
    plt.close(fig)
    return Path(path)
```

ReminderService вычисляет следующую дату повторяющегося платежа и рассылает уведомления о наступивших платежах.

Листинг 4.8 — Периодичность платежей и рассылка уведомлений

```
def next_due_date(current: date, period: str) -> date:
    if period == "daily":
        return current + timedelta(days=1)
    if period == "weekly":
        return current + timedelta(weeks=1)
    if period == "monthly":
        return _add_months(current, 1)
    if period == "yearly":
        return _add_months(current, 12)
    raise ValueError(f"Неизвестная периодичность: {period!r}")

def notify_due(self, notifier: Notifier, until: date | None = None) ->
int:
    due = self.upcoming(until)
    for reminder in due:
        notifier.notify(title=f"Платёж: {reminder.title}",
```

```

message=f"Сумма {reminder.amount}, срок
{reminder.due_date}")
return len(due)

```

4.7 Сборка компонентов

Класс `AppContext` связывает слои: создаёт репозитории поверх общего соединения и предоставляет готовые сервисы слою представления (реализация принципа внедрения зависимостей).

Листинг 4.9 — Контейнер зависимостей (app_context.py)

```

class AppContext:
def __init__(self, db: Database, notifier: Optional[Notifier] =
None):
    self.db = db
    self.notifier = notifier or default_notifier()
    conn = db.connection

    self.transactions = TransactionRepository(conn)
    self.accounts = AccountRepository(conn)
    # ... прочие репозитории

    self.transaction_service = TransactionService(self.transactions,
self.accounts)
    self.analytics = AnalyticsService(self.transactions,
self.categories)
    self.chart_service = ChartService(self.analytics)
    # ... прочие сервисы

```

4.8 Пользовательский интерфейс

Интерфейс построен на KivyMD. Разметка экранов описана на декларативном языке KV, обработка действий и связь с сервисами — в классе `WalletApp`. Навигация организована нижней панелью вкладок (Главная, Операции, Платежи, Цели, Аналитика); вход выполняется по мастер-пароллю.

Листинг 4.10 — Фрагмент разметки экрана входа (KV)

```

<LoginScreen>:
MDBoxLayout:
orientation: 'vertical'
MDTextField:
id: password
hint_text: 'Пароль'
password: True

```

```
MDRaisedButton:
    text: 'Войти'
    on_release: app.login()
MDFlatButton:
    text: 'Создать хранилище'
    on_release: app.register()
```

Листинг 4.11 — Обработчик добавления операции (ui/app.py)

```
def add_transaction(self):
    ids = self._main_ids()
    try:
        amount = Decimal(ids.tx_amount.text)
    except (InvalidOperation, ValueError):
        self._toast("Введите корректную сумму")
        return
    is_expense = ids.tx_type.text == "Расход"
    tx_type = TransactionType.EXPENSE if is_expense else
TransactionType.INCOME
    created_at = datetime.combine(self.tx_date, datetime.now().time())
    self.context.transaction_service.add(Transaction(
        amount=amount, type=tx_type, account_id=self.account.id,
        category_id=self._resolve_category(ids.tx_category.text, ...),
        note=ids.tx_note.text, created_at=created_at))
    self.context.save()
    self.refresh_all()
```

Перед удалением любого элемента (операции, категории, цели, платежа) выводится диалог подтверждения; фактическое удаление выполняется только после согласия пользователя.

4.9 Организация кода и контроль версий

Разработка велась с применением системы контроля версий Git. Использована модель ветвления Git flow: основная ветка `main`, ветка разработки `develop` и тематические ветки `feature/*`, вливаемые в `develop` слиянием с сохранением истории (`--no-ff`). Сообщения коммитов оформлены по соглашению Conventional Commits (`feat`, `fix`, `refactor`, `test`, `chore`).

4.10 Сборка и развёртывание под Android

Сборка мобильного приложения выполняется инструментом **Buildozer**, который посредством `python-for-android` упаковывает Python-проект в Android-

пакет (APK). Buildozer работает в среде Linux, поэтому, поскольку разработка велась на ОС Windows, сборка выполнена в **Docker-контейнере** на основе официального образа `kivy/buildozer`.

Параметры сборки задаются в файле `buildozer.spec`: название и идентификатор приложения, список зависимостей (`python3`, `kivy`, `kivymd`, `pillow`, `plyer`, `pyjnius`, `pycryptodome`), запрашиваемые разрешения (`POST_NOTIFICATIONS` для уведомлений, запрашивается также в рантайме на Android 13+), целевые версии Android API и архитектуры процессора (`arm64-v8a`, `armeabi-v7a`). Для неинтерактивной сборки в контейнере включено автопринятие лицензий Android SDK (`android.accept_sdk_license`) и отключён запрос подтверждения запуска от имени root (`warn_on_root`). Точкой входа служит файл `main.py` в корне проекта.

Сборка запускается одной командой; каталог проекта монтируется в контейнер, а кэш Android SDK/NDK сохраняется в именованном томе для ускорения повторных сборок.

Листинг 4.12 — Команда сборки APK в Docker

```
docker run --rm \
-v "${PWD}:/home/user/hostcwd" \
-v "wallet_buildozer:/home/user/.buildozer" \
kivy/buildozer android debug
```

Первая сборка занимает продолжительное время: загружаются Android SDK/NDK и компилируются зависимости под обе архитектуры. По завершении готовый файл `*-debug.apk` помещается в каталог `bin/` и устанавливается на устройство.

При подготовке мобильной версии учтены особенности платформы:

- библиотека шифрования — **pycryptodome**, которая собирается под Android без компилятора Rust (в отличие от `cryptography`);

- каталог данных приложения определяется через `user_data_dir` (приватное хранилище приложения на устройстве), а не домашний каталог пользователя;
- визуализация в интерфейсе выполняется нативными средствами KivyMD, поэтому диаграммы работают на всех платформах; библиотека `matplotlib` (модуль `ChartService`) в мобильную сборку не включается как тяжёлая для `python-for-android` и используется только в тестах и настольной версии.

Приложение успешно собрано в APK, установлено и запущено на Android-устройстве, что подтверждает прохождение полного цикла разработки — от идеи до развёртывания.

4.11 Объём реализации

Разработанный продукт реализует весь набор функциональных требований, сформулированных в разделе 1. При этом приняты следующие проектные ограничения прототипа:

- приложение является **однопользовательским** — вход выполняется по мастер-паролю, защищающему зашифрованное хранилище; сущность пользователя (`User`) и связи `user_id` заложены в модель данных как задел для будущей многопользовательности;
- учёт ведётся по **одному счёту по умолчанию**; модель и сервис счетов поддерживают несколько счетов, вывод управления счетами в интерфейс отнесён к перспективам развития;
- уведомления о платежах являются **локальными внутри приложения** (рассылаются при входе) и отправляются на Android нативно через системный API (`pyjnius`, с созданием канала уведомлений); фоновые уведомления при закрытом приложении требуют отдельной интеграции (системный планировщик) и рассмотрены в заключении.

4.12 Выводы по разделу

В разделе описаны ключевые компоненты приложения и приведены фрагменты кода, демонстрирующие реализацию спроектированной архитектуры: модели предметной области, модуль безопасности (PBKDF2, AES-256-GCM), зашифрованное хранилище, слой доступа к данным (Repository), бизнес-логику, построение диаграмм и пользовательский интерфейс на KivyMD. Реализация соответствует архитектуре и требованиям, определённым в предыдущих разделах.

5 ТЕСТИРОВАНИЕ

В разделе описаны методика и результаты тестирования приложения, оценка покрытия кода, статический анализ безопасности (SAST), проверка зависимостей (SCA), анализ производительности и перечень исправленных дефектов.

5.1 Методика тестирования

Для автоматизированного тестирования применяется фреймворк **pytest**. Тесты разделены на модульные (проверка отдельных функций и классов) и интеграционные (проверка совместной работы слоёв). Бизнес-логика и слой доступа к данным тестируются независимо от графического интерфейса: в тестах используется база данных в оперативной памяти (`Database()` без файла), а общие объекты создаются через фикстуры (`tests/conftest.py`).

Запуск всех тестов выполняется командой **pytest**. Контроль качества (тесты, покрытие, SAST, SCA) автоматизирован в Docker-контейнере и описан в п. 5.6.

5.2 Модульное и интеграционное тестирование

Набор тестов содержит **66 тестов**, все проходят успешно. Распределение по компонентам приведено в таблице 5.1.

Таблица 5.1 — Состав набора тестов

Файл тестов	Что проверяется
<code>test_security.py</code>	вывод ключа (PBKDF2), хеширование/проверка пароля, шифрование/расшифрование AES-256-GCM, ошибка при неверном ключе
<code>test_db_encryption.py</code>	сохранение данных в зашифрованный файл, нечитаемость файла, отказ при неверном пароле
<code>test_vault.py</code>	создание/открытие хранилища, неверный пароль, смена пароля

test_transaction_service.py	учёт доходов/расходов, пересчёт баланса, фильтрация, валидация
test_transaction_edit.py	редактирование операции (сумма, тип, категория) с пересчётом баланса
test_account_service.py	создание счетов, суммарный баланс
test_category_service.py	предопределённые и пользовательские категории, удаление с обнулением ссылок
test_goal_service.py	создание, пополнение, прогресс, правка, удаление целей
test_reminder_service.py, test_reminder_recurrence.py	создание, периодичность, перенос, правка, удаление платежей
test_analytics_service.py	расходы по категориям, доходы/расходы, динамика по месяцам
test_chart_service.py	построение диаграмм (формат PNG)
test_notifications.py	отправка уведомлений, выборка наступивших платежей
test_integration.py	сквозной сценарий и персистентность: создание хранилища → ввод операций/целей/платежей → проверка баланса и аналитики → сохранение → повторное открытие тем же паролем и проверка данных; отказ при неверном пароле; сквозная смена пароля

Интеграционные тесты подтверждают корректную совместную работу всех слоёв и сохранение данных в зашифрованном хранилище между сессиями.

5.3 Покрытие кода

Покрытие измерено инструментом **pytest-cov**. Итоговые показатели приведены в таблице 5.2 (полный отчёт — в приложении, файл `reports/coverage.txt`).

Таблица 5.2 — Покрытие кода тестами

Слой / модуль	Покрытие
Модели (models)	99%
Безопасность и БД (core)	80–98%
Репозитории (repositories)	85–100%

Сервисы (services)	88–100%
Сборка компонентов (app_context)	100%
Бизнес-логика и инфраструктура (без UI)	≈ 93%
Слой представления (ui/app.py)	0% (не покрывается юнит-тестами)
Всего по проекту	55%

Высокое покрытие бизнес-логики и инфраструктуры ($\approx 93\%$) обеспечивается модульными и интеграционными тестами. Слой представления (GUI на KivyMD) модульными тестами не покрывается, так как требует графической среды; его корректность проверялась запуском приложения, а также реальной сборкой и запуском на Android-устройстве. Этим объясняется итоговый показатель 55%.

5.4 Статический анализ безопасности (SAST)

Статический анализ выполнен инструментом **Bandit** (`bandit -r wallet`). При первом запуске обнаружено 6 предупреждений; каждое проанализировано (таблица 5.3).

Таблица 5.3 — Результаты статического анализа и их разбор

Код	Severity	Суть	Анализ и решение
B413	High	«Библиотек а pyCrypto устарела»	Ложное срабатывание: используется <code>pycryptodome</code> — поддерживаемая замена с тем же пространством имён <code>Crypto</code> . Bandit их не различает. Подавлено с обоснованием (# <code>nosec B413</code>)
B608 (×4)	Medium	Возможная SQL-инъекция через f-строку	Ложное срабатывание: имена таблиц/столбцов — внутренние константы, все значения передаются параметрами запроса (?). Инъекция невозможна. Подавлено с обоснованием (# <code>nosec B608</code>)
B110	Low	<code>try/except/pass</code>	Намеренный fail-safe: сбой необязательного backend уведомлений не должен прерывать вход. Подавлено с обоснованием (# <code>nosec B110</code>)

После анализа и обоснованных подавлений повторный запуск Bandit показывает **0 проблем** (6 предупреждений помечены как проверенные). Реальных уязвимостей не выявлено. Отчёт — `reports/bandit.txt`.

5.5 Проверка зависимостей (SCA)

Проверка зависимостей на известные уязвимости выполнена инструментом **pip-audit** (`pip-audit -r requirements.txt`). Результат: **известных уязвимостей не обнаружено** (No known vulnerabilities found). Отчёт — `reports/pip-audit.txt`.

5.6 Непрерывная интеграция (Docker)

Контроль качества автоматизирован в контейнере Docker. Образ (Dockerfile) на базе `python:3.12-slim` устанавливает зависимости ядра и инструменты качества и запускает единым шагом тесты с покрытием, Bandit и `pip-audit`:

```
CMD ["sh", "-c",  
"pytest --cov=wallet --cov-report=term-missing && \  
bandit -r wallet && \  
pip-audit -r requirements.txt"]
```

Сборка и запуск выполняются командами:

```
docker build -t wallet-ci .  
docker run --rm wallet-ci
```

Прогон в контейнере проходит успешно: тесты зелёные, Bandit — 0 проблем, `pip-audit` — 0 уязвимостей. Эти же шаги описаны в конфигурации GitHub Actions (`.github/workflows/ci.yml`) и выполняются при каждом `push` и `pull request`. GUI-зависимости (Kivy/KivyMD) в CI-образ не включаются, так как слой представления не покрывается юнит-тестами, — это делает образ лёгким.

quality

succeeded 3 minutes ago in 38s

- >  Set up job
- >  Checkout
- >  Build CI image
- >  Run tests, SAST and SCA
- >  Post Checkout
- >  Complete job

Рисунок 5.1 – Прохождение тестов на github

```

===== test session starts =====
platform linux -- Python 3.12.13, pytest-9.0.3, pluggy-1.6.0 -- /usr/local/bin/python3.12
cachedir: .pytest_cache
rootdir: /app
configfile: pytest.ini
testpaths: tests
plugins: cov-7.1.0
collecting ... collected 66 items

tests/test_account_service.py::test_create_account PASSED [ 1%]
tests/test_account_service.py::test_empty_name_rejected PASSED [ 3%]
tests/test_account_service.py::test_total_balance PASSED [ 4%]
tests/test_analytics_service.py::test_expenses_by_category PASSED [ 6%]
tests/test_analytics_service.py::test_monthly_dynamics PASSED [ 7%]
tests/test_analytics_service.py::test_income_vs_expense PASSED [ 9%]
tests/test_auth_service.py::test_register_and_login PASSED [ 10%]
tests/test_auth_service.py::test_login_with_wrong_password PASSED [ 12%]
tests/test_auth_service.py::test_password_is_not_stored_plaintext PASSED [ 13%]
tests/test_auth_service.py::test_duplicate_registration_rejected PASSED [ 15%]
tests/test_category_service.py::test_ensure_defaults_creates_predefined PASSED [ 16%]
tests/test_category_service.py::test_add_custom_category PASSED [ 18%]
tests/test_category_service.py::test_delete_category_nullifies_transactions PASSED [ 19%]
tests/test_chart_service.py::test_expenses_pie_creates_png PASSED [ 21%]
tests/test_chart_service.py::test_income_expense_bar_creates_png PASSED [ 22%]
tests/test_chart_service.py::test_pie_with_no_data PASSED [ 24%]
tests/test_chart_service.py::test_dynamics_chart_creates_png PASSED [ 25%]
tests/test_db_encryption.py::test_data_persists_through_encrypted_file PASSED [ 27%]
tests/test_db_encryption.py::test_file_is_not_plaintext PASSED [ 28%]
tests/test_db_encryption.py::test_wrong_password_cannot_open PASSED [ 30%]
tests/test_goal_service.py::test_create_and_progress PASSED [ 31%]
tests/test_goal_service.py::test_contribute_updates_progress PASSED [ 33%]
tests/test_goal_service.py::test_goal_reached PASSED [ 34%]
tests/test_goal_service.py::test_invalid_target_rejected PASSED [ 36%]
tests/test_goal_service.py::test_update_goal PASSED [ 37%]
tests/test_goal_service.py::test_delete_goal PASSED [ 39%]
tests/test_integration.py::test_full_scenario_and_persistence PASSED [ 40%]
tests/test_integration.py::test_persistence_rejects_wrong_password PASSED [ 42%]
tests/test_integration.py::test_password_change_end_to_end PASSED [ 43%]
tests/test_notifications.py::test_null_notifier_records_messages PASSED [ 45%]
tests/test_notifications.py::test_default_notifier_is_notifier PASSED [ 46%]
tests/test_notifications.py::test_notify_due_sends_only_due PASSED [ 48%]
tests/test_reminder_recurrence.py::test_next_due_date[current0-daily-expected0] PASSED [ 50%]
tests/test_reminder_recurrence.py::test_next_due_date[current1-weekly-expected1] PASSED [ 51%]
tests/test_reminder_recurrence.py::test_next_due_date[current2-monthly-expected2] PASSED [ 53%]
tests/test_reminder_recurrence.py::test_next_due_date[current3-yearly-expected3] PASSED [ 54%]
tests/test_reminder_recurrence.py::test_unknown_period_rejected PASSED [ 56%]
tests/test_reminder_recurrence.py::test_advance_repeating_reminder PASSED [ 57%]
tests/test_reminder_recurrence.py::test_advance_non_repeating_keeps_date PASSED [ 59%]
tests/test_reminder_service.py::test_upcoming_includes_due_and_excludes_future PASSED [ 60%]
tests/test_reminder_service.py::test_update_reminder PASSED [ 62%]
tests/test_reminder_service.py::test_update_reminder_period PASSED [ 63%]
tests/test_reminder_service.py::test_delete_reminder PASSED [ 65%]
tests/test_reminder_service.py::test_upcoming_with_horizon PASSED [ 66%]
tests/test_security.py::test_derive_key_is_deterministic PASSED [ 68%]
tests/test_security.py::test_derive_key_length_is_256_bit PASSED [ 69%]
tests/test_security.py::test_different_salt_gives_different_key PASSED [ 71%]
tests/test_security.py::test_password_verification PASSED [ 72%]
tests/test_security.py::test_encryption_round_trip PASSED [ 74%]
tests/test_security.py::test_decrypt_with_wrong_key_fails PASSED [ 75%]
tests/test_security.py::test_invalid_key_length_rejected PASSED [ 77%]
tests/test_transaction_edit.py::test_edit_amount_recalculates_balance PASSED [ 78%]
tests/test_transaction_edit.py::test_edit_can_change_category PASSED [ 80%]
tests/test_transaction_edit.py::test_edit_can_clear_category PASSED [ 81%]
tests/test_transaction_edit.py::test_edit_without_category_keeps_it PASSED [ 83%]
tests/test_transaction_edit.py::test_edit_type_recalculates_balance PASSED [ 84%]
tests/test_transaction_service.py::test_income_increases_balance PASSED [ 86%]
tests/test_transaction_service.py::test_expense_decreases_balance PASSED [ 87%]
tests/test_transaction_service.py::test_negative_amount_rejected PASSED [ 89%]
tests/test_transaction_service.py::test_delete_reverts_balance PASSED [ 90%]
tests/test_transaction_service.py::test_filter_by_type PASSED [ 92%]
tests/test_vault.py::test_init_creates_files PASSED [ 93%]
tests/test_vault.py::test_data_persists_with_correct_password PASSED [ 95%]
tests/test_vault.py::test_wrong_password_raises PASSED [ 96%]
tests/test_vault.py::test_open_missing_vault_raises PASSED [ 98%]
tests/test_vault.py::test_change_password PASSED [100%]

```

Рисунок 5.2 – Локальный запуск тестов

```

===== tests coverage =====
coverage: platform linux, python 3.12.13-final-0
Name                               Stmts  Miss  Cover   Missing
-----
wallet/__init__.py                  1      0  100%
wallet/app_context.py               41      0  100%
wallet/core/__init__.py              0      0  100%
wallet/core/db.py                   49      9   82%    103, 113,
131, 133, 145-146, 149-151
wallet/core/notifications.py        52     29   44%    37-39, 54-
83, 90-93, 97
wallet/core/security.py             44      1   98%    72
wallet/core/vault.py               53      3   94%    47, 68, 70
wallet/models/__init__.py           2      0  100%
wallet/models/entities.py           69      1   99%    93
wallet/repositories/__init__.py      7      0  100%
wallet/repositories/account_repository.py 17      0  100%
wallet/repositories/base.py         20      0  100%
wallet/repositories/category_repository.py 16      2   88%    41-45
wallet/repositories/goal_repository.py 18      0  100%
wallet/repositories/reminder_repository.py 21      0  100%
wallet/repositories/transaction_repository.py 40      6   85%    76-77, 82-
83, 85-86
wallet/repositories/user_repository.py 22      2   91%    41-45
wallet/services/__init__.py          9      0  100%
wallet/services/account_service.py  16      0  100%
wallet/services/analytics_service.py 35      0  100%
wallet/services/auth_service.py     23      2   91%    23, 34
wallet/services/category_service.py  25      1   96%    27
wallet/services/chart_service.py    56      1   98%    107
wallet/services/goal_service.py     42      6   86%    35, 37, 51
, 54, 58, 67
wallet/services/reminder_service.py  70      7   90%    48, 56, 65
, 86, 89, 93, 96
wallet/services/transaction_service.py 52      4   92%    41, 60, 65
, 98
wallet/ui/__init__.py               0      0  100%
wallet/ui/app.py                    593    593    0%    11-1260
-----
TOTAL                               1393    667   52%
===== 66 passed in 3.56s =====
[main] INFO profile include tests: None
[main] INFO profile exclude tests: None
[main] INFO cli include tests: None
[main] INFO cli exclude tests: None
[main] INFO running on Python 3.12.13
[tester] WARNING nosec encountered (B608), but no failed test on file wal
let/repositories/base.py:49
[tester] WARNING nosec encountered (B608), but no failed test on file wal
let/repositories/base.py:56
[tester] WARNING nosec encountered (B608), but no failed test on file wal
let/repositories/base.py:61
[tester] WARNING nosec encountered (B608), but no failed test on file wal
let/repositories/transaction_repository.py:92
[manager] WARNING Test in comment: Android is not a test name or id, ignor
ing
Run started:2026-06-11 14:56:17.904784+00:00

Test results:
>> Issue: [B110:try_except_pass] Try, Except, Pass detected.
Severity: Low Confidence: High
CWE: CWE-703 (https://cwe.mitre.org/data/definitions/703.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b110\_try\_except\_pas
s.html
Location: wallet/core/notifications.py:92:8
91         return AndroidNotifier()
92         except Exception: # noqa: BLE001
93             pass
94     try:

-----

Code scanned:
Total lines of code: 2537
Total lines skipped (#nosec): 0
Total potential issues skipped due to specifically being disabled (e.g.,
#nosec BXXX): 6

Run metrics:
Total issues (by severity):
Undefined: 0
Low: 1
Medium: 0
High: 0
Total issues (by confidence):
Undefined: 0
Low: 0
Medium: 0
High: 1
Files skipped (0):

```

Рисунок 5.3 Локальный запуск тестов

5.7 Анализ производительности

Выполнен замер производительности слоя доступа к данным на наборе из 5000 операций:

- вставка 5000 операций — **0,053 с** ($\approx$ **0,011 мс** на операцию);
- 100 запросов истории с фильтром по категории по 5000 операций — **0,154 с** ($\approx$ **1,5 мс** на запрос, по 500 записей в выборке).

Показатели подтверждают, что при типовых для личных финансов объёмах данных приложение работает с большим запасом по скорости. Ускорению выборок (история, фильтры, аналитика) способствуют индексы по полям `account_id`, `category_id`, `type`, `created_at` и `due_date`, добавленные в схему БД.

5.8 Исправленные дефекты

В ходе тестирования, отладки и сборки выявлены и устранены дефекты, перечисленные в таблице 5.4.

Таблица 5.4 — Выявленные и исправленные дефекты

Дефект	Решение
Сборка под Android прерывалась на запросе подтверждения root	В <code>buildozer.spec</code> отключён интерактивный вопрос (<code>warn_on_root = 0</code>)
Не устанавливались build-tools (лицензии SDK не приняты)	Включено автопринятие лицензий (<code>android.accept_sdk_license = True</code>)
Библиотека <code>cryptography</code> не собирается под Android (требуется Rust)	Шифрование переведено на <code>pycryptodome</code> (AES-256-GCM, тот же алгоритм и схема ключа)
Запись данных в домашний каталог недоступна на Android	Каталог данных определяется через <code>user_data_dir</code> (приватное хранилище приложения)
<code>matplotlib</code> тяжёл для мобильной сборки (<code>python-for-android</code>)	Визуализация в интерфейсе переведена на нативные виджеты KivyMD (работает на всех платформах); <code>matplotlib</code> (<code>ChartService</code>) используется только в

	тестах/настольной версии и в APK не включается
Предупреждения SAST (Bandit)	Проанализированы; реальных уязвимостей нет, ложные срабатывания обоснованно подавлены (п. 5.4)

5.9 Выводы по разделу

Проведено модульное и интеграционное тестирование (66 тестов, все успешны) с покрытием бизнес-логики и инфраструктуры около 93%. Статический анализ (Bandit) и проверка зависимостей (pip-audit) не выявили реальных уязвимостей. Контроль качества автоматизирован в контейнере Docker и настроен как непрерывная интеграция. Анализ производительности подтвердил высокую скорость работы с данными. Выявленные в ходе отладки и сборки дефекты устранены. Качество и безопасность программного продукта обеспечены.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы разработан сложный программный продукт — кроссплатформенное приложение для управления личными финансами «Wallet» — и пройден полный цикл разработки: от анализа предметной области и проектирования до реализации, тестирования и развёртывания на мобильном устройстве. Тем самым достигнута поставленная цель работы.

Выводы

В процессе работы решены все поставленные задачи.

1. **Проведён анализ предметной области и сформулированы требования.** Рассмотрена область персональных финансов, выполнен сравнительный анализ пяти приложений-аналогов, выявлены их недостатки (облачное хранение данных, отсутствие шифрования, ограниченность бесплатных версий) и на этой основе сформулированы функциональные и нефункциональные требования к продукту.
2. **Спроектирована архитектура приложения.** Выбрана многослойная архитектура с применением паттерна Repository; разработаны диаграммы в нотациях C4, UML (вариантов использования, классов, компонентов, последовательности) и ER-модель базы данных; определены интерфейсы взаимодействия слоёв.
3. **Реализован программный продукт.** Средствами языка Python с фреймворком Kivu и библиотекой KivuMD реализованы учёт доходов и расходов, категоризация операций, визуализация данных в виде диаграмм, финансовые цели, напоминания о платежах и история операций с фильтрацией. Ключевая особенность — локальное хранение данных с шифрованием базы данных (AES-256-GCM, ключ из пароля через PBKDF2), обеспечивающее конфиденциальность без передачи данных третьим лицам.

4. **Обеспечено качество и безопасность.** Разработан набор из 66 модульных и интеграционных тестов (покрытие бизнес-логики и инфраструктуры около 93 %); статический анализ (Bandit) и проверка зависимостей (pip-audit) не выявили реальных уязвимостей; контроль качества автоматизирован в контейнере Docker и оформлен как непрерывная интеграция; разработка велась с использованием системы контроля версий Git по модели Git flow с семантическими коммитами.
5. **Подготовлена документация и пройдено развёртывание.** Составлена пояснительная записка, оформлены README и документация кода; приложение собрано в Android-пакет (APK) с помощью Buildozer и успешно запущено на мобильном устройстве.

Таким образом, разработанный продукт полностью соответствует сформулированным требованиям и подтверждает прохождение полного цикла создания программного обеспечения с применением современных технологий и инженерных практик.

Перспективы развития

Дальнейшее развитие приложения возможно по следующим направлениям:

- **многопользовательский режим** — задействование заложенной в модель данных сущности пользователя (вход по учётной записи, разделение данных между пользователями);
- **поддержка нескольких счетов** — учёт по разным счетам (наличные, карты, кошельки) и переводы между ними; соответствующая модель и сервис в продукте уже подготовлены;
- **бюджеты и лимиты по категориям** — установка месячных лимитов расходов с предупреждением о превышении;
- **экспорт и импорт данных** (в формате CSV), усиливающие принцип владения пользователем своими данными;

- **улучшения интерфейса** — тёмная тема оформления, поиск и сортировка в истории операций;
- **фоновые уведомления на Android при закрытом приложении** — реализация через системный планировщик AlarmManager и широковещательный приёмник (BroadcastReceiver) с использованием rujnius; в текущей версии уведомления о платежах являются локальными внутри приложения.

Перечисленные направления позволят расширить функциональность продукта, сохранив его ключевое преимущество — конфиденциальность и автономность хранения финансовых данных.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Лутц М. Изучаем Python. В 2 т. — 5-е изд. — М. : Вильямс, 2020. — 1600 с.
2. Рамальо Л. Python. К вершинам мастерства. — 2-е изд. — М. : ДМК Пресс, 2022. — 898 с.
3. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. — СПб. : Питер, 2018. — 352 с.
4. Brown S. The C4 model for visualising software architecture : сайт. — URL: <https://c4model.com> (дата обращения: 20.04.2026).
5. Python Software Foundation. Python 3 documentation : официальная документация. — URL: <https://docs.python.org/3/> (дата обращения: 20.04.2026).
6. Kivy Organization. Kivy: Cross-platform Python Framework for GUI apps : официальная документация. — URL: <https://kivy.org/doc/stable/> (дата обращения: 22.04.2026).
7. KivyMD. Material Design components for Kivy : официальная документация. — URL: <https://kivymd.readthedocs.io> (дата обращения: 22.04.2026).
8. SQLite. SQLite Documentation : официальная документация. — URL: <https://www.sqlite.org/docs.html> (дата обращения: 25.04.2026).
9. PyCryptodome. PyCryptodome Documentation : официальная документация. — URL: <https://pycryptodome.readthedocs.io> (дата обращения: 28.04.2026).
10. National Institute of Standards and Technology. FIPS PUB 197: Advanced Encryption Standard (AES). — Gaithersburg : NIST, 2001. — URL: <https://csrc.nist.gov/publications/detail/fips/197/final> (дата обращения: 28.04.2026).
11. Turan M. S. [et al.] NIST Special Publication 800-132: Recommendation for Password-Based Key Derivation (PBKDF2). — Gaithersburg : NIST, 2010. —

- URL: <https://csrc.nist.gov/publications/detail/sp/800-132/final> (дата обращения: 28.04.2026).
- 12.pytest Development Team. pytest: helps you write better programs : официальная документация. — URL: <https://docs.pytest.org> (дата обращения: 05.05.2026).
- 13.PyCQA. Bandit: a tool designed to find common security issues in Python code : документация. — URL: <https://bandit.readthedocs.io> (дата обращения: 05.05.2026).
- 14.pip-audit: Audits Python environments and dependency trees for known vulnerabilities : репозиторий. — URL: <https://github.com/pyupio/pip-audit> (дата обращения: 05.05.2026).
- 15.Kivy Organization. Buildozer documentation : официальная документация. — URL: <https://buildozer.readthedocs.io> (дата обращения: 10.05.2026).
- 16.Kivy Organization. python-for-android documentation : официальная документация. — URL: <https://python-for-android.readthedocs.io> (дата обращения: 10.05.2026).
- 17.Matplotlib Development Team. Matplotlib: Visualization with Python : официальная документация. — URL: <https://matplotlib.org/stable/> (дата обращения: 12.05.2026).
- 18.Docker Inc. Docker Documentation : официальная документация. — URL: <https://docs.docker.com> (дата обращения: 12.05.2026).
- 19.Conventional Commits : спецификация. — URL: <https://www.conventionalcommits.org> (дата обращения: 15.05.2026).
- 20.ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. — М. : Стандартинформ, 2017. — 27 с.

ПРИЛОЖЕНИЯ

Приложение А. Исходный код

Полный исходный код приложения размещён в репозитории системы контроля версий Git:

<https://github.com/s-t-r-i-x-h-u-b/Wallet>

Структура репозитория:

```
Wallet/
├── main.py                # точка входа для сборки
├── buildozer.spec          # конфигурация сборки Android (APK)
├── Dockerfile              # образ CI: тесты + Bandit + pip-audit
├── requirements.txt        # зависимости
├── .github/workflows/ci.yml# конфигурация непрерывной интеграции
├── wallet/
│   ├── models/            # модели предметной области
│   ├── core/              # security, db, vault, notifications
│   ├── repositories/      # слой доступа к данным (Repository)
│   ├── services/          # бизнес-логика
│   ├── ui/                # интерфейс на KivyMD
│   └── app_context.py      # сборка компонентов (DI)
├── tests/                 # модульные и интеграционные тесты
└── reports/               # отчёты: покрытие, Bandit, pip-audit
```

Ключевые фрагменты кода с пояснениями приведены в разделе 4 «Реализация». Отчёты автоматизированного контроля качества (покрытие кода, статический анализ, проверка зависимостей) находятся в каталоге `reports/`.

Приложение Б. Снимки экрана приложения

Снимки экрана сделаны на устройстве под управлением Android.

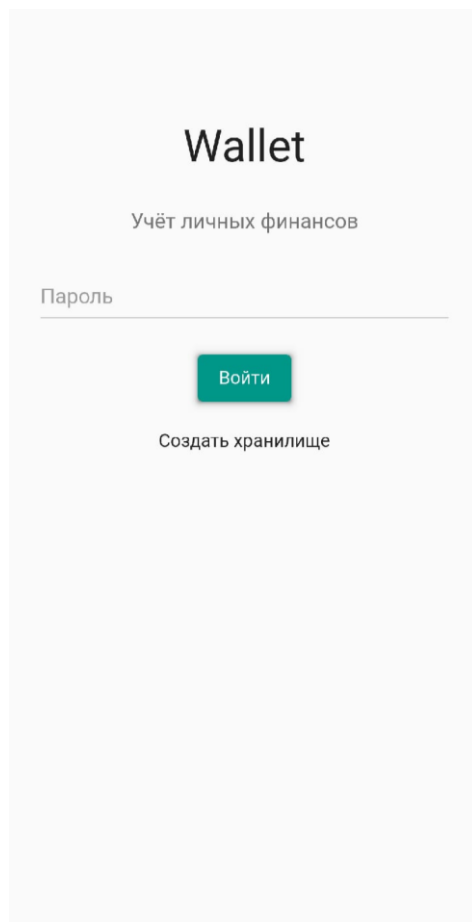


Рисунок Б.1 — Экран входа по мастер-паролю

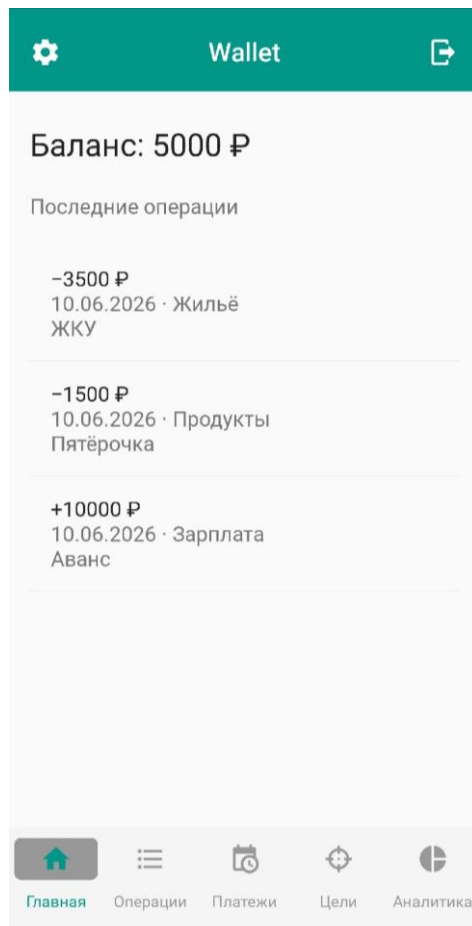


Рисунок Б.2 — Главный экран: баланс и последние операции

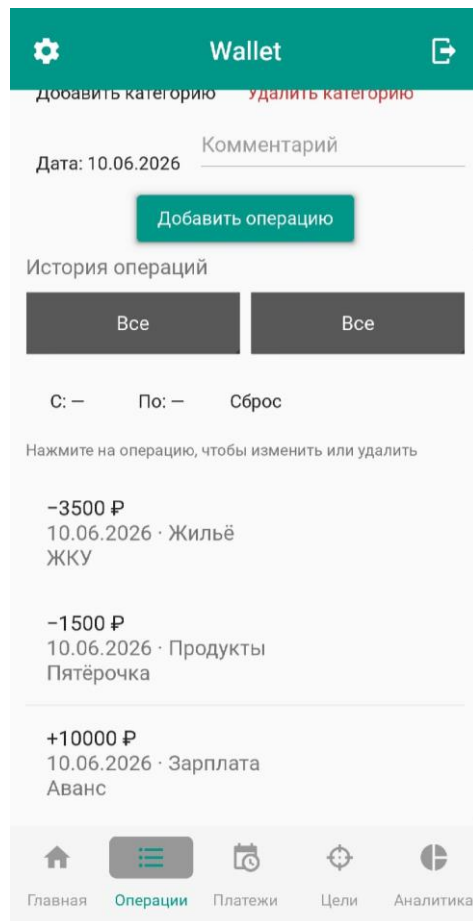


Рисунок Б.3 — Операции: добавление, выбор категории, фильтры и история

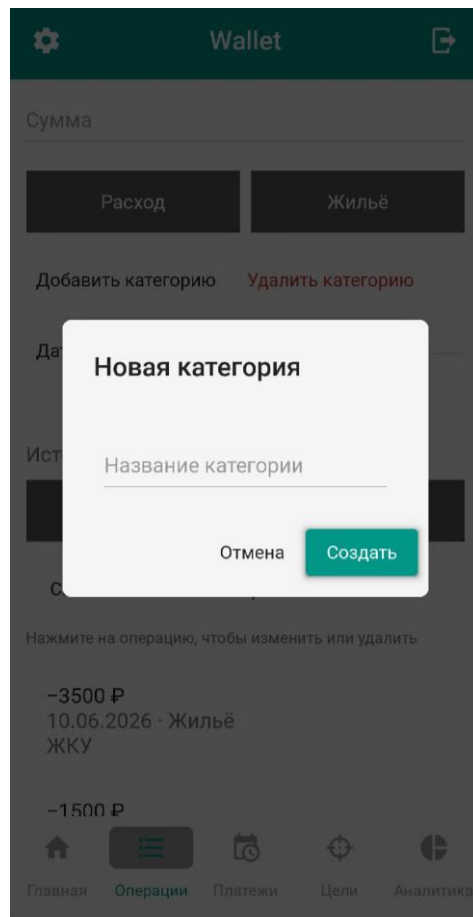


Рисунок Б.4 — Добавление пользовательской категории

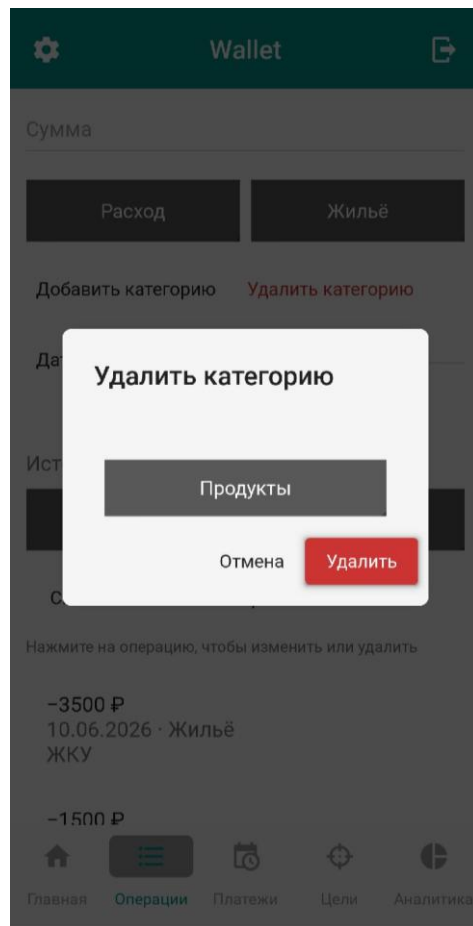


Рисунок Б.5 — Удаление категории

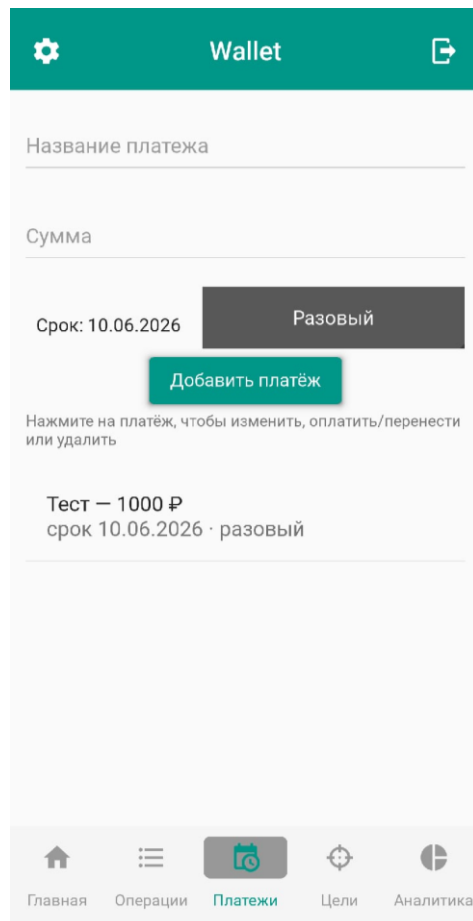
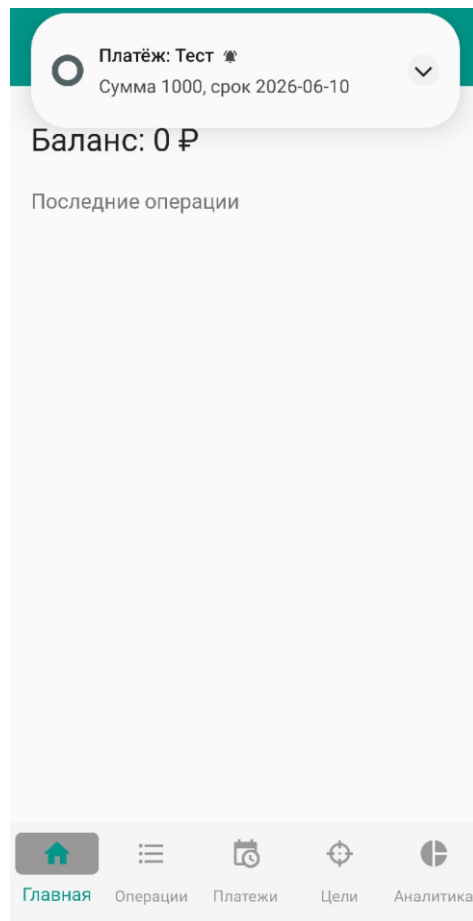


Рисунок Б.6 — Платежи: напоминания о предстоящих платежах



*Рисунок Б.7 — Системное уведомление о наступившем платеже
(Android)*

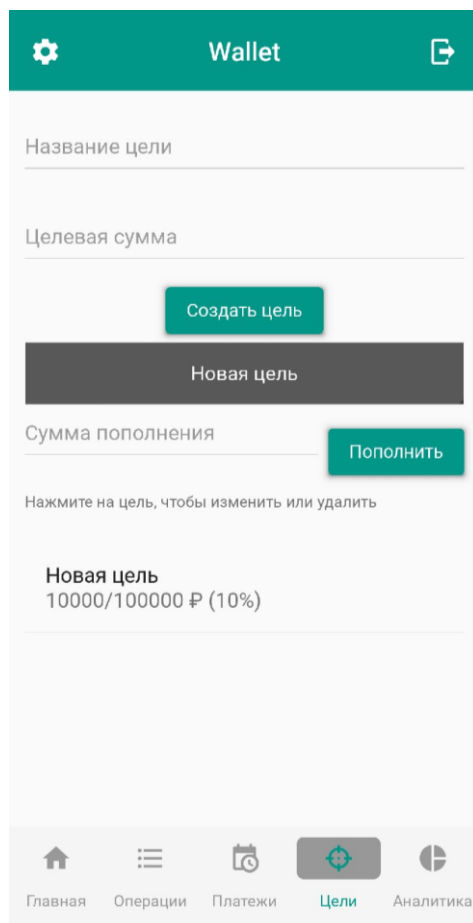


Рисунок Б.8 — Финансовые цели: создание, пополнение и прогресс

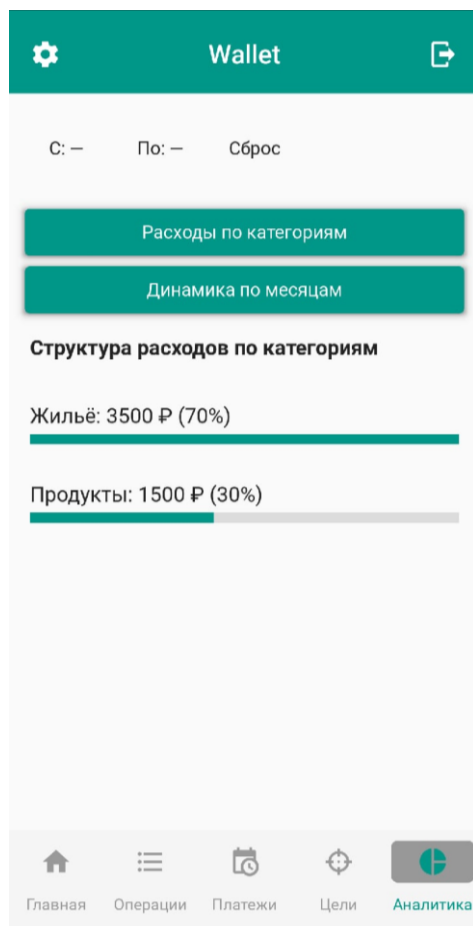


Рисунок Б.9 — Аналитика: структура расходов по категориям

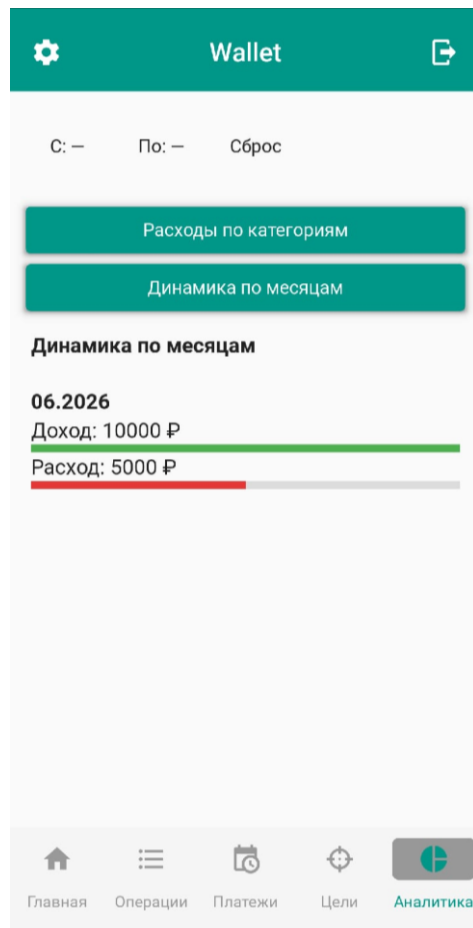


Рисунок Б.10 — Аналитика: динамика доходов и расходов по месяцам

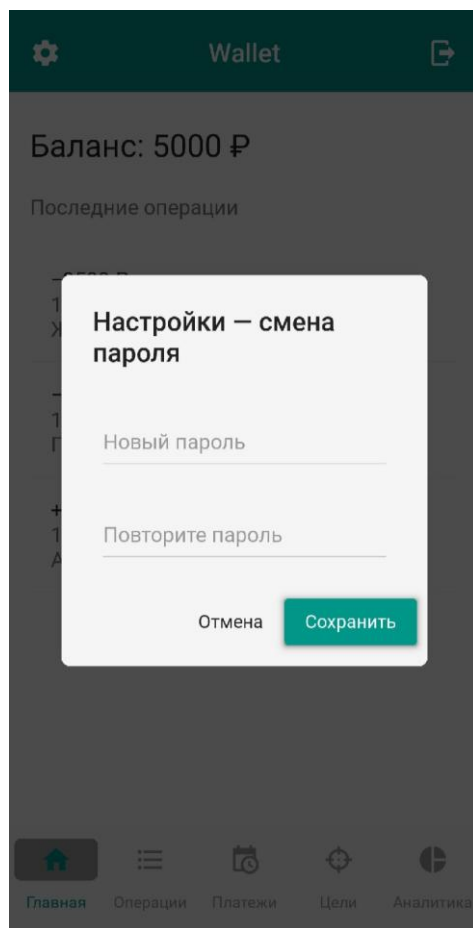


Рисунок Б.11 — Настройки: смена пароля

Приложение В. Листинги кода приложения

Ниже приведён полный исходный код приложения (точка входа и пакет *wallet/*: модели, инфраструктура, репозитории, сервисы, интерфейс).

Листинг В.1 — *main.py*

```
"""Точка входа приложения для сборки (Buildozer ищет main.py в корне)."""

from wallet.ui.app import run

if __name__ == "__main__":
    run()
```

Листинг В.2 — *wallet/init.py*

```
"""Wallet – кроссплатформенное приложение для учёта личных финансов.

Пакет организован по слоям:
    models          – модели предметной области;
    core             – инфраструктура (безопасность, доступ к БД);
    repositories     – слой доступа к данным (паттерн Repository);
    services         – слой бизнес-логики;
    ui               – слой представления (Kivy).
"""

__version__ = "0.1.0"
```

Листинг В.3 — *wallet/app_context.py*

```
"""Первичная интеграция компонентов приложения.

Класс AppContext связывает слой доступа к данным и слой бизнес-логики:
создаёт репозитории поверх общего соединения с БД и предоставляет готовые
сервисы слою представления.
"""

from __future__ import annotations

from pathlib import Path
from typing import Optional

from wallet.core.db import Database
from wallet.core.notifications import Notifier, default_notifier
from wallet.repositories import (
    AccountRepository,
    CategoryRepository,
    GoalRepository,
    ReminderRepository,
    TransactionRepository,
    UserRepository,
)
```

```

from wallet.services import (
    AccountService,
    AnalyticsService,
    AuthService,
    CategoryService,
    ChartService,
    GoalService,
    ReminderService,
    TransactionService,
)

class AppContext:
    """Контейнер зависимостей приложения."""

    def __init__(self, db: Database, notifier: Optional[Notifier] =
None):
        self.db = db
        self.notifier = notifier or default_notifier()
        conn = db.connection

        # Слой доступа к данным
        self.users = UserRepository(conn)
        self.accounts = AccountRepository(conn)
        self.categories = CategoryRepository(conn)
        self.transactions = TransactionRepository(conn)
        self.goals = GoalRepository(conn)
        self.reminders = ReminderRepository(conn)

        # Слой бизнес-логики
        self.auth = AuthService(self.users)
        self.account_service = AccountService(self.accounts)
        self.transaction_service = TransactionService(self.transactions,
self.accounts)
        self.category_service = CategoryService(self.categories)
        self.goal_service = GoalService(self.goals)
        self.reminder_service = ReminderService(self.reminders)
        self.analytics = AnalyticsService(self.transactions,
self.categories)
        self.chart_service = ChartService(self.analytics)

    @classmethod
    def init_vault(cls, data_dir: str | Path, password: str) ->
"AppContext":
        """Создать новое зашифрованное хранилище и открыть контекст."""
        from wallet.core import vault

        return cls(vault.init_vault(data_dir, password))

    @classmethod
    def open_vault(cls, data_dir: str | Path, password: str) ->

```

```

"AppContext":
    """Открыть существующее хранилище по паролю."""
    from wallet.core import vault

    return cls(vault.open_vault(data_dir, password))

    def change_password(self, new_password: str) -> None:
        """Сменить пароль (пин-код) хранилища."""
        from wallet.core import vault

        vault.change_password(self.db, new_password)

    def save(self) -> None:
        self.db.save()

    def close(self) -> None:
        self.db.close()

```

Листинг В.4 — wallet/core/init.py

"""Инфраструктурный слой: безопасность и доступ к базе данных."""

Листинг В.5 — wallet/core/db.py

"""Доступ к базе данных SQLite с шифрованием файла «в покое».

Во время работы база данных находится в оперативной памяти (:memory:). При сохранении содержимое сериализуется и шифруется (AES-256-GCM), при открытии – расшифровывается и загружается обратно. Без корректного ключа (выведенного из пароля пользователя) данные недоступны.

```

from __future__ import annotations

import os
import sqlite3
from pathlib import Path
from typing import Optional

from wallet.core.security import EncryptionManager

SCHEMA = """
CREATE TABLE IF NOT EXISTS users (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    name          TEXT NOT NULL,
    password_hash TEXT NOT NULL,
    salt          TEXT NOT NULL,
    created_at    TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS accounts (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

        user_id INTEGER REFERENCES users(id),
        name     TEXT NOT NULL,
        type     TEXT NOT NULL,
        balance  TEXT NOT NULL,
        currency TEXT NOT NULL
    );

CREATE TABLE IF NOT EXISTS categories (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id       INTEGER REFERENCES users(id),
    name          TEXT NOT NULL,
    kind          TEXT NOT NULL,
    icon          TEXT,
    is_predefined INTEGER NOT NULL DEFAULT 0
);

CREATE TABLE IF NOT EXISTS transactions (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    account_id    INTEGER NOT NULL REFERENCES accounts(id),
    category_id   INTEGER REFERENCES categories(id),
    amount        TEXT NOT NULL,
    type          TEXT NOT NULL,
    note          TEXT,
    created_at    TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS goals (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id       INTEGER REFERENCES users(id),
    title         TEXT NOT NULL,
    target_amount TEXT NOT NULL,
    current_amount TEXT NOT NULL DEFAULT '0',
    deadline      TEXT
);

CREATE TABLE IF NOT EXISTS reminders (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    category_id   INTEGER REFERENCES categories(id),
    title         TEXT NOT NULL,
    amount        TEXT NOT NULL,
    due_date      TEXT NOT NULL,
    is_repeating  INTEGER NOT NULL DEFAULT 0,
    period        TEXT
);

-- Индексы для ускорения частых выборок (история, фильтры, аналитика)
CREATE INDEX IF NOT EXISTS idx_tx_account ON transactions(account_id);
CREATE INDEX IF NOT EXISTS idx_tx_category ON transactions(category_id);
CREATE INDEX IF NOT EXISTS idx_tx_type ON transactions(type);
CREATE INDEX IF NOT EXISTS idx_tx_created ON transactions(created_at);
CREATE INDEX IF NOT EXISTS idx_reminders_due ON reminders(due_date);

```

```
"""
```

```
class Database:
```

```
    """Обёртка над соединением SQLite с поддержкой шифрования файла.
```

```
    :param path: путь к зашифрованному файлу БД (.db.enc). Если None –  
        база существует только в памяти (удобно для тестов).
```

```
    :param encryption: менеджер шифрования; обязателен для работы с  
    файлом.  
    """
```

```
    def __init__(  
        self,  
        path: Optional[str | Path] = None,  
        encryption: Optional[EncryptionManager] = None,  
    ):  
        self.path = Path(path) if path else None  
        self.encryption = encryption  
        self._conn: Optional[sqlite3.Connection] = None
```

```
    @property
```

```
    def connection(self) -> sqlite3.Connection:  
        if self._conn is None:  
            raise RuntimeError("База данных не открыта. Вызовите  
connect().")  
        return self._conn
```

```
    def connect(self) -> sqlite3.Connection:  
        """Открыть БД: расшифровать существующий файл или создать  
        новую."""  
        self._conn = sqlite3.connect(":memory:")  
        self._conn.row_factory = sqlite3.Row  
  
        if self.path and self.path.exists():  
            if self.encryption is None:  
                raise ValueError("Для открытия зашифрованного файла нужен  
ключ")  
            plaintext = self.encryption.decrypt(self.path.read_bytes())  
            self._conn.deserialize(plaintext)  
  
        self._conn.execute("PRAGMA foreign_keys = ON")  
        self._conn.executescript(SCHEMA)  
        self._conn.commit()  
        return self._conn
```

```
    def save(self) -> None:
```

```
        """Сериализовать БД, зашифровать и атомарно записать в файл.
```

```
        Запись выполняется во временный файл с последующей атомарной  
        заменой
```

```

        (os.replace). Это исключает повреждение существующего файла при
сбое
        в процессе записи: целевой `.db.enc` либо остаётся прежним, либо
        полностью заменяется новым.
        """
        if not self.path:
            return # in-memory режим – сохранять некуда
        if self.encryption is None:
            raise ValueError("Для сохранения зашифрованного файла нужен
ключ")
        data = self.connection.serialize()
        tmp_path = self.path.with_name(self.path.name + ".tmp")
        tmp_path.write_bytes(self.encryption.encrypt(data))
        os.replace(tmp_path, self.path)

    def close(self) -> None:
        if self._conn is not None:
            self._conn.close()
            self._conn = None

    def __enter__(self) -> "Database":
        self.connect()
        return self

    def __exit__(self, exc_type, exc, tb) -> None:
        if exc_type is None:
            self.save()
            self.close()

```

Листинг B.6 — wallet/core/notifications.py

```

"""Локальные уведомления с реализациями под разные платформы.

Notifier – протокол отправки уведомления. Реализации:
- AndroidNotifier – системные уведомления Android напрямую через API
(pyjnius)
  с созданием NotificationChannel (обязателен на Android 8+; без него
  система
  молча не показывает уведомление – типичная причина «не приходит»);
- PlyerNotifier – кроссплатформенная отправка через plyer (десктоп);
- NullNotifier – заглушка (тесты/среды без графической оболочки),
  запоминает
  отправленное.
"""

```

```

from __future__ import annotations

import os
from typing import Protocol, runtime_checkable

```

```

@runtime_checkable
class Notifier(Protocol):
    def notify(self, title: str, message: str) -> None: ...

class NullNotifier:
    """Заглушка: не показывает уведомления, но сохраняет их историю."""

    def __init__(self) -> None:
        self.sent: list[tuple[str, str]] = []

    def notify(self, title: str, message: str) -> None:
        self.sent.append((title, message))

class PlyerNotifier:
    """Системные уведомления через plyer (десктоп-платформы)."""

    def notify(self, title: str, message: str) -> None:
        from plyer import notification

        notification.notify(title=title, message=message)

class AndroidNotifier:
    """Системные уведомления Android через нативный API (pyjnius).

    Создаёт канал уведомлений (Android 8+) и публикует уведомление
    напрямую
    через NotificationManager.
    """

    _channel_ready = False
    _counter = 1
    _CHANNEL_ID = "wallet_payments"

    def notify(self, title: str, message: str) -> None:
        from jnius import autoclass

        PythonActivity = autoclass("org.kivy.android.PythonActivity")
        Context = autoclass("android.content.Context")
        Builder = autoclass("android.app.Notification$Builder")
        NotificationManager =
autoclass("android.app.NotificationManager")
        VERSION = autoclass("android.os.Build$VERSION")

        activity = PythonActivity.mActivity
        service = activity.getSystemService(Context.NOTIFICATION_SERVICE)

        if VERSION.SDK_INT >= 26:
            if not AndroidNotifier._channel_ready:

```

```

        NotificationChannel =
autoclass("android.app.NotificationChannel")
        channel = NotificationChannel(
            AndroidNotifier._CHANNEL_ID, "Платежи",
            NotificationManager.IMPORTANCE_HIGH)
        service.createNotificationChannel(channel)
        AndroidNotifier._channel_ready = True
        builder = Builder(activity, AndroidNotifier._CHANNEL_ID)
    else:
        builder = Builder(activity)

    builder.setSmallIcon(activity.getApplicationInfo().icon)
    builder.setContentTitle(title)
    builder.setContentText(message)
    builder.setAutoCancel(True)

    AndroidNotifier._counter += 1
    service.notify(AndroidNotifier._counter, builder.build())

def default_notifier() -> Notifier:
    """Выбрать реализацию под текущую платформу."""
    # python-for-android выставляет переменную окружения ANDROID_ARGUMENT
    if os.environ.get("ANDROID_ARGUMENT") is not None:
        try:
            return AndroidNotifier()
        except Exception: # noqa: BLE001 # nosec B110 - откат к
            # plyer/Null ниже
            pass
        try:
            import plyer # noqa: F401

            return PlyerNotifier()
        except Exception: # noqa: BLE001
            return NullNotifier()

```

Листинг В.7 — wallet/core/security.py

"""Модуль безопасности: вывод ключа из пароля и шифрование данных.

Ключ шифрования не хранится в открытом виде, а выводится из пароля пользователя функцией PBKDF2-HMAC-SHA256 с использованием соли. Шифрование данных выполняется алгоритмом AES-256 в режиме GCM, что обеспечивает конфиденциальность и контроль целостности.

В прототипе шифруется весь файл базы данных «в покое». В производственной версии аналогичную задачу решает расширение SQLCipher (прозрачное постраничное шифрование); используемый алгоритм (AES-256) и схема вывода ключа (PBKDF2) при этом совпадают.

"""


```

from __future__ import annotations

import base64
import hashlib
import hmac
import os

# Импорт из пакета ruscryptodome (актуальная поддерживаемая замена
# ruscrypto
# с тем же namespace `Crypto`). Bandit B413 здесь – ложное срабатывание:
# предупреждение относится к устаревшему ruscrypto, который не
# используется.
from Crypto.Cipher import AES # носек B413

PBKDF2_ITERATIONS = 200_000
KEY_LENGTH = 32 # 256 бит -> AES-256
SALT_LENGTH = 16
NONCE_LENGTH = 12
TAG_LENGTH = 16

def generate_salt() -> bytes:
    """Сгенерировать криптографически случайную соль."""
    return os.urandom(SALT_LENGTH)

def derive_key(password: str, salt: bytes, iterations: int =
PBKDF2_ITERATIONS) -> bytes:
    """Вывести 256-битный ключ из пароля и соли (PBKDF2-HMAC-SHA256)."""
    return hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), salt, iterations,
        dklen=KEY_LENGTH
    )

def hash_password(password: str, salt: bytes | None = None) -> tuple[str,
str]:
    """Вернуть (хеш пароля, соль) в base64 для хранения в БД."""
    if salt is None:
        salt = generate_salt()
    key = derive_key(password, salt)
    return base64.b64encode(key).decode(),
base64.b64encode(salt).decode()

def verify_password(password: str, password_hash_b64: str, salt_b64: str)
-> bool:
    """Проверить пароль по сохранённым хешу и соли (защита от тайминг-
атак)."""
    salt = base64.b64decode(salt_b64)
    expected = base64.b64decode(password_hash_b64)

```

```

    actual = derive_key(password, salt)
    return hmac.compare_digest(expected, actual)

class EncryptionManager:
    """Шифрование/расшифрование произвольных данных ключом AES-256-
    GCM."""

    def __init__(self, key: bytes):
        if len(key) != KEY_LENGTH:
            raise ValueError(f"Ключ должен быть длиной {KEY_LENGTH}
байт")
        self._key = key

    @classmethod
    def from_password(cls, password: str, salt: bytes) ->
    "EncryptionManager":
        """Создать менеджер, выведя ключ из пароля пользователя."""
        return cls(derive_key(password, salt))

    def encrypt(self, data: bytes) -> bytes:
        """Зашифровать данные. Результат: nonce(12) + tag(16) +
    шифртекст."""
        nonce = os.urandom(NONCE_LENGTH)
        cipher = AES.new(self._key, AES.MODE_GCM, nonce=nonce)
        ciphertext, tag = cipher.encrypt_and_digest(data)
        return nonce + tag + ciphertext

    def decrypt(self, token: bytes) -> bytes:
        """Расшифровать данные. При неверном ключе будет исключение
    (ValueError)."""
        nonce = token[:NONCE_LENGTH]
        tag = token[NONCE_LENGTH:NONCE_LENGTH + TAG_LENGTH]
        ciphertext = token[NONCE_LENGTH + TAG_LENGTH:]
        cipher = AES.new(self._key, AES.MODE_GCM, nonce=nonce)
        return cipher.decrypt_and_verify(ciphertext, tag)

```

Листинг B.8 — `wallet/core/vault.py`

"""Хранилище зашифрованной БД, защищённое паролем пользователя.

Каталог данных содержит два файла:

`wallet.salt` — соль для PBKDF2 (не является секретом, хранится открыто);

`wallet.db.enc` — зашифрованная (AES-256-GCM) база данных.

Ключ шифрования выводится из пароля и соли. Соль хранится отдельно, чтобы её можно было прочитать до ввода пароля; сами данные без верного пароля расшифровать невозможно (при неверном пароле проверка целостности GCM завершается ошибкой).

"""

```

from __future__ import annotations

import os
from pathlib import Path

from wallet.core.db import Database
from wallet.core.security import EncryptionManager, derive_key,
generate_salt

SALT_FILE = "wallet.salt"
DB_FILE = "wallet.db.enc"

class InvalidPasswordError(Exception):
    """Введён неверный пароль – расшифровать базу данных не удалось."""

def _salt_path(data_dir: Path) -> Path:
    return data_dir / SALT_FILE

def _db_path(data_dir: Path) -> Path:
    return data_dir / DB_FILE

def vault_exists(data_dir: str | Path) -> bool:
    data_dir = Path(data_dir)
    return _salt_path(data_dir).exists() and _db_path(data_dir).exists()

def init_vault(data_dir: str | Path, password: str) -> Database:
    """Создать новое хранилище с заданным паролем."""
    data_dir = Path(data_dir)
    data_dir.mkdir(parents=True, exist_ok=True)
    if vault_exists(data_dir):
        raise FileExistsError("Хранилище уже существует в данном каталоге")

    salt = generate_salt()
    _salt_path(data_dir).write_bytes(salt)
    key = derive_key(password, salt)

    db = Database(path=_db_path(data_dir),
encryption=EncryptionManager(key))
    db.connect()
    db.save()
    return db

def change_password(db: Database, new_password: str) -> None:

```

"""Сменить пароль открытого хранилища: новая соль, ключ и перешифрование.

Порядок операций обеспечивает устойчивость к сбоям: сначала база атомарно перешифровывается новым ключом (db.save), и только затем атомарно обновляется файл соли. Обе записи выполняются через временный файл с os.replace, поэтому окно несогласованности сведено к минимуму.

```
if db.path is None:
    raise ValueError("Хранилище не связано с файлом")
if not new_password:
    raise ValueError("Пароль не может быть пустым")
data_dir = db.path.parent
salt = generate_salt()

# 1) перешифровать и атомарно сохранить БД новым ключом
db.encryption = EncryptionManager(derive_key(new_password, salt))
db.save()

# 2) затем атомарно заменить файл соли (новая соль соответствует
новому ключу)
salt_path = _salt_path(data_dir)
salt_tmp = salt_path.with_name(salt_path.name + ".tmp")
salt_tmp.write_bytes(salt)
os.replace(salt_tmp, salt_path)

def open_vault(data_dir: str | Path, password: str) -> Database:
    """Открыть существующее хранилище. При неверном пароле – ошибка."""
    data_dir = Path(data_dir)
    if not vault_exists(data_dir):
        raise FileNotFoundError("Хранилище не найдено")

    salt = _salt_path(data_dir).read_bytes()
    key = derive_key(password, salt)
    db = Database(path=_db_path(data_dir),
encryption=EncryptionManager(key))
    try:
        db.connect()
    except Exception as exc: # поqa: BLE001 - неверный ключ -> ошибка
расшифровки
        db.close()
        raise InvalidPasswordError("Неверный пароль") from exc
    return db
```

Листинг B.9 — wallet/models/init.py

"""Модели предметной области."""

```
from wallet.models.entities import (
```

```

    Account,
    Category,
    CategoryKind,
    Goal,
    Reminder,
    Transaction,
    TransactionType,
    User,
)

__all__ = [
    "Account",
    "Category",
    "CategoryKind",
    "Goal",
    "Reminder",
    "Transaction",
    "TransactionType",
    "User",
]

```

Листинг B.10 — *wallet/models/entities.py*

"""Модели (сущности) предметной области «персональные финансы».

Денежные суммы хранятся в виде Decimal во избежание ошибок округления, характерных для чисел с плавающей точкой.

```

from __future__ import annotations

from dataclasses import dataclass, field
from datetime import date, datetime
from decimal import Decimal
from enum import Enum
from typing import Optional

class TransactionType(str, Enum):
    """Тип транзакции."""

    INCOME = "income"
    EXPENSE = "expense"

class CategoryKind(str, Enum):
    """Принадлежность категории к доходам или расходам."""

    INCOME = "income"
    EXPENSE = "expense"

```

```

@dataclass
class User:
    """Пользователь приложения."""

    name: str
    password_hash: str
    salt: str
    id: Optional[int] = None
    created_at: datetime = field(default_factory=datetime.now)

@dataclass
class Account:
    """Счёт пользователя (наличные, карта, электронный кошелёк)."""

    name: str
    type: str = "cash"
    balance: Decimal = Decimal("0")
    currency: str = "RUB"
    user_id: Optional[int] = None
    id: Optional[int] = None

@dataclass
class Category:
    """Категория операции."""

    name: str
    kind: CategoryKind = CategoryKind.EXPENSE
    icon: str = ""
    is_predefined: bool = False
    user_id: Optional[int] = None
    id: Optional[int] = None

@dataclass
class Transaction:
    """Финансовая операция (доход или расход)."""

    amount: Decimal
    type: TransactionType
    account_id: int
    category_id: Optional[int] = None
    note: str = ""
    created_at: datetime = field(default_factory=datetime.now)
    id: Optional[int] = None

@dataclass
class Goal:

```

```

"""Финансовая цель пользователя."""

title: str
target_amount: Decimal
current_amount: Decimal = Decimal("0")
deadline: Optional[date] = None
user_id: Optional[int] = None
id: Optional[int] = None

@property
def progress(self) -> float:
    """Доля достижения цели в диапазоне [0.0, 1.0]."""
    if self.target_amount <= 0:
        return 0.0
    return min(float(self.current_amount / self.target_amount), 1.0)

@property
def is_reached(self) -> bool:
    return self.current_amount >= self.target_amount

@dataclass
class Reminder:
    """Напоминание о предстоящем платеже."""

    title: str
    amount: Decimal
    due_date: date
    category_id: Optional[int] = None
    is_repeating: bool = False
    period: str = ""
    id: Optional[int] = None

```

Листинг В.11 — wallet/repositories/init.py

```

"""Слой доступа к данным (паттерн Repository)."""

from wallet.repositories.account_repository import AccountRepository
from wallet.repositories.category_repository import CategoryRepository
from wallet.repositories.goal_repository import GoalRepository
from wallet.repositories.reminder_repository import ReminderRepository
from wallet.repositories.transaction_repository import TransactionRepository
from wallet.repositories.user_repository import UserRepository

__all__ = [
    "AccountRepository",
    "CategoryRepository",
    "GoalRepository",
    "ReminderRepository",
    "TransactionRepository",

```

```

        "UserRepository",
    ]

Листинг В.12 — wallet/repositories/account_repository.py

"""Репозиторий счетов."""

from __future__ import annotations

import sqlite3
from decimal import Decimal

from wallet.models import Account
from wallet.repositories.base import BaseRepository

class AccountRepository(BaseRepository[Account]):
    table = "accounts"

    def _row_to_entity(self, row: sqlite3.Row) -> Account:
        return Account(
            id=row["id"],
            user_id=row["user_id"],
            name=row["name"],
            type=row["type"],
            balance=Decimal(row["balance"]),
            currency=row["currency"],
        )

    def add(self, entity: Account) -> Account:
        cur = self.conn.execute(
            "INSERT INTO accounts (user_id, name, type, balance,
currency) "
            "VALUES (?, ?, ?, ?, ?)",
            (
                entity.user_id,
                entity.name,
                entity.type,
                str(entity.balance),
                entity.currency,
            ),
        )
        self.conn.commit()
        entity.id = cur.lastrowid
        return entity

    def update(self, entity: Account) -> None:
        self.conn.execute(
            "UPDATE accounts SET name = ?, type = ?, balance = ?,
currency = ? "
            "WHERE id = ?",

```



```

        (entity.name, entity.type, str(entity.balance),
entity.currency, entity.id),
    )
    self.conn.commit()

```

Листинг B.13 — wallet/repositories/base.py

```

"""Базовый репозиторий и интерфейс доступа к данным."""

```

```

from __future__ import annotations

```

```

import sqlite3
from abc import ABC, abstractmethod
from typing import Generic, Optional, TypeVar

```

```

T = TypeVar("T")

```

```

class IRepository(ABC, Generic[T]):
    """Интерфейс репозитория: единый контракт CRUD-операций."""

```

```

    @abstractmethod
    def add(self, entity: T) -> T: ...

```

```

    @abstractmethod
    def get(self, entity_id: int) -> Optional[T]: ...

```

```

    @abstractmethod
    def list(self) -> list[T]: ...

```

```

    @abstractmethod
    def update(self, entity: T) -> None: ...

```

```

    @abstractmethod
    def delete(self, entity_id: int) -> None: ...

```

```

class BaseRepository(IRepository[T]):
    """Общая реализация для работы с одной таблицей SQLite."""

```

```

    table: str = ""

```

```

    def __init__(self, conn: sqlite3.Connection):
        self.conn = conn

```

```

    def _row_to_entity(self, row: sqlite3.Row) -> T: # pragma: no cover
        raise NotImplementedError

```

```

    # Примечание по безопасности: имя таблицы (self.table) – внутренняя
    # константа класса-репозитория, не пользовательский ввод. Все
    значения,

```

*# приходящие извне, передаются параметрами запроса (?). Поэтому SQL-
инъекция невозможна, предупреждения B608 – ложные срабатывания.*

```
def get(self, entity_id: int) -> Optional[T]:
    cur = self.conn.execute(
        f"SELECT * FROM {self.table} WHERE id = ?", (entity_id,) #
nosed B608
    )
    row = cur.fetchone()
    return self._row_to_entity(row) if row else None

def list(self) -> list[T]:
    cur = self.conn.execute(
        f"SELECT * FROM {self.table} ORDER BY id") # nosed B608
    return [self._row_to_entity(r) for r in cur.fetchall()]

def delete(self, entity_id: int) -> None:
    self.conn.execute(
        f"DELETE FROM {self.table} WHERE id = ?", (entity_id,) #
nosed B608
    self.conn.commit()
```

Листинг B.14 — wallet/repositories/category_repository.py

"""Репозиторий категорий."""

```
from __future__ import annotations

import sqlite3

from wallet.models import Category, CategoryKind
from wallet.repositories.base import BaseRepository

class CategoryRepository(BaseRepository[Category]):
    table = "categories"

    def _row_to_entity(self, row: sqlite3.Row) -> Category:
        return Category(
            id=row["id"],
            user_id=row["user_id"],
            name=row["name"],
            kind=CategoryKind(row["kind"]),
            icon=row["icon"] or "",
            is_predefined=bool(row["is_predefined"]),
        )

    def add(self, entity: Category) -> Category:
        cur = self.conn.execute(
            "INSERT INTO categories (user_id, name, kind, icon,
is_predefined) "
```

```

        "VALUES (?, ?, ?, ?, ?)",
        (
            entity.user_id,
            entity.name,
            entity.kind.value,
            entity.icon,
            int(entity.is_predefined),
        ),
    )
    self.conn.commit()
    entity.id = cur.lastrowid
    return entity

def update(self, entity: Category) -> None:
    self.conn.execute(
        "UPDATE categories SET name = ?, kind = ?, icon = ? WHERE id
= ?",
        (entity.name, entity.kind.value, entity.icon, entity.id),
    )
    self.conn.commit()

```

Листинг B.15 — wallet/repositories/goal_repository.py

"""Репозиторий финансовых целей."""

```

from __future__ import annotations

import sqlite3
from datetime import date
from decimal import Decimal

from wallet.models import Goal
from wallet.repositories.base import BaseRepository

class GoalRepository(BaseRepository[Goal]):
    table = "goals"

    def _row_to_entity(self, row: sqlite3.Row) -> Goal:
        return Goal(
            id=row["id"],
            user_id=row["user_id"],
            title=row["title"],
            target_amount=Decimal(row["target_amount"]),
            current_amount=Decimal(row["current_amount"]),
            deadline=date.fromisoformat(row["deadline"]) if
row["deadline"] else None,
        )

    def add(self, entity: Goal) -> Goal:
        cur = self.conn.execute(

```

```

        "INSERT INTO goals "
        "(user_id, title, target_amount, current_amount, deadline) "
        "VALUES (?, ?, ?, ?, ?)",
        (
            entity.user_id,
            entity.title,
            str(entity.target_amount),
            str(entity.current_amount),
            entity.deadline.isoformat() if entity.deadline else None,
        ),
    )
    self.conn.commit()
    entity.id = cur.lastrowid
    return entity

    def update(self, entity: Goal) -> None:
        self.conn.execute(
            "UPDATE goals SET title = ?, target_amount = ?,
current_amount = ?, "
            "deadline = ? WHERE id = ?",
            (
                entity.title,
                str(entity.target_amount),
                str(entity.current_amount),
                entity.deadline.isoformat() if entity.deadline else None,
                entity.id,
            ),
        )
        self.conn.commit()

```

Листинг В.16 — wallet/repositories/reminder_repository.py

```

"""Репозиторий напоминаний о платежах."""

from __future__ import annotations

import sqlite3
from datetime import date
from decimal import Decimal

from wallet.models import Reminder
from wallet.repositories.base import BaseRepository

class ReminderRepository(BaseRepository[Reminder]):
    table = "reminders"

    def _row_to_entity(self, row: sqlite3.Row) -> Reminder:
        return Reminder(
            id=row["id"],
            category_id=row["category_id"],

```

```

        title=row["title"],
        amount=Decimal(row["amount"]),
        due_date=date.fromisoformat(row["due_date"]),
        is_repeating=bool(row["is_repeating"]),
        period=row["period"] or "",
    )

    def add(self, entity: Reminder) -> Reminder:
        cur = self.conn.execute(
            "INSERT INTO reminders "
            "(category_id, title, amount, due_date, is_repeating, period)
            "VALUES (?, ?, ?, ?, ?, ?)",
            (
                entity.category_id,
                entity.title,
                str(entity.amount),
                entity.due_date.isoformat(),
                int(entity.is_repeating),
                entity.period,
            ),
        )
        self.conn.commit()
        entity.id = cur.lastrowid
        return entity

    def update(self, entity: Reminder) -> None:
        self.conn.execute(
            "UPDATE reminders SET title = ?, amount = ?, due_date = ?, "
            "is_repeating = ?, period = ? WHERE id = ?",
            (
                entity.title,
                str(entity.amount),
                entity.due_date.isoformat(),
                int(entity.is_repeating),
                entity.period,
                entity.id,
            ),
        )
        self.conn.commit()

    def list_upcoming(self, until: date) -> list[Reminder]:
        """Напоминания со сроком не позднее указанной даты."""
        cur = self.conn.execute(
            "SELECT * FROM reminders WHERE due_date <= ? ORDER BY
due_date",
            (until.isoformat(),),
        )
        return [self._row_to_entity(r) for r in cur.fetchall()]

```

```
"""Репозиторий транзакций."""
```

```
from __future__ import annotations
```

```
import sqlite3
```

```
from datetime import datetime
```

```
from decimal import Decimal
```

```
from typing import Optional
```

```
from wallet.models import Transaction, TransactionType
```

```
from wallet.repositories.base import BaseRepository
```

```
class TransactionRepository(BaseRepository[Transaction]):
```

```
    table = "transactions"
```

```
    def _row_to_entity(self, row: sqlite3.Row) -> Transaction:
```

```
        return Transaction(
```

```
            id=row["id"],
```

```
            account_id=row["account_id"],
```

```
            category_id=row["category_id"],
```

```
            amount=Decimal(row["amount"]),
```

```
            type=TransactionType(row["type"]),
```

```
            note=row["note"] or "",
```

```
            created_at=datetime.fromisoformat(row["created_at"]),
```

```
        )
```

```
    def add(self, entity: Transaction) -> Transaction:
```

```
        cur = self.conn.execute(
```

```
            "INSERT INTO transactions "
```

```
            "(account_id, category_id, amount, type, note, created_at) "
```

```
            "VALUES (?, ?, ?, ?, ?, ?)",
```

```
            (
```

```
                entity.account_id,
```

```
                entity.category_id,
```

```
                str(entity.amount),
```

```
                entity.type.value,
```

```
                entity.note,
```

```
                entity.created_at.isoformat(),
```

```
            ),
```

```
        )
```

```
        self.conn.commit()
```

```
        entity.id = cur.lastrowid
```

```
        return entity
```

```
    def update(self, entity: Transaction) -> None:
```

```
        self.conn.execute(
```

```
            "UPDATE transactions SET account_id = ?, category_id = ?,
```

```
amount = ?, "
```

```
            "type = ?, note = ? WHERE id = ?",
```

```
            (
```

```

        entity.account_id,
        entity.category_id,
        str(entity.amount),
        entity.type.value,
        entity.note,
        entity.id,
    ),
)
self.conn.commit()

def list_filtered(
    self,
    account_id: Optional[int] = None,
    category_id: Optional[int] = None,
    tx_type: Optional[TransactionType] = None,
    date_from: Optional[datetime] = None,
    date_to: Optional[datetime] = None,
) -> list[Transaction]:
    """Вернуть транзакции с фильтрацией по счёту, категории, типу,
    периоду."""
    clauses: list[str] = []
    params: list = []
    if account_id is not None:
        clauses.append("account_id = ?")
        params.append(account_id)
    if category_id is not None:
        clauses.append("category_id = ?")
        params.append(category_id)
    if tx_type is not None:
        clauses.append("type = ?")
        params.append(tx_type.value)
    if date_from is not None:
        clauses.append("created_at >= ?")
        params.append(date_from.isoformat())
    if date_to is not None:
        clauses.append("created_at <= ?")
        params.append(date_to.isoformat())

    # where собирается только из внутренних имён столбцов;
    # значения передаются параметрами (params) -> инъекция
    # невозможна.
    where = f" WHERE {' AND '.join(clauses)}" if clauses else ""
    cur = self.conn.execute(
        f"SELECT * FROM transactions{where} ORDER BY created_at
DESC", # nsec B608
        params,
    )
    return [self._row_to_entity(r) for r in cur.fetchall()]

```

Листинг B.18 — wallet/repositories/user_repository.py

```
"""Репозиторий пользователей."""
```

```
from __future__ import annotations
```

```
import sqlite3
```

```
from datetime import datetime
```

```
from typing import Optional
```

```
from wallet.models import User
```

```
from wallet.repositories.base import BaseRepository
```

```
class UserRepository(BaseRepository[User]):
```

```
    table = "users"
```

```
    def _row_to_entity(self, row: sqlite3.Row) -> User:
```

```
        return User(
            id=row["id"],
            name=row["name"],
            password_hash=row["password_hash"],
            salt=row["salt"],
            created_at=datetime.fromisoformat(row["created_at"]),
        )
```

```
    def add(self, entity: User) -> User:
```

```
        cur = self.conn.execute(
            "INSERT INTO users (name, password_hash, salt, created_at) "
            "VALUES (?, ?, ?, ?)",
            (
                entity.name,
                entity.password_hash,
                entity.salt,
                entity.created_at.isoformat(),
            ),
        )
        self.conn.commit()
        entity.id = cur.lastrowid
        return entity
```

```
    def update(self, entity: User) -> None:
```

```
        self.conn.execute(
            "UPDATE users SET name = ?, password_hash = ?, salt = ? WHERE "
            "id = ?",
            (entity.name, entity.password_hash, entity.salt, entity.id),
        )
        self.conn.commit()
```

```
    def get_by_name(self, name: str) -> Optional[User]:
```

```
        cur = self.conn.execute("SELECT * FROM users WHERE name = ?",
                                  (name,))
```



```

        row = cur.fetchone()
        return self._row_to_entity(row) if row else None

```

Листинг B.19 — wallet/services/init.py

```

"""Слой бизнес-логики (сервисы)."""

from wallet.services.account_service import AccountService
from wallet.services.analytics_service import AnalyticsService
from wallet.services.auth_service import AuthService
from wallet.services.category_service import CategoryService
from wallet.services.chart_service import ChartService
from wallet.services.goal_service import GoalService
from wallet.services.reminder_service import ReminderService
from wallet.services.transaction_service import TransactionService

__all__ = [
    "AccountService",
    "AnalyticsService",
    "AuthService",
    "CategoryService",
    "ChartService",
    "GoalService",
    "ReminderService",
    "TransactionService",
]

```

Листинг B.20 — wallet/services/account_service.py

```

"""Сервис управления счетами."""

from __future__ import annotations

from decimal import Decimal
from typing import Optional

from wallet.models import Account
from wallet.repositories import AccountRepository

class AccountService:
    """Создание счетов и получение сводной информации по балансу."""

    def __init__(self, accounts: AccountRepository):
        self.accounts = accounts

    def create(
        self,
        name: str,
        type: str = "cash",
        currency: str = "RUB",
        initial_balance: Decimal = Decimal("0"),

```

```

        user_id: Optional[int] = None,
    ) -> Account:
        if not name:
            raise ValueError("Название счёта не может быть пустым")
        return self.accounts.add(
            Account(
                name=name,
                type=type,
                balance=initial_balance,
                currency=currency,
                user_id=user_id,
            )
        )

    def list(self) -> list[Account]:
        return self.accounts.list()

    def total_balance(self) -> Decimal:
        """Суммарный баланс по всем счетам."""
        return sum((a.balance for a in self.accounts.list()),
Decimal("0"))

```

Листинг B.21 — wallet/services/analytics_service.py

```

"""Сервис аналитики и подготовки данных для визуализации."""

from __future__ import annotations

from datetime import datetime
from decimal import Decimal
from typing import Optional

from wallet.models import TransactionType
from wallet.repositories import CategoryRepository, TransactionRepository

class AnalyticsService:
    """Агрегация финансовых данных для диаграмм и сводок."""

    def __init__(
        self,
        transactions: TransactionRepository,
        categories: CategoryRepository,
    ):
        self.transactions = transactions
        self.categories = categories

    def expenses_by_category(
        self,
        date_from: Optional[datetime] = None,
        date_to: Optional[datetime] = None,

```

```

    ) -> dict[str, Decimal]:
        """Суммы расходов, сгруппированные по названию категории."""
        cat_names = {c.id: c.name for c in self.categories.list()}
        result: dict[str, Decimal] = {}
        txs = self.transactions.list_filtered(
            tx_type=TransactionType.EXPENSE, date_from=date_from,
date_to=date_to
        )
        for tx in txs:
            name = cat_names.get(tx.category_id, "Без категории")
            result[name] = result.get(name, Decimal("0")) + tx.amount
        return result

    def income_vs_expense(
        self,
        date_from: Optional[datetime] = None,
        date_to: Optional[datetime] = None,
    ) -> dict[str, Decimal]:
        """Итоговые суммы доходов и расходов за период."""
        income = sum(
            (t.amount for t in self.transactions.list_filtered(
                tx_type=TransactionType.INCOME, date_from=date_from,
date_to=date_to)),
            Decimal("0"),
        )
        expense = sum(
            (t.amount for t in self.transactions.list_filtered(
                tx_type=TransactionType.EXPENSE, date_from=date_from,
date_to=date_to)),
            Decimal("0"),
        )
        return {"income": income, "expense": expense, "balance": income -
expense}

    def monthly_dynamics(
        self,
        date_from: Optional[datetime] = None,
        date_to: Optional[datetime] = None,
    ) -> list[tuple[str, Decimal, Decimal]]:
        """Динамика доходов и расходов по месяцам за указанный период.

        Возвращает список кортежей (метка_месяца, доходы, расходы) по
всем
месяцам с операциями в диапазоне, в хронологическом порядке.
"""
        buckets: dict[tuple[int, int], dict[str, Decimal]] = {}
        for tx in self.transactions.list_filtered(
            date_from=date_from, date_to=date_to
        ):
            key = (tx.created_at.year, tx.created_at.month)
            bucket = buckets.setdefault(

```

```

        key, {"income": Decimal("0"), "expense": Decimal("0")}
    )
    if tx.type == TransactionType.INCOME:
        bucket["income"] += tx.amount
    else:
        bucket["expense"] += tx.amount

    result: list[tuple[str, Decimal, Decimal]] = []
    for year, month in sorted(buckets):
        data = buckets[(year, month)]
        result.append((f"{month:02d}.{year}", data["income"],
data["expense"])))
    return result

```

Листинг B.22 — wallet/services/auth_service.py

```

"""Сервис аутентификации и регистрации пользователя."""

from __future__ import annotations

from typing import Optional

from wallet.core import security
from wallet.models import User
from wallet.repositories import UserRepository

class AuthService:
    """Регистрация и вход по паролю.

    Пароль не хранится в открытом виде – сохраняются его хеш (PBKDF2) и
    соль.
    """

    def __init__(self, users: UserRepository):
        self.users = users

    def register(self, name: str, password: str) -> User:
        if not name or not password:
            raise ValueError("Имя и пароль не могут быть пустыми")
        if self.users.get_by_name(name) is not None:
            raise ValueError("Пользователь с таким именем уже
существует")
        password_hash, salt = security.hash_password(password)
        user = User(name=name, password_hash=password_hash, salt=salt)
        return self.users.add(user)

    def login(self, name: str, password: str) -> Optional[User]:
        """Вернуть пользователя при верном пароле, иначе None."""
        user = self.users.get_by_name(name)
        if user is None:

```

```

        return None
    if security.verify_password(password, user.password_hash,
user.salt):
        return user
    return None

```

Листинг В.23 — *wallet/services/category_service.py*

```

"""Сервис управления категориями."""

from __future__ import annotations

from wallet.models import Category, CategoryKind
from wallet.repositories import CategoryRepository

DEFAULT_CATEGORIES: list[tuple[str, CategoryKind]] = [
    ("Продукты", CategoryKind.EXPENSE),
    ("Транспорт", CategoryKind.EXPENSE),
    ("Жильё", CategoryKind.EXPENSE),
    ("Развлечения", CategoryKind.EXPENSE),
    ("Здоровье", CategoryKind.EXPENSE),
    ("Зарплата", CategoryKind.INCOME),
    ("Прочие доходы", CategoryKind.INCOME),
]

class CategoryService:
    """Создание пользовательских и предопределённых категорий."""

    def __init__(self, categories: CategoryRepository):
        self.categories = categories

    def add(self, name: str, kind: CategoryKind, icon: str = "") ->
Category:
        if not name:
            raise ValueError("Название категории не может быть пустым")
        return self.categories.add(Category(name=name, kind=kind,
icon=icon))

    def list(self) -> list[Category]:
        return self.categories.list()

    def delete(self, category_id: int) -> None:
        """Удалить категорию, обнулив ссылки на неё у операций и
платежей."""
        conn = self.categories.conn
        conn.execute(
            "UPDATE transactions SET category_id = NULL WHERE category_id
= ?",
            (category_id,),
        )

```

```

        conn.execute(
            "UPDATE reminders SET category_id = NULL WHERE category_id =
?",
            (category_id,),
        )
        self.categories.delete(category_id)

    def ensure_defaults(self) -> list[Category]:
        """Создать predefined категории, если их ещё нет."""
        existing = {c.name for c in self.categories.list()}
        created: list[Category] = []
        for name, kind in DEFAULT_CATEGORIES:
            if name not in existing:
                created.append(
                    self.categories.add(
                        Category(name=name, kind=kind,
is_predefined=True)
                    )
                )
        return created

```

Листинг B.24 — wallet/services/chart_service.py

"""Построение диаграмм для визуализации финансовых данных.

Используется неинтерактивный backend matplotlib (Agg), что позволяет сохранять диаграммы в файлы изображений без графической оболочки. В UI полученные PNG отображаются как картинки.

```

from __future__ import annotations

from datetime import datetime
from pathlib import Path
from typing import Optional

from wallet.services.analytics_service import AnalyticsService

class ChartService:
    """Готовит изображения диаграмм на основе данных AnalyticsService.

    matplotlib импортируется по требованию: библиотека тяжёлая и не
    всегда доступна на мобильной платформе, поэтому её отсутствие не должно
    мешать запуску приложения. Если matplotlib не установлен, методы построения
    диаграмм возбуждают ImportError, который обрабатывается слоем
    интерфейса.
    """

```

```

def __init__(self, analytics: AnalyticsService):
    self.analytics = analytics

@staticmethod
def _pyplot():
    import matplotlib

    matplotlib.use("Agg") # backend без графической оболочки
    import matplotlib.pyplot as plt

    return plt

def expenses_pie(
    self,
    path: str | Path,
    date_from: Optional[datetime] = None,
    date_to: Optional[datetime] = None,
) -> Path:
    """Круговая диаграмма структуры расходов по категориям."""
    plt = self._pyplot()
    data = self.analytics.expenses_by_category(date_from, date_to)
    fig, ax = plt.subplots()
    if data:
        ax.pie(
            [float(v) for v in data.values()],
            labels=list(data.keys()),
            autopct="%1.1f%%",
        )
    else:
        ax.text(0.5, 0.5, "Нет данных", ha="center", va="center")
    ax.set_title("Структура расходов по категориям")
    fig.savefig(path)
    plt.close(fig)
    return Path(path)

def income_expense_bar(
    self,
    path: str | Path,
    date_from: Optional[datetime] = None,
    date_to: Optional[datetime] = None,
) -> Path:
    """Столбчатая диаграмма соотношения доходов и расходов."""
    plt = self._pyplot()
    totals = self.analytics.income_vs_expense(date_from, date_to)
    fig, ax = plt.subplots()
    ax.bar(
        ["Доходы", "Расходы"],
        [float(totals["income"]), float(totals["expense"])],
        color=["#4caf50", "#e53935"],
    )
    ax.set_ylabel("Сумма")

```

```

        ax.set_title("Доходы и расходы")
        fig.savefig(path)
        plt.close(fig)
        return Path(path)

    def dynamics_chart(
        self,
        path: str | Path,
        date_from: Optional[datetime] = None,
        date_to: Optional[datetime] = None,
    ) -> Path:
        """Диаграмма динамики доходов и расходов по месяцам за период."""
        plt = self._pyplot()
        data = self.analytics.monthly_dynamics(date_from, date_to)
        labels = [d[0] for d in data]
        incomes = [float(d[1]) for d in data]
        expenses = [float(d[2]) for d in data]

        fig, ax = plt.subplots()
        if data:
            positions = range(len(labels))
            width = 0.4
            ax.bar([p - width / 2 for p in positions], incomes, width,
                    label="Доходы", color="#4caf50")
            ax.bar([p + width / 2 for p in positions], expenses, width,
                    label="Расходы", color="#e53935")
            ax.set_xticks(list(positions))
            ax.set_xticklabels(labels, rotation=45, ha="right")
            ax.legend()
        else:
            ax.text(0.5, 0.5, "Нет данных", ha="center", va="center")
        ax.set_title("Динамика по месяцам")
        fig.tight_layout()
        fig.savefig(path)
        plt.close(fig)
        return Path(path)

```

Листинг B.25 — wallet/services/goal_service.py

```

"""Сервис управления финансовыми целями."""

from __future__ import annotations

from datetime import date
from decimal import Decimal
from typing import Optional

from wallet.models import Goal
from wallet.repositories import GoalRepository

```



```

class GoalService:
    """Постановка целей и отслеживание прогресса их достижения."""

    def __init__(self, goals: GoalRepository):
        self.goals = goals

    def create(
        self,
        title: str,
        target_amount: Decimal,
        deadline: Optional[date] = None,
    ) -> Goal:
        if target_amount <= 0:
            raise ValueError("Целевая сумма должна быть положительной")
        return self.goals.add(
            Goal(title=title, target_amount=target_amount,
deadline=deadline)
        )

    def contribute(self, goal_id: int, amount: Decimal) -> Goal:
        """Пополнить накопления по цели."""
        goal = self.goals.get(goal_id)
        if goal is None:
            raise ValueError("Цель не найдена")
        if amount <= 0:
            raise ValueError("Сумма пополнения должна быть
положительной")
        goal.current_amount += amount
        self.goals.update(goal)
        return goal

    def update(
        self,
        goal_id: int,
        title: Optional[str] = None,
        target_amount: Optional[Decimal] = None,
    ) -> Goal:
        """Изменить название и/или целевую сумму цели."""
        goal = self.goals.get(goal_id)
        if goal is None:
            raise ValueError("Цель не найдена")
        if title is not None:
            if not title.strip():
                raise ValueError("Название цели не может быть пустым")
            goal.title = title.strip()
        if target_amount is not None:
            if target_amount <= 0:
                raise ValueError("Целевая сумма должна быть
положительной")
            goal.target_amount = target_amount
        self.goals.update(goal)

```

```

        return goal

    def delete(self, goal_id: int) -> None:
        self.goals.delete(goal_id)

    def get(self, goal_id: int) -> Optional[Goal]:
        return self.goals.get(goal_id)

    def list(self) -> list[Goal]:
        return self.goals.list()

```

Листинг B.26 — wallet/services/reminder_service.py

"""Сервис напоминаний о платежах."""

```

from __future__ import annotations

import calendar
from datetime import date, timedelta
from decimal import Decimal

from wallet.core.notifications import Notifier
from wallet.models import Reminder
from wallet.repositories import ReminderRepository

def _add_months(d: date, months: int) -> date:
    """Прибавить месяцы с корректировкой числа под длину месяца."""
    index = d.month - 1 + months
    year = d.year + index // 12
    month = index % 12 + 1
    day = min(d.day, calendar.monthrange(year, month)[1])
    return date(year, month, day)

def next_due_date(current: date, period: str) -> date:
    """Вычислить следующую дату платежа по периодичности."""
    if period == "daily":
        return current + timedelta(days=1)
    if period == "weekly":
        return current + timedelta(weeks=1)
    if period == "monthly":
        return _add_months(current, 1)
    if period == "yearly":
        return _add_months(current, 12)
    raise ValueError(f"Неизвестная периодичность: {period!r}")

class ReminderService:
    """Создание напоминаний, выборка предстоящих и рассылка уведомлений.

```

Доставка уведомлений делегируется объекту Notifier (plyer на устройстве, заглушка в средах без графической оболочки).

```
def __init__(self, reminders: ReminderRepository):
    self.reminders = reminders

def schedule(self, reminder: Reminder) -> Reminder:
    if reminder.amount <= 0:
        raise ValueError("Сумма напоминания должна быть
положительной")
    return self.reminders.add(reminder)

def upcoming(self, until: date | None = None) -> list[Reminder]:
    """Напоминания со сроком не позднее указанной даты (по умолчанию
сегодня)."""
    return self.reminders.list_upcoming(until or date.today())

def get(self, reminder_id: int) -> Reminder | None:
    return self.reminders.get(reminder_id)

def list(self) -> list[Reminder]:
    return self.reminders.list()

def advance(self, reminder_id: int) -> Reminder:
    """Перенести повторяющееся напоминание на следующий период."""
    reminder = self.reminders.get(reminder_id)
    if reminder is None:
        raise ValueError("Напоминание не найдено")
    if reminder.is_repeating and reminder.period:
        reminder.due_date = next_due_date(reminder.due_date,
reminder.period)
        self.reminders.update(reminder)
    return reminder

def update(
    self,
    reminder_id: int,
    title: str | None = None,
    amount: Decimal | None = None,
    due_date: date | None = None,
    period: str | None = None,
) -> Reminder:
    """Изменить название, сумму, срок и/или периодичность платежа.

    Если передан `period` (пустая строка – разовый платёж), признак
    повторяемости пересчитывается автоматически.
    """
    reminder = self.reminders.get(reminder_id)
    if reminder is None:
```

```

        raise ValueError("Напоминание не найдено")
    if title is not None:
        if not title.strip():
            raise ValueError("Название не может быть пустым")
        reminder.title = title.strip()
    if amount is not None:
        if amount <= 0:
            raise ValueError("Сумма должна быть положительной")
        reminder.amount = amount
    if due_date is not None:
        reminder.due_date = due_date
    if period is not None:
        reminder.period = period
        reminder.is_repeating = bool(period)
    self.reminders.update(reminder)
    return reminder

    def delete(self, reminder_id: int) -> None:
        self.reminders.delete(reminder_id)

    def notify_due(self, notifier: Notifier, until: date | None = None) -
    > int:
        """Разослать уведомления по наступившим напоминаниям. Возвращает
        их число."""
        due = self.upcoming(until)
        for reminder in due:
            notifier.notify(
                title=f"Платёж: {reminder.title}",
                message=f"Сумма {reminder.amount}, срок
{reminder.due_date}",
            )
        return len(due)

```

Листинг B.27 — wallet/services/transaction_service.py

```

"""Сервис учёта доходов и расходов."""

```

```

from __future__ import annotations

```

```

from datetime import datetime

```

```

from decimal import Decimal

```

```

from typing import Optional

```

```

from wallet.models import Transaction, TransactionType

```

```

from wallet.repositories import AccountRepository, TransactionRepository

```

```

# Маркер «значение не передано»: позволяет отличить «не менять поле» от
# «установить None» (например, снять категорию) в методе edit.

```

```

_UNSET = object()

```

```

class TransactionService:
    """Добавление, удаление и выборка транзакций с обновлением баланса
    счёта."""

    def __init__(
        self,
        transactions: TransactionRepository,
        accounts: AccountRepository,
    ):
        self.transactions = transactions
        self.accounts = accounts

    def get(self, transaction_id: int) -> Optional[Transaction]:
        return self.transactions.get(transaction_id)

    def add(self, transaction: Transaction) -> Transaction:
        if transaction.amount <= 0:
            raise ValueError("Сумма операции должна быть положительной")
        saved = self.transactions.add(transaction)
        self._apply_to_balance(saved, sign=1)
        return saved

    def delete(self, transaction_id: int) -> None:
        tx = self.transactions.get(transaction_id)
        if tx is None:
            return
        self._apply_to_balance(tx, sign=-1)
        self.transactions.delete(transaction_id)

    def edit(
        self,
        transaction_id: int,
        amount: Optional[Decimal] = None,
        category_id=_UNSET,
        note: Optional[str] = None,
        tx_type: Optional[TransactionType] = None,
    ) -> Transaction:
        """Изменить транзакцию с корректным пересчётом баланса счёта.

        Для category_id используется маркер _UNSET: если аргумент не
        передан –
        категория не меняется; передача None – снимает категорию.
        """
        tx = self.transactions.get(transaction_id)
        if tx is None:
            raise ValueError("Транзакция не найдена")

        self._apply_to_balance(tx, sign=-1) # откатить прежнее влияние
        if amount is not None:
            if amount <= 0:
                raise ValueError("Сумма операции должна быть

```

```

положительной")
    tx.amount = amount
    if category_id is not _UNSET:
        tx.category_id = category_id
    if note is not None:
        tx.note = note
    if tx_type is not None:
        tx.type = tx_type

    self.transactions.update(tx)
    self._apply_to_balance(tx, sign=1) # применить новое влияние
    return tx

def list(
    self,
    account_id: Optional[int] = None,
    category_id: Optional[int] = None,
    tx_type: Optional[TransactionType] = None,
    date_from: Optional[datetime] = None,
    date_to: Optional[datetime] = None,
) -> list[Transaction]:
    return self.transactions.list_filtered(
        account_id=account_id,
        category_id=category_id,
        tx_type=tx_type,
        date_from=date_from,
        date_to=date_to,
    )

def _apply_to_balance(self, tx: Transaction, sign: int) -> None:
    """Изменить баланс счёта: доход увеличивает, расход уменьшает."""
    account = self.accounts.get(tx.account_id)
    if account is None:
        return
    delta = tx.amount if tx.type == TransactionType.INCOME else -
tx.amount
    account.balance += Decimal(sign) * delta
    self.accounts.update(account)

```

Листинг B.28 — *wallet/ui/init.py*

"""Слой представления (Kivy)."""

Листинг B.29 — *wallet/ui/app.py*

"""Пользовательский интерфейс на KivyMD (Material Design).

Экран входа открывает зашифрованное хранилище по паролю, далее – нижняя панель вкладок: Главная, Операции, Платежи, Цели, Аналитика.

KivyMD/Kivy импортируются «мягко»: если фреймворк не установлен (например,

в среде запуска тестов), модуль остаётся импортируемым, а попытка запустить GUI сообщает о необходимости установить зависимости.

"""

```
from __future__ import annotations

from datetime import date, datetime
from decimal import Decimal, InvalidOperation
from pathlib import Path

from wallet.app_context import AppContext
from wallet.core import vault
from wallet.models import CategoryKind, Reminder, Transaction,
TransactionType

NO_CATEGORY = "(без категории)"
PERIOD_MAP = {
    "Ежемесячно": "monthly",
    "Еженедельно": "weekly",
    "Ежедневно": "daily",
    "Ежегодно": "yearly",
}
PERIOD_LABELS = {v: k for k, v in PERIOD_MAP.items()}

try: # pragma: no cover - наличие GUI зависит от среды
    from kivy.lang import Builder
    from kivy.metrics import dp
    from kivy.ui.image import Image # noqa: F401 (используется в KV)
    from kivy.ui.spinner import Spinner
    from kivymd.app import MDApp
    from kivymd.ui.boxlayout import MDBoxLayout
    from kivymd.ui.button import MDFlatButton, MDRaisedButton
    from kivymd.ui.dialog import MDDialog
    from kivymd.ui.label import MDLabel
    from kivymd.ui.list import ThreeLineListItem, TwoLineListItem
    from kivymd.ui.progressbar import MDProgressBar
    from kivymd.ui.screen import MDScreen
    from kivymd.ui.textfield import MDTextField

    KIVYMD_AVAILABLE = True
except Exception: # noqa: BLE001
    KIVYMD_AVAILABLE = False

if KIVYMD_AVAILABLE:

    KV = """
ScreenManager:
    LoginScreen:
        name: 'login'
    """
```

```

MainScreen:
    name: 'main'

<LoginScreen>:
    MDBoxLayout:
        orientation: 'vertical'
        padding: dp(24)
        spacing: dp(12)
        Widget:
            size_hint_y: None
            height: dp(40)
        MDLabel:
            text: 'Wallet'
            halign: 'center'
            font_style: 'H4'
            size_hint_y: None
            height: dp(60)
        MDLabel:
            text: 'Учёт личных финансов'
            halign: 'center'
            theme_text_color: 'Secondary'
            size_hint_y: None
            height: dp(30)
        MDTextField:
            id: password
            hint_text: 'Пароль'
            password: True
        MDRaisedButton:
            text: 'Войти'
            pos_hint: {'center_x': 0.5}
            on_release: app.login()
        MDFlatButton:
            text: 'Создать хранилище'
            pos_hint: {'center_x': 0.5}
            on_release: app.register()
        MDLabel:
            id: status
            text: ''
            halign: 'center'
            theme_text_color: 'Error'
            size_hint_y: None
            height: dp(30)
        Widget:

<MainScreen>:
    MDBoxLayout:
        orientation: 'vertical'
    MDTopAppBar:
        title: 'Wallet'
        left_action_items: [['cog', lambda x: app.open_settings()]]
        right_action_items: [['logout', lambda x: app.logout()]]

```



```

MDBottomNavigation:
    id: nav

MDBottomNavigationItem:
    name: 'dashboard'
    text: 'Главная'
    icon: 'home'
    MDBoxLayout:
        orientation: 'vertical'
        padding: dp(16)
        spacing: dp(8)
        MDLabel:
            id: balance
            text: 'Баланс: –'
            font_style: 'H5'
            size_hint_y: None
            height: dp(50)
        MDLabel:
            text: 'Последние операции'
            theme_text_color: 'Secondary'
            size_hint_y: None
            height: dp(28)
        ScrollView:
            MDList:
                id: recent_list

MDBottomNavigationItem:
    name: 'tx'
    text: 'Операции'
    icon: 'format-list-bulleted'
    ScrollView:
        MDBoxLayout:
            orientation: 'vertical'
            adaptive_height: True
            padding: dp(12)
            spacing: dp(6)
            MDTextField:
                id: tx_amount
                hint_text: 'Сумма'
                input_filter: 'float'
                size_hint_y: None
                height: dp(48)
            MDBoxLayout:
                size_hint_y: None
                height: dp(48)
                spacing: dp(8)
                Spinner:
                    id: tx_type
                    text: 'Расход'
                    values: ['Расход', 'Доход']
            Spinner:

```

```

        id: tx_category
        text: 'Категория'
MDBoxLayout:
    size_hint_y: None
    height: dp(48)
    spacing: dp(8)
    MDFlatButton:
        text: 'Добавить категорию'
        on_release: app.open_add_category()
    MDFlatButton:
        text: 'Удалить категорию'
        theme_text_color: 'Custom'
        text_color: 0.8, 0.2, 0.2, 1
        on_release: app.open_delete_category()
MDBoxLayout:
    size_hint_y: None
    height: dp(48)
    spacing: dp(8)
    MDFlatButton:
        id: tx_date_btn
        text: 'Дата: сегодня'
        on_release: app.open_tx_date_picker()
    MDTextField:
        id: tx_note
        hint_text: 'Комментарий'
MDRaisedButton:
    text: 'Добавить операцию'
    pos_hint: {'center_x': 0.5}
    on_release: app.add_transaction()
MDLabel:
    text: 'История операций'
    theme_text_color: 'Secondary'
    size_hint_y: None
    height: dp(26)
MDBoxLayout:
    size_hint_y: None
    height: dp(48)
    spacing: dp(8)
    Spinner:
        id: flt_type
        text: 'Все'
        values: ['Все', 'Доход', 'Расход']
        on_text: app.apply_filter()
    Spinner:
        id: flt_category
        text: 'Все'
        on_text: app.apply_filter()
MDBoxLayout:
    size_hint_y: None
    height: dp(48)
    spacing: dp(8)

```

```

        MDFlatButton:
            id: flt_from_btn
            text: 'C: -'
            on_release: app.open_flt_from()
        MDFlatButton:
            id: flt_to_btn
            text: 'По: -'
            on_release: app.open_flt_to()
        MDFlatButton:
            text: 'Сброс'
            on_release: app.reset_flt()
    MDLabel:
        text: 'Нажмите на операцию, чтобы изменить
или удалить'

        theme_text_color: 'Secondary'
        font_style: 'Caption'
        size_hint_y: None
        height: dp(26)
    MDBoxLayout:
        id: tx_list
        orientation: 'vertical'
        adaptive_height: True

MDBottomNavigationItem:
    name: 'payments'
    text: 'Платежи'
    icon: 'calendar-clock'
    MDBoxLayout:
        orientation: 'vertical'
        padding: dp(12)
        spacing: dp(6)
        MDTextField:
            id: rem_title
            hint_text: 'Название платежа'
        MDTextField:
            id: rem_amount
            hint_text: 'Сумма'
            input_filter: 'float'
        MDBoxLayout:
            size_hint_y: None
            height: dp(48)
            spacing: dp(8)
            MDFlatButton:
                id: rem_date_btn
                text: 'Срок: сегодня'
                on_release: app.open_rem_date_picker()
            Spinner:
                id: rem_period
                text: 'Разовый'
                values: ['Разовый', 'Ежемесячно',
'Еженедельно', 'Ежедневно', 'Ежегодно']

```

```

MDRaisedButton:
    text: 'Добавить платёж'
    pos_hint: {'center_x': 0.5}
    on_release: app.add_reminder()
MDLabel:
    text: 'Нажмите на платёж, чтобы изменить,
оплатить/перенести или удалить'
    theme_text_color: 'Secondary'
    font_style: 'Caption'
    size_hint_y: None
    height: dp(28)
ScrollView:
    MDList:
        id: rem_list

MDBottomNavigationItem:
    name: 'goals'
    text: 'Цели'
    icon: 'target'
MDBoxLayout:
    orientation: 'vertical'
    padding: dp(12)
    spacing: dp(6)
MDTextField:
    id: goal_title
    hint_text: 'Название цели'
MDTextField:
    id: goal_target
    hint_text: 'Целевая сумма'
    input_filter: 'float'
MDRaisedButton:
    text: 'Создать цель'
    pos_hint: {'center_x': 0.5}
    on_release: app.add_goal()
Spinner:
    id: goal_select
    text: 'Цель'
    size_hint_y: None
    height: dp(48)
MDBoxLayout:
    size_hint_y: None
    height: dp(48)
    spacing: dp(8)
MDTextField:
    id: goal_amount
    hint_text: 'Сумма пополнения'
    input_filter: 'float'
MDRaisedButton:
    text: 'Пополнить'
    size_hint_x: None
    width: dp(140)

```

```

        on_release: app.contribute_goal()
MDLabel:
    text: 'Нажмите на цель, чтобы изменить или
удалить'

    theme_text_color: 'Secondary'
    font_style: 'Caption'
    size_hint_y: None
    height: dp(28)
ScrollView:
    MDList:
        id: goals_list

MDBottomNavigationBar:
    name: 'analytics'
    text: 'Аналитика'
    icon: 'chart-pie'
MDBoxLayout:
    orientation: 'vertical'
    padding: dp(12)
    spacing: dp(8)
MDBoxLayout:
    size_hint_y: None
    height: dp(48)
    spacing: dp(8)
MDFlatButton:
    id: an_from_btn
    text: 'С: —'
    on_release: app.open_an_from()
MDFlatButton:
    id: an_to_btn
    text: 'По: —'
    on_release: app.open_an_to()
MDFlatButton:
    text: 'Сброс'
    on_release: app.reset_an()
MDBoxLayout:
    orientation: 'vertical'
    size_hint_y: None
    height: dp(100)
    spacing: dp(8)
MDRaisedButton:
    text: 'Расходы по категориям'
    size_hint_x: 1
    on_release: app.build_pie()
MDRaisedButton:
    text: 'Динамика по месяцам'
    size_hint_x: 1
    on_release: app.build_dynamics()
ScrollView:
    MDBoxLayout:
        id: chart_box

```

```

orientation: 'vertical'
adaptive_height: True
padding: dp(4)
spacing: dp(10)
"""

class LoginScreen(MDScreen):
    pass

class MainScreen(MDScreen):
    pass

class _EditContent(MDBoxLayout):
    """Содержимое диалога редактирования с произвольным числом
    полей."""

    def __init__(self, *widgets, **kwargs):
        super().__init__(orientation="vertical", spacing=dp(8),
                          padding=dp(8), size_hint_y=None, **kwargs)
        self.fields = list(widgets)
        self.height = dp(64) * len(widgets) + dp(16)
        for widget in widgets:
            self.add_widget(widget)

class WalletApp(MDApp):
    """Корневое приложение KivyMD."""

    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.context: AppContext | None = None
        self.account = None
        self.tx_date = date.today()
        self.rem_date = date.today()
        self._dialog = None
        self._confirm = None           # диалог подтверждения удаления
        self._edit_due = None          # дата срока при редактировании
        self._edit_due_btn = None
        self.flt_from = None           # фильтр истории: начало периода
        self.flt_to = None             # фильтр истории: конец периода
        self.an_from = None            # аналитика: начало периода
        self.an_to = None              # аналитика: конец периода
        self._cat_map = None           # кэш id->имя категории на время
        # перерисовки

    def build(self):
        self.title = "Wallet"
        self.theme_cls.primary_palette = "Teal"
        self.theme_cls.theme_style = "Light"
        return Builder.load_string(KV)

```

```

def on_start(self):
    # На Android 13+ для показа уведомлений нужно разрешение в
рантайме.
    self._request_android_permissions()

    @staticmethod
    def _request_android_permissions():
        try:
            from android.permissions import Permission,
request_permissions

            request_permissions([Permission.POST_NOTIFICATIONS])
        except Exception: # noqa: BLE001 # носек B110 - не Android
/ модуль недоступен
            pass # на десктопе модуля android нет – разрешения не
требуются

    # --- вспомогательное ---

    def _login_ids(self):
        return self.root.get_screen("login").ids

    def _main_ids(self):
        return self.root.get_screen("main").ids

    def _data_dir(self) -> Path:
        """Каталог данных приложения (приватный, зависит от
платформы)."""
        base = Path(self.user_data_dir)
        base.mkdir(parents=True, exist_ok=True)
        return base

    def _toast(self, text: str) -> None:
        try:
            from kivymd.uix.snackbar import Snackbar

            Snackbar(text=text).open()
        except Exception: # noqa: BLE001
            print(text)

    def _confirm_delete(self, message: str, on_confirm) -> None:
        """Показать диалог подтверждения удаления."""
        self._confirm = MDDialog(
            title="Подтверждение",
            text=message,
            buttons=[
                MDFlatButton(text="Отмена",
                             on_release=lambda *_:
self._confirm.dismiss()),
                MDRaisedButton(text="Удалить", md_bg_color=(0.8, 0.2,
0.2, 1),

```

```

on_release=lambda *_:
self._run_confirmed(on_confirm)),
    ],
    )
    self._confirm.open()

def _run_confirmed(self, on_confirm) -> None:
    self._confirm.dismiss()
    on_confirm()

def _categories(self):
    return self.context.category_service.list()

def _category_by_name(self, name: str):
    for c in self._categories():
        if c.name == name:
            return c
    return None

def _cat_name(self, category_id):
    if category_id is None:
        return ""
    # используем кэш, построенный в refresh_all (избегаем O(n^2)
    # при отрисовке)
    if self._cat_map is not None:
        return self._cat_map.get(category_id, "")
    for c in self._categories():
        if c.id == category_id:
            return c.name
    return ""

# --- аутентификация ---

def register(self):
    password = self._login_ids().password.text
    if not password:
        self._login_ids().status.text = "Введите пароль"
        return
    try:
        self.context = AppContext.init_vault(self._data_dir(),
password)
    except FileExistsError:
        self._login_ids().status.text = "Хранилище уже существует
– войдите"
        return
    self._enter_app()

def login(self):
    password = self._login_ids().password.text
    if not vault.vault_exists(self._data_dir()):
        self._login_ids().status.text = "Хранилище не создано –

```



```

        создайте его"
        return
    try:
        self.context = AppContext.open_vault(self._data_dir(),
password)
    except vault.InvalidPasswordError:
        self._login_ids().status.text = "Неверный пароль"
        return
    self._enter_app()

def logout(self):
    if self.context:
        self.context.save()
        self.context.close()
        self.context = None
    self._login_ids().password.text = ""
    self._login_ids().status.text = ""
    self.root.current = "login"

def _enter_app(self):
    self.context.category_service.ensure_defaults()
    accounts = self.context.account_service.list()
    self.account = accounts[0] if accounts else \
        self.context.account_service.create("Основной счёт")
    self.context.save()
    self.tx_date = date.today()
    self.rem_date = date.today()
    self._login_ids().status.text = ""
    self.root.current = "main"
    ids = self._main_ids()
    ids.tx_date_btn.text = f"Дата:
{self.tx_date.strftime('%d.%m.%Y')}}"
    ids.rem_date_btn.text = f"Срок:
{self.rem_date.strftime('%d.%m.%Y')}}"
    self.refresh_all()
    self._notify_due_payments()

def _notify_due_payments(self):
    """Разослать уведомления о наступивших платежах при входе.

    Ошибки backend уведомлений (например, в среде без графической
    оболочки) не должны мешать работе приложения.
    """
    try:

self.context.reminder_service.notify_due(self.context.notifier)
    except Exception as exc: # noqa: BLE001 - уведомления
необязательны
        from kivy.logger import Logger

        Logger.warning("Wallet: не удалось отправить уведомления:

```

```

%S", exc)

# --- выбор даты ---

def open_tx_date_picker(self):
    self._open_date_dialog(self.tx_date, self._set_tx_date)

def _set_tx_date(self, value):
    self.tx_date = value
    self._main_ids().tx_date_btn.text = f"Дата:
{value.strftime('%d.%m.%Y')}}"

def open_rem_date_picker(self):
    self._open_date_dialog(self.rem_date, self._set_rem_date)

def _set_rem_date(self, value):
    self.rem_date = value
    self._main_ids().rem_date_btn.text = f"Срок:
{value.strftime('%d.%m.%Y')}}"

# --- категории ---

def _resolve_category(self, name: str, kind: CategoryKind):
    name = (name or "").strip()
    if not name or name in (NO_CATEGORY, "Категория"):
        return None
    existing = self._category_by_name(name)
    if existing:
        return existing.id
    return self.context.category_service.add(name, kind).id

def open_add_category(self):
    content = _EditContent(MDTextField(hint_text="Название
категории"))
    self._dialog = MDDialog(
        title="Новая категория",
        type="custom",
        content_cls=content,
        buttons=[
            MDFlatButton(text="Отмена",
                           on_release=lambda *_:
self._dialog.dismiss()),
            MDRaisedButton(text="Создать",
                           on_release=lambda *_:
self._create_category(content)),
        ],
    )
    self._dialog.open()

def _create_category(self, content):
    name = content.fields[0].text.strip()

```

```

    if not name:
        self._toast("Введите название категории")
        return
    ids = self._main_ids()
    kind = (CategoryKind.EXPENSE if ids.tx_type.text == "Расход"
            else CategoryKind.INCOME)
    if self._category_by_name(name) is None:
        self.context.category_service.add(name, kind)
        self.context.save()
    self._dialog.dismiss()
    self._populate_spinners()
    ids.tx_category.text = name
    self._toast("Категория добавлена")

def open_delete_category(self):
    names = [c.name for c in self._categories()]
    if not names:
        self._toast("Нет категорий для удаления")
        return
    content = _EditContent(
        Spinner(text=names[0], values=names, size_hint_y=None,
height=dp(44)),
    )
    self._dialog = MDDialog(
        title="Удалить категорию",
        type="custom",
        content_cls=content,
        buttons=[
            MDFlatButton(text="Отмена",
on_release=lambda *_:
self._dialog.dismiss()),
            MDRaisedButton(text="Удалить", md_bg_color=(0.8, 0.2,
0.2, 1),
on_release=lambda *_:
self._ask_remove_category(content)),
        ],
    )
    self._dialog.open()

def _ask_remove_category(self, content):
    name = content.fields[0].text
    category = self._category_by_name(name)
    if category is None:
        self._toast("Категория не найдена")
        return
    self._dialog.dismiss()
    self._confirm_delete(
        f"Удалить категорию «{name}»? ",
        lambda: self._remove_category_confirmed(category.id,
name),
    )

```

```

def _remove_category_confirmed(self, category_id, name):
    self.context.category_service.delete(category_id)
    self.context.save()
    if self._main_ids().tx_category.text == name:
        self._main_ids().tx_category.text = NO_CATEGORY
    self._populate_spinners()
    self.refresh_all()
    self._toast("Категория удалена")

# --- выбор диапазона дат ---

def _pick(self, callback):
    self._open_date_dialog(date.today(), callback)

def _open_date_dialog(self, initial, on_pick):
    """Лёгкий выбор даты тремя спиннерами (быстрее тяжёлого
MDDatePicker)."""
    import calendar

    base = initial or date.today()
    years = [str(y) for y in range(2020, date.today().year + 3)]
    day_sp = Spinner(text=str(base.day),
                     values=[str(d) for d in range(1, 32)],
                     size_hint_y=None, height=dp(44))
    month_sp = Spinner(text=str(base.month),
                      values=[str(m) for m in range(1, 13)],
                      size_hint_y=None, height=dp(44))
    year_sp = Spinner(text=str(base.year), values=years,
                     size_hint_y=None, height=dp(44))
    content = MDBoxLayout(orientation="horizontal",
spacing=dp(8),
padding=dp(8), size_hint_y=None,
height=dp(60))
    content.add_widget(day_sp)
    content.add_widget(month_sp)
    content.add_widget(year_sp)

    def confirm(*_):
        try:
            year, month = int(year_sp.text), int(month_sp.text)
            day = min(int(day_sp.text), calendar.monthrange(year,
month)[1])

            picked = date(year, month, day)
        except (ValueError, TypeError):
            self._toast("Некорректная дата")
            return
        self._dialog.dismiss()
        on_pick(picked)

    self._dialog = MDDialog(

```

```

        title="Выбор даты (день / месяц / год)",
        type="custom",
        content_cls=content,
        buttons=[
            MDFlatButton(text="Отмена",
                          on_release=lambda *_:
self._dialog.dismiss()),
            MDRaisedButton(text="OK", on_release=confirm),
        ],
    )
    self._dialog.open()

    @staticmethod
    def _day_start(d):
        return datetime.combine(d, datetime.min.time()) if d else
None

    @staticmethod
    def _day_end(d):
        return datetime.combine(d, datetime.max.time()) if d else
None

    def open_flt_from(self):
        self._pick(self._set_flt_from)

    def open_flt_to(self):
        self._pick(self._set_flt_to)

    def _set_flt_from(self, value):
        self.flt_from = value
        self._main_ids().flt_from_btn.text = f"C:
{value.strftime('%d.%m.%Y')}"
        self._refresh_transactions()

    def _set_flt_to(self, value):
        self.flt_to = value
        self._main_ids().flt_to_btn.text = f"По:
{value.strftime('%d.%m.%Y')}"
        self._refresh_transactions()

    def reset_flt(self):
        self.flt_from = self.flt_to = None
        ids = self._main_ids()
        ids.flt_from_btn.text = "C: —"
        ids.flt_to_btn.text = "По: —"
        self._refresh_transactions()

    def open_an_from(self):
        self._pick(self._set_an_from)

    def open_an_to(self):

```

```

        self._pick(self._set_an_to)

    def _set_an_from(self, value):
        self.an_from = value
        self._main_ids().an_from_btn.text = f"C:
{value.strftime('%d.%m.%Y')}"

    def _set_an_to(self, value):
        self.an_to = value
        self._main_ids().an_to_btn.text = f"По:
{value.strftime('%d.%m.%Y')}"

    def reset_an(self):
        self.an_from = self.an_to = None
        ids = self._main_ids()
        ids.an_from_btn.text = "C: —"
        ids.an_to_btn.text = "По: —"

    def _set_edit_due(self, value):
        self._edit_due = value
        if self._edit_due_btn is not None:
            self._edit_due_btn.text = f"Срок:
{value.strftime('%d.%m.%Y')}"

    # --- настройки ---

    def open_settings(self):
        if not self.context:
            return
        content = _EditContent(
            MDTextField(hint_text="Новый пароль", password=True),
            MDTextField(hint_text="Повторите пароль", password=True),
        )
        self._dialog = MDDialog(
            title="Настройки — смена пароля",
            type="custom",
            content_cls=content,
            buttons=[
                MDFlatButton(text="Отмена",
                              on_release=lambda *_:
self._dialog.dismiss()),
                MDRaisedButton(text="Сохранить",
                              on_release=lambda *_:
self._save_settings(content)),
            ],
        )
        self._dialog.open()

    def _save_settings(self, content):
        new_pwd = content.fields[0].text
        confirm = content.fields[1].text

```

```

    if not new_pwd:
        self._toast("Пароль не может быть пустым")
        return
    if new_pwd != confirm:
        self._toast("Пароли не совпадают")
        return
    self.context.change_password(new_pwd)
    self._dialog.dismiss()
    self._toast("Пароль изменён")

# --- операции ---

def add_transaction(self):
    ids = self._main_ids()
    try:
        amount = Decimal(ids.tx_amount.text)
    except (InvalidOperation, ValueError):
        self._toast("Введите корректную сумму")
        return
    is_expense = ids.tx_type.text == "Расход"
    tx_type = TransactionType.EXPENSE if is_expense else
TransactionType.INCOME
    kind = CategoryKind.EXPENSE if is_expense else
CategoryKind.INCOME
    created_at = datetime.combine(self.tx_date,
datetime.now().time())
    try:
        self.context.transaction_service.add(
            Transaction(
                amount=amount,
                type=tx_type,
                account_id=self.account.id,

category_id=self._resolve_category(ids.tx_category.text, kind),
                note=ids.tx_note.text,
                created_at=created_at,
            )
        )
    except ValueError as exc:
        self._toast(str(exc))
        return
    self.context.save()
    ids.tx_amount.text = ""
    ids.tx_note.text = ""
    self.tx_date = date.today()
    ids.tx_date_btn.text = f"Дата:
{self.tx_date.strftime('%d.%m.%Y')}}"
    self.refresh_all()
    self._toast("Операция добавлена")

def apply_filter(self):

```

```

        if not self.context:
            return
        self._refresh_transactions()

    def open_tx_dialog(self, tx_id):
        tx = self.context.transaction_service.get(tx_id)
        if tx is None:
            return
        names = [c.name for c in self._categories()]
        content = _EditContent(
            MDTextField(text=str(tx.amount), hint_text="Сумма",
                        input_filter="float"),
            Spinner(text="Расход" if tx.type ==
TransactionType.EXPENSE else "Доход",
                    values=["Расход", "Доход"], size_hint_y=None,
height=dp(44)),
            Spinner(text=self._cat_name(tx.category_id) or
NO_CATEGORY,
                    values=[NO_CATEGORY] + names, size_hint_y=None,
height=dp(44)),
            MDTextField(text=tx.note, hint_text="Комментарий"),
        )
        self._dialog = MDDialog(
            title="Операция",
            type="custom",
            content_cls=content,
            buttons=[
                MDFlatButton(text="Удалить",
theme_text_color="Custom",
                                text_color=(0.8, 0.1, 0.1, 1),
                                on_release=lambda *_:
self._ask_delete_tx(tx_id)),
                MDFlatButton(text="Отмена",
                                on_release=lambda *_:
self._dialog.dismiss()),
                MDRaisedButton(text="Сохранить",
                                on_release=lambda *_:
self._save_tx(tx_id, content)),
            ],
        )
        self._dialog.open()

    def _save_tx(self, tx_id, content):
        try:
            amount = Decimal(content.fields[0].text)
        except (InvalidOperation, ValueError):
            self._toast("Некорректная сумма")
            return
        is_expense = content.fields[1].text == "Расход"
        tx_type = TransactionType.EXPENSE if is_expense else
TransactionType.INCOME

```



```

        kind = CategoryKind.EXPENSE if is_expense else
CategoryKind.INCOME
        category_id = self._resolve_category(content.fields[2].text,
kind)
        note = content.fields[3].text
        try:
            self.context.transaction_service.edit(
                tx_id, amount=amount, note=note, tx_type=tx_type,
                category_id=category_id) # все изменения – через
сервис
        except ValueError as exc:
            self._toast(str(exc))
            return
        self.context.save()
        self._dialog.dismiss()
        self.refresh_all()
        self._toast("Операция изменена")

    def _ask_delete_tx(self, tx_id):
        self._dialog.dismiss()
        self._confirm_delete("Удалить операцию?",
                               lambda: self._delete_tx(tx_id))

    def _delete_tx(self, tx_id):
        self.context.transaction_service.delete(tx_id)
        self.context.save()
        self.refresh_all()
        self._toast("Операция удалена")

# --- цели ---

    def add_goal(self):
        ids = self._main_ids()
        title = ids.goal_title.text.strip()
        if not title:
            self._toast("Введите название цели")
            return
        try:
            target = Decimal(ids.goal_target.text)
            self.context.goal_service.create(title, target)
        except (InvalidOperation, ValueError):
            self._toast("Введите корректную целевую сумму")
            return
        self.context.save()
        ids.goal_title.text = ""
        ids.goal_target.text = ""
        self.refresh_all()
        self._toast("Цель создана")

    def contribute_goal(self):
        ids = self._main_ids()

```

```

title = ids.goal_select.text
goal = next((g for g in self.context.goal_service.list()
             if g.title == title), None)
if goal is None:
    self._toast("Выберите цель")
    return
try:
    amount = Decimal(ids.goal_amount.text)
    self.context.goal_service.contribute(goal.id, amount)
except (InvalidOperation, ValueError) as exc:
    self._toast("Введите корректную сумму пополнения"
               if isinstance(exc, InvalidOperation) else
str(exc))
    return
self.context.save()
ids.goal_amount.text = ""
self.refresh_all()
self._toast("Цель пополнена")

def open_goal_dialog(self, goal_id):
    goal = self.context.goal_service.get(goal_id)
    if goal is None:
        return
    content = _EditContent(
        MDTextField(text=goal.title, hint_text="Название цели"),
        MDTextField(text=str(goal.target_amount),
hint_text="Целевая сумма",
                    input_filter="float"),
    )
    self._dialog = MDDialog(
        title="Редактирование цели",
        type="custom",
        content_cls=content,
        buttons=[
            MDFlatButton(text="Удалить",
theme_text_color="Custom",
                        text_color=(0.8, 0.1, 0.1, 1),
                        on_release=lambda *_:
self._ask_delete_goal(goal_id)),
            MDFlatButton(text="Отмена",
                        on_release=lambda *_:
self._dialog.dismiss()),
            MDRaisedButton(text="Сохранить",
                        on_release=lambda *_:
self._save_goal(goal_id, content)),
        ],
    )
    self._dialog.open()

def _save_goal(self, goal_id, content):
    try:

```

```

        target = Decimal(content.fields[1].text)
    except (InvalidOperation, ValueError):
        self._toast("Некорректная целевая сумма")
        return
    try:
        self.context.goal_service.update(
            goal_id, title=content.fields[0].text,
target_amount=target)
    except ValueError as exc:
        self._toast(str(exc))
        return
    self.context.save()
    self._dialog.dismiss()
    self.refresh_all()
    self._toast("Цель обновлена")

def _ask_delete_goal(self, goal_id):
    self._dialog.dismiss()
    self._confirm_delete("Удалить цель?",
                        lambda: self._delete_goal(goal_id))

def _delete_goal(self, goal_id):
    self.context.goal_service.delete(goal_id)
    self.context.save()
    self.refresh_all()
    self._toast("Цель удалена")

# --- платежи (напоминания) ---

def add_reminder(self):
    ids = self._main_ids()
    title = ids.rem_title.text.strip()
    if not title:
        self._toast("Введите название платежа")
        return
    try:
        amount = Decimal(ids.rem_amount.text)
    except (InvalidOperation, ValueError):
        self._toast("Введите корректную сумму")
        return
    period = PERIOD_MAP.get(ids.rem_period.text, "")
    try:
        self.context.reminder_service.schedule(
            Reminder(
                title=title,
                amount=amount,
                due_date=self.rem_date,
                is_repeating=bool(period),
                period=period,
            )
        )
    )

```

```

except ValueError as exc:
    self._toast(str(exc))
    return
self.context.save()
ids.rem_title.text = ""
ids.rem_amount.text = ""
self.rem_date = date.today()
ids.rem_date_btn.text = f"Срок:
{self.rem_date.strftime('%d.%m.%Y')}}"
self.refresh_all()
self._toast("Платёж добавлен")

def pay_reminder(self, reminder_id: int, repeating: bool):
    if repeating:
        self.context.reminder_service.advance(reminder_id)
        self._toast("Платёж перенесён на следующий период")
    else:
        self.context.reminder_service.delete(reminder_id)
        self._toast("Платёж отмечен оплаченным")
    self.context.save()
    self.refresh_all()

def open_reminder_dialog(self, reminder_id, repeating):
    reminder = self.context.reminder_service.get(reminder_id)
    if reminder is None:
        return
    period_spinner = Spinner(
        text=PERIOD_LABELS.get(reminder.period, "Разовый"),
        values=["Разовый"] + list(PERIOD_MAP.keys()),
        size_hint_y=None, height=dp(44),
    )
    self._edit_due = reminder.due_date
    self._edit_due_btn = MDFlatButton(
        text=f"Срок: {reminder.due_date.strftime('%d.%m.%Y')}}"
    self._edit_due_btn.bind(
        on_release=lambda _: self._pick(self._set_edit_due))
    content = _EditContent(
        MDTextField(text=reminder.title, hint_text="Название
платежа"),
        MDTextField(text=str(reminder.amount), hint_text="Сумма",
            input_filter="float"),
        period_spinner,
        self._edit_due_btn,
    )
    pay_label = "Перенести" if repeating else "Оплатить"
    self._dialog = MDDialog(
        title="Платёж",
        type="custom",
        content_cls=content,
        buttons=[
            MDFlatButton(text=pay_label,

```

```

on_release=lambda *_:
self._pay_from_dialog(
    reminder_id, repeating)),
    MDFlatButton(text="Удалить",
theme_text_color="Custom",
    text_color=(0.8, 0.1, 0.1, 1),
on_release=lambda *_:
self._ask_delete_reminder(reminder_id)),
    MDRaisedButton(text="Сохранить",
on_release=lambda *_:
self._save_reminder(
    reminder_id, content)),
    ],
)
self._dialog.open()

def _pay_from_dialog(self, reminder_id, repeating):
    self._dialog.dismiss()
    self.pay_reminder(reminder_id, repeating)

def _save_reminder(self, reminder_id, content):
    try:
        amount = Decimal(content.fields[1].text)
    except (InvalidOperation, ValueError):
        self._toast("Некорректная сумма")
        return
    period = PERIOD_MAP.get(content.fields[2].text, "")
    try:
        self.context.reminder_service.update(
            reminder_id, title=content.fields[0].text,
amount=amount,
            period=period, due_date=self._edit_due)
    except ValueError as exc:
        self._toast(str(exc))
        return
    self.context.save()
    self._dialog.dismiss()
    self.refresh_all()
    self._toast("Платёж обновлён")

def _ask_delete_reminder(self, reminder_id):
    self._dialog.dismiss()
    self._confirm_delete("Удалить платёж?",
        lambda:
self._delete_reminder(reminder_id))

def _delete_reminder(self, reminder_id):
    self.context.reminder_service.delete(reminder_id)
    self.context.save()
    self.refresh_all()
    self._toast("Платёж удалён")

```

```

# --- аналитика ---

def _chart_title(self, text):
    return MDLabel(text=text, bold=True, size_hint_y=None,
height=dp(28))

def build_pie(self):
    """Структура расходов по категориям – нативные горизонтальные
бары."""
    data = self.context.analytics.expenses_by_category(
        self._day_start(self.an_from), self._day_end(self.an_to))
    box = self._main_ids().chart_box
    box.clear_widgets()
    box.add_widget(self._chart_title("Структура расходов по
категориям"))
    if not data:
        box.add_widget(MDLabel(text="Нет данных",
theme_text_color="Secondary",
size_hint_y=None, height=dp(28)))
    return
    total = sum(data.values())
    maxv = max(data.values())
    for name, value in sorted(data.items(), key=lambda kv: kv[1],
reverse=True):
        percent = float(value / total * 100)
        row = MDBoxLayout(orientation="vertical",
size_hint_y=None,
height=dp(50), spacing=dp(2))
        row.add_widget(MDLabel(
            text=f"{name}: {value} Р ({percent:.0f}%)",
            size_hint_y=None, height=dp(24)))
        row.add_widget(MDProgressBar(value=float(value / maxv *
100),
size_hint_y=None,
height=dp(8)))
        box.add_widget(row)

def build_dynamics(self):
    """Динамика доходов и расходов по месяцам – нативные бары."""
    data = self.context.analytics.monthly_dynamics(
        self._day_start(self.an_from), self._day_end(self.an_to))
    box = self._main_ids().chart_box
    box.clear_widgets()
    box.add_widget(self._chart_title("Динамика по месяцам"))
    if not data:
        box.add_widget(MDLabel(text="Нет данных",
theme_text_color="Secondary",
size_hint_y=None, height=dp(28)))
    return
    maxv = max((max(inc, exp) for _, inc, exp in data),

```

```

default=Decimal("1"))
    maxv = maxv or Decimal("1")
    for label, inc, exp in data:
        row = MDBoxLayout(orientation="vertical",
size_hint_y=None,
                                height=dp(86), spacing=dp(1))
        row.add_widget(MDLabel(text=label, bold=True,
                                size_hint_y=None, height=dp(22)))
        row.add_widget(MDLabel(text=f"Доход: {inc} ₺",
                                size_hint_y=None, height=dp(20)))
        income_bar = MDProgressBar(value=float(inc / maxv * 100),
                                size_hint_y=None,
height=dp(6))
        income_bar.color = (0.30, 0.69, 0.31, 1)
        row.add_widget(income_bar)
        row.add_widget(MDLabel(text=f"Расход: {exp} ₺",
                                size_hint_y=None, height=dp(20)))
        expense_bar = MDProgressBar(value=float(exp / maxv *
100),
                                size_hint_y=None,
height=dp(6))
        expense_bar.color = (0.90, 0.22, 0.21, 1)
        row.add_widget(expense_bar)
        box.add_widget(row)

# --- обновление экранов ---

def refresh_all(self):
    if not self.context:
        return
    # один проход по категориям на всю перерисовку (кэш для
_cat_name)
    self._cat_map = {c.id: c.name for c in self._categories()}
    self._populate_spinners()
    self._refresh_dashboard()
    self._refresh_transactions()
    self._refresh_goals()
    self._refresh_reminders()

def _populate_spinners(self):
    ids = self._main_ids()
    names = [c.name for c in self._categories()]
    ids.tx_category.values = [NO_CATEGORY] + names
    if ids.tx_category.text in ("Категория", ""):
        ids.tx_category.text = NO_CATEGORY
    ids.flt_category.values = ["Все"] + names
    goals = [g.title for g in self.context.goal_service.list()]
    ids.goal_select.values = goals
    if goals and ids.goal_select.text in ("Цель", ""):
        ids.goal_select.text = goals[0]

```

```

def _tx_item(self, tx):
    sign = "-" if tx.type == TransactionType.EXPENSE else "+"
    category = self._cat_name(tx.category_id) or "без категории"
    item = ThreeLineListItem(
        text=f"{sign}{tx.amount} ₽",
        secondary_text=f"{tx.created_at.strftime('%d.%m.%Y')}"
        {category}",
        tertiary_text=tx.note or "-",
    )
    item.bind(on_release=lambda inst, tid=tx.id:
self.open_tx_dialog(tid))
    return item

def _refresh_dashboard(self):
    ids = self._main_ids()
    total = self.context.account_service.total_balance()
    ids.balance.text = f"Баланс: {total} ₽"
    ids.recent_list.clear_widgets()
    for tx in self.context.transaction_service.list():
        ids.recent_list.add_widget(self._tx_item(tx))

def _refresh_transactions(self):
    ids = self._main_ids()
    tx_type = None
    if ids.flt_type.text == "Доход":
        tx_type = TransactionType.INCOME
    elif ids.flt_type.text == "Расход":
        tx_type = TransactionType.EXPENSE
    category_id = None
    if ids.flt_category.text not in ("Все", ""):
        cat = self._category_by_name(ids.flt_category.text)
        category_id = cat.id if cat else -1
    ids.tx_list.clear_widgets()
    for tx in self.context.transaction_service.list(
        category_id=category_id, tx_type=tx_type,
        date_from=self._day_start(self.flt_from),
        date_to=self._day_end(self.flt_to),
    ):
        ids.tx_list.add_widget(self._tx_item(tx))

def _refresh_goals(self):
    ids = self._main_ids()
    ids.goals_list.clear_widgets()
    for goal in self.context.goal_service.list():
        percent = int(goal.progress * 100)
        status = " (выполнена)" if goal.is_reached else ""
        item = TwoLineListItem(
            text=f"{goal.title}{status}",
            secondary_text=f"{goal.current_amount}/{goal.target_amount} ₽"
            f" ({percent}%)",

```



```

    )
    item.bind(on_release=lambda inst, gid=goal.id:
               self.open_goal_dialog(gid))
    ids.goals_list.add_widget(item)

def _refresh_reminders(self):
    ids = self._main_ids()
    ids.rem_list.clear_widgets()
    for r in sorted(self.context.reminder_service.list(),
                    key=lambda x: x.due_date):
        period = PERIOD_LABELS.get(r.period, "разовый")
        item = TwoLineListItem(
            text=f"{r.title} – {r.amount} ₰",
            secondary_text=f"срок
{r.due_date.strftime('%d.%m.%Y')} · {period}",
        )
        item.bind(
            on_release=lambda inst, rid=r.id, rep=r.is_repeating:
            self.open_reminder_dialog(rid, rep)
        )
        ids.rem_list.add_widget(item)

def run() -> None:
    """Точка входа GUI."""
    if not KIVYMD_AVAILABLE:
        raise RuntimeError(
            "Kivy/KivyMD не установлены. Установите: pip install kivy
kivymd"
        )
    WalletApp().run()

if __name__ == "__main__": # pragma: no cover
    run()

```

Приложение Г. Листинги тестов

Ниже приведён код модульных и интеграционных тестов (*pytest*).

Листинг Г.1 — *tests/conftest.py*

```
"""Общие фикстуры для тестов."""

from __future__ import annotations

from decimal import Decimal

import pytest

from wallet.app_context import AppContext
from wallet.core.db import Database
from wallet.models import Account

@pytest.fixture
def context() -> AppContext:
    """Контекст приложения с БД в памяти."""
    db = Database() # in-memory
    db.connect()
    return AppContext(db)

@pytest.fixture
def account(context: AppContext) -> Account:
    """Тестовый счёт с нулевым балансом."""
    return context.accounts.add(
        Account(name="Наличные", type="cash", balance=Decimal("0"))
    )
```

Листинг Г.2 — *tests/test_account_service.py*

```
"""Тесты сервиса счетов."""

from __future__ import annotations

from decimal import Decimal

import pytest

def test_create_account(context):
    acc = context.account_service.create("Карта", type="card",
                                         initial_balance=Decimal("100"))
    assert acc.id is not None
    assert acc.balance == Decimal("100")
```

```

def test_empty_name_rejected(context):
    with pytest.raises(ValueError):
        context.account_service.create("")

def test_total_balance(context):
    context.account_service.create("Наличные",
    initial_balance=Decimal("300"))
    context.account_service.create("Карта",
    initial_balance=Decimal("700"))
    assert context.account_service.total_balance() == Decimal("1000")

```

Листинг Г.3 — tests/test_analytics_service.py

"""Тесты сервиса аналитики."""

```

from __future__ import annotations

from decimal import Decimal

from wallet.models import Category, CategoryKind, Transaction,
TransactionType

def _add_expense(context, account, category_id, amount):
    context.transaction_service.add(
        Transaction(amount=Decimal(amount), type=TransactionType.EXPENSE,
        account_id=account.id, category_id=category_id)
    )

def test_expenses_by_category(context, account):
    food = context.categories.add(Category(name="Продукты",
    kind=CategoryKind.EXPENSE))
    transport = context.categories.add(Category(name="Транспорт",
    kind=CategoryKind.EXPENSE))

    _add_expense(context, account, food.id, "300")
    _add_expense(context, account, food.id, "200")
    _add_expense(context, account, transport.id, "150")

    by_cat = context.analytics.expenses_by_category()
    assert by_cat["Продукты"] == Decimal("500")
    assert by_cat["Транспорт"] == Decimal("150")

def test_monthly_dynamics(context, account):
    from datetime import datetime

    context.transaction_service.add(
        Transaction(amount=Decimal("1000"), type=TransactionType.INCOME,

```

```

        account_id=account.id, created_at=datetime(2026, 1,
10))
    )
    context.transaction_service.add(
        Transaction(amount=Decimal("300"), type=TransactionType.EXPENSE,
20))
        account_id=account.id, created_at=datetime(2026, 1,
    )
    context.transaction_service.add(
        Transaction(amount=Decimal("500"), type=TransactionType.INCOME,
5))
        account_id=account.id, created_at=datetime(2026, 2,
    )
    dynamics = context.analytics.monthly_dynamics()
    assert dynamics == [
        ("01.2026", Decimal("1000"), Decimal("300")),
        ("02.2026", Decimal("500"), Decimal("0")),
    ]

def test_income_vs_expense(context, account):
    context.transaction_service.add(
        Transaction(amount=Decimal("2000"), type=TransactionType.INCOME,
    )
        account_id=account.id)
    context.transaction_service.add(
        Transaction(amount=Decimal("750"), type=TransactionType.EXPENSE,
    )
        account_id=account.id)
    totals = context.analytics.income_vs_expense()
    assert totals["income"] == Decimal("2000")
    assert totals["expense"] == Decimal("750")
    assert totals["balance"] == Decimal("1250")

```

Листинг Г.4 — tests/test_auth_service.py

"""Тесты сервиса аутентификации."""

```

from __future__ import annotations

```

```

import pytest

```

```

def test_register_and_login(context):
    context.auth.register("alex", "secret123")
    user = context.auth.login("alex", "secret123")
    assert user is not None
    assert user.name == "alex"

```

```

def test_login_with_wrong_password(context):

```

```

context.auth.register("alex", "secret123")
assert context.auth.login("alex", "bad") is None

def test_password_is_not_stored_plaintext(context):
    user = context.auth.register("alex", "secret123")
    assert user.password_hash != "secret123"
    assert "secret123" not in user.password_hash

def test_duplicate_registration_rejected(context):
    context.auth.register("alex", "secret123")
    with pytest.raises(ValueError):
        context.auth.register("alex", "another")

```

Листинг Г.5 — tests/test_category_service.py

```

"""Тесты сервиса категорий."""

from __future__ import annotations

from decimal import Decimal

from wallet.models import Category, CategoryKind, Transaction,
TransactionType

def test_ensure_defaults_creates_predefined(context):
    created = context.category_service.ensure_defaults()
    assert len(created) > 0
    # повторный вызов не дублирует
    assert context.category_service.ensure_defaults() == []

def test_add_custom_category(context):
    cat = context.category_service.add("Кафе", CategoryKind.EXPENSE)
    assert cat.id is not None
    assert any(c.name == "Кафе" for c in context.category_service.list())

def test_delete_category_nullifies_transactions(context, account):
    cat = context.category_service.add("Такси", CategoryKind.EXPENSE)
    tx = context.transaction_service.add(
        Transaction(amount=Decimal("300"), type=TransactionType.EXPENSE,
                    account_id=account.id, category_id=cat.id)
    )
    context.category_service.delete(cat.id)
    assert all(c.id != cat.id for c in context.category_service.list())
    # операция сохранилась, но без категории
    assert context.transactions.get(tx.id).category_id is None

```

Листинг Г.6 — tests/test_chart_service.py

"""Тесты построения диаграмм."""

```
from __future__ import annotations

from decimal import Decimal

from wallet.models import Category, CategoryKind, Transaction,
TransactionType

PNG_MAGIC = b"\x89PNG\r\n\x1a\n"

def _seed(context, account):
    food = context.categories.add(Category(name="Продукты",
kind=CategoryKind.EXPENSE))
    context.transaction_service.add(
        Transaction(amount=Decimal("1000"), type=TransactionType.INCOME,
            account_id=account.id)
    )
    context.transaction_service.add(
        Transaction(amount=Decimal("250"), type=TransactionType.EXPENSE,
            account_id=account.id, category_id=food.id)
    )

def test_expenses_pie_creates_png(context, account, tmp_path):
    _seed(context, account)
    path = context.chart_service.expenses_pie(tmp_path / "pie.png")
    assert path.exists()
    assert path.read_bytes()[8] == PNG_MAGIC

def test_income_expense_bar_creates_png(context, account, tmp_path):
    _seed(context, account)
    path = context.chart_service.income_expense_bar(tmp_path / "bar.png")
    assert path.exists()
    assert path.read_bytes()[8] == PNG_MAGIC

def test_pie_with_no_data(context, tmp_path):
    path = context.chart_service.expenses_pie(tmp_path / "empty.png")
    assert path.exists()

def test_dynamics_chart_creates_png(context, account, tmp_path):
    _seed(context, account)
    path = context.chart_service.dynamics_chart(tmp_path / "dyn.png")
    assert path.exists()
    assert path.read_bytes()[8] == PNG_MAGIC
```

Листинг Г.7 — tests/test_db_encryption.py

```
"""Тесты шифрования базы данных «в покое»."""
```

```
from __future__ import annotations

from decimal import Decimal

import pytest

from wallet.core.db import Database
from wallet.core.security import EncryptionManager, derive_key
from wallet.models import Account
from wallet.repositories import AccountRepository

SALT = b"fixed-salt-16byte"

def _manager(password: str) -> EncryptionManager:
    return EncryptionManager(derive_key(password, SALT))

def test_data_persists_through_encrypted_file(tmp_path):
    path = tmp_path / "wallet.db.enc"

    db = Database(path=path, encryption=_manager("pass"))
    db.connect()
    AccountRepository(db.connection).add(
        Account(name="Карта", type="card", balance=Decimal("100.50"))
    )
    db.save()
    db.close()

    assert path.exists()

    db2 = Database(path=path, encryption=_manager("pass"))
    db2.connect()
    accounts = AccountRepository(db2.connection).list()
    assert len(accounts) == 1
    assert accounts[0].name == "Карта"
    assert accounts[0].balance == Decimal("100.50")

def test_file_is_not_plaintext(tmp_path):
    path = tmp_path / "wallet.db.enc"
    db = Database(path=path, encryption=_manager("pass"))
    db.connect()
    AccountRepository(db.connection).add(Account(name="SecretAccount"))
    db.save()
    db.close()

    raw = path.read_bytes()
    assert b"SecretAccount" not in raw # имя счёта не должно читаться в
```

открытом виде

```
def test_wrong_password_cannot_open(tmp_path):
    path = tmp_path / "wallet.db.enc"
    db = Database(path=path, encryption=_manager("right"))
    db.connect()
    db.save()
    db.close()

    db_wrong = Database(path=path, encryption=_manager("wrong"))
    with pytest.raises(Exception):
        db_wrong.connect()
```

Листинг Г.8 — tests/test_goal_service.py

"""Тесты сервиса финансовых целей."""

```
from __future__ import annotations

from decimal import Decimal

import pytest


def test_create_and_progress(context):
    goal = context.goal_service.create("Отпуск", Decimal("1000"))
    assert goal.progress == 0.0
    assert goal.is_reached is False


def test_contribute_updates_progress(context):
    goal = context.goal_service.create("Ноябрь", Decimal("1000"))
    updated = context.goal_service.contribute(goal.id, Decimal("250"))
    assert updated.current_amount == Decimal("250")
    assert updated.progress == 0.25


def test_goal_reached(context):
    goal = context.goal_service.create("Резерв", Decimal("500"))
    context.goal_service.contribute(goal.id, Decimal("500"))
    reached = context.goals.get(goal.id)
    assert reached.is_reached is True
    assert reached.progress == 1.0


def test_invalid_target_rejected(context):
    with pytest.raises(ValueError):
        context.goal_service.create("Плохая цель", Decimal("0"))
```



```

def test_update_goal(context):
    goal = context.goal_service.create("Старое", Decimal("1000"))
    context.goal_service.update(goal.id, title="Новое",
target_amount=Decimal("2000"))
    updated = context.goals.get(goal.id)
    assert updated.title == "Новое"
    assert updated.target_amount == Decimal("2000")

def test_delete_goal(context):
    goal = context.goal_service.create("Удалить", Decimal("1000"))
    context.goal_service.delete(goal.id)
    assert context.goals.get(goal.id) is None

```

Листинг Г.9 — tests/test_integration.py

"""Интеграционные тесты: сквозной сценарий и персистентность хранилища.

Проверяется совместная работа всех слоёв (vault → БД → репозитории → сервисы) и сохранение данных в зашифрованном хранилище между сессиями.

```

from __future__ import annotations

from datetime import date, datetime
from decimal import Decimal

import pytest

from wallet.app_context import AppContext
from wallet.core import vault
from wallet.models import Reminder, Transaction, TransactionType

def test_full_scenario_and_persistence(tmp_path):
    # --- сессия 1: создание хранилища и ввод данных ---
    ctx = AppContext.init_vault(tmp_path, "secret")
    ctx.category_service.ensure_defaults()
    salary = next(c for c in ctx.category_service.list() if c.name ==
"Зарплата")
    food = next(c for c in ctx.category_service.list() if c.name ==
"Продукты")

    account = ctx.account_service.create("Карта",
initial_balance=Decimal("0"))

    ctx.transaction_service.add(Transaction(
        amount=Decimal("50000"), type=TransactionType.INCOME,
        account_id=account.id, category_id=salary.id, note="Зарплата",
        created_at=datetime(2026, 5, 5)))
    ctx.transaction_service.add(Transaction(

```

```

        amount=Decimal("1500"), type=TransactionType.EXPENSE,
        account_id=account.id, category_id=food.id, note="Магазин",
        created_at=datetime(2026, 5, 7)))

    goal = ctx.goal_service.create("Отпуск", Decimal("100000"))
    ctx.goal_service.contribute(goal.id, Decimal("20000"))
    ctx.reminder_service.schedule(Reminder(
        title="Аренда", amount=Decimal("25000"), due_date=date(2026, 6,
1),
        is_repeating=True, period="monthly"))

    # баланс и аналитика в текущей сессии
    assert ctx.account_service.total_balance() == Decimal("48500")
    totals = ctx.analytics.income_vs_expense()
    assert totals["income"] == Decimal("50000")
    assert totals["expense"] == Decimal("1500")
    assert ctx.analytics.expenses_by_category()["Продукты"] ==
Decimal("1500")

    ctx.save()
    ctx.close()

    # --- сессия 2: повторное открытие тем же паролем ---
    reopened = AppContext.open_vault(tmp_path, "secret")
    assert reopened.account_service.total_balance() == Decimal("48500")
    assert len(reopened.transaction_service.list()) == 2
    assert reopened.goal_service.list()[0].current_amount ==
Decimal("20000")
    assert reopened.reminder_service.list()[0].title == "Аренда"
    reopened.close()

def test_persistence_rejects_wrong_password(tmp_path):
    ctx = AppContext.init_vault(tmp_path, "right")
    ctx.account_service.create("Наличные",
initial_balance=Decimal("100"))
    ctx.save()
    ctx.close()

    with pytest.raises(vault.InvalidPasswordError):
        AppContext.open_vault(tmp_path, "wrong")

def test_password_change_end_to_end(tmp_path):
    ctx = AppContext.init_vault(tmp_path, "old")
    ctx.account_service.create("Карта", initial_balance=Decimal("500"))
    ctx.change_password("new")
    ctx.close()

    with pytest.raises(vault.InvalidPasswordError):
        AppContext.open_vault(tmp_path, "old")

```

```

reopened = AppContext.open_vault(tmp_path, "new")
assert reopened.account_service.total_balance() == Decimal("500")
reopened.close()

```

Листинг Г.10 — tests/test_notifications.py

"""Тесты уведомлений и рассылки по напоминаниям."""

```

from __future__ import annotations

from datetime import date, timedelta
from decimal import Decimal

from wallet.core.notifications import Notifier, NullNotifier,
default_notifier
from wallet.models import Reminder

def test_null_notifier_records_messages():
    notifier = NullNotifier()
    notifier.notify("Заголовок", "Текст")
    assert notifier.sent == [("Заголовок", "Текст")]

def test_default_notifier_is_notifier():
    assert isinstance(default_notifier(), Notifier)

def test_notify_due_sends_only_due(context):
    today = date.today()
    context.reminder_service.schedule(
        Reminder(title="Аренда", amount=Decimal("15000"), due_date=today)
    )
    context.reminder_service.schedule(
        Reminder(title="Будущее", amount=Decimal("100"),
            due_date=today + timedelta(days=10))
    )
    notifier = NullNotifier()
    count = context.reminder_service.notify_due(notifier, until=today)
    assert count == 1
    assert "Аренда" in notifier.sent[0][0]

```

Листинг Г.11 — tests/test_reminder_recurrence.py

"""Тесты периодичности напоминаний."""

```

from __future__ import annotations

from datetime import date
from decimal import Decimal

import pytest

```

```

from wallet.models import Reminder
from wallet.services.reminder_service import next_due_date

@pytest.mark.parametrize(
    "current,period,expected",
    [
        (date(2026, 1, 15), "daily", date(2026, 1, 16)),
        (date(2026, 1, 15), "weekly", date(2026, 1, 22)),
        (date(2026, 1, 31), "monthly", date(2026, 2, 28)), #
        корректировка под февраль
        (date(2026, 1, 15), "yearly", date(2027, 1, 15)),
    ],
)
def test_next_due_date(current, period, expected):
    assert next_due_date(current, period) == expected

def test_unknown_period_rejected():
    with pytest.raises(ValueError):
        next_due_date(date.today(), "hourly")

def test_advance_repeating_reminder(context):
    r = context.reminder_service.schedule(
        Reminder(title="Аренда", amount=Decimal("15000"),
            due_date=date(2026, 1, 31), is_repeating=True,
period="monthly")
    )
    advanced = context.reminder_service.advance(r.id)
    assert advanced.due_date == date(2026, 2, 28)

def test_advance_non_repeating_keeps_date(context):
    r = context.reminder_service.schedule(
        Reminder(title="Разовый", amount=Decimal("500"),
due_date=date(2026, 1, 10))
    )
    advanced = context.reminder_service.advance(r.id)
    assert advanced.due_date == date(2026, 1, 10)

```

Листинг Г.12 — tests/test_reminder_service.py

"""Тесты сервиса напоминаний."""

```

from __future__ import annotations

from datetime import date, timedelta
from decimal import Decimal

```

```

from wallet.models import Reminder

def test_upcoming_includes_due_and_excludes_future(context):
    today = date.today()
    context.reminder_service.schedule(
        Reminder(title="Аренда", amount=Decimal("15000"), due_date=today)
    )
    context.reminder_service.schedule(
        Reminder(title="Подписка", amount=Decimal("500"),
            due_date=today + timedelta(days=30))
    )

    upcoming = context.reminder_service.upcoming(until=today)
    titles = [r.title for r in upcoming]
    assert "Аренда" in titles
    assert "Подписка" not in titles

def test_update_reminder(context):
    r = context.reminder_service.schedule(
        Reminder(title="Старое", amount=Decimal("100"),
            due_date=date.today())
    )
    context.reminder_service.update(r.id, title="Новое",
        amount=Decimal("250"))
    updated = context.reminders.get(r.id)
    assert updated.title == "Новое"
    assert updated.amount == Decimal("250")

def test_update_reminder_period(context):
    r = context.reminder_service.schedule(
        Reminder(title="Подписка", amount=Decimal("500"),
            due_date=date.today())
    )
    assert r.is_repeating is False
    updated = context.reminder_service.update(r.id, period="monthly")
    assert updated.period == "monthly"
    assert updated.is_repeating is True
    # снятие повторяемости
    again = context.reminder_service.update(r.id, period="")
    assert again.is_repeating is False

def test_delete_reminder(context):
    r = context.reminder_service.schedule(
        Reminder(title="Удалить", amount=Decimal("100"),
            due_date=date.today())
    )
    context.reminder_service.delete(r.id)

```

```
assert context.reminders.get(r.id) is None
```

```
def test_upcoming_with_horizon(context):
    today = date.today()
    context.reminder_service.schedule(
        Reminder(title="Кредит", amount=Decimal("8000"),
                due_date=today + timedelta(days=5))
    )
    upcoming = context.reminder_service.upcoming(until=today +
timedelta(days=7))
    assert [r.title for r in upcoming] == ["Кредит"]
```

Листинг Г.13 — tests/test_security.py

"""Тесты модуля безопасности: вывод ключа, пароли, шифрование."""

```
from __future__ import annotations
```

```
import pytest
```

```
from wallet.core.security import (
    KEY_LENGTH,
    EncryptionManager,
    derive_key,
    generate_salt,
    hash_password,
    verify_password,
)
```

```
def test_derive_key_is_deterministic():
    salt = generate_salt()
    assert derive_key("secret", salt) == derive_key("secret", salt)
```

```
def test_derive_key_length_is_256_bit():
    assert len(derive_key("secret", generate_salt())) == KEY_LENGTH
```

```
def test_different_salt_gives_different_key():
    assert derive_key("secret", generate_salt()) != derive_key("secret",
generate_salt())
```

```
def test_password_verification():
    pw_hash, salt = hash_password("p@ssw0rd")
    assert verify_password("p@ssw0rd", pw_hash, salt) is True
    assert verify_password("wrong", pw_hash, salt) is False
```

```

def test_encryption_round_trip():
    manager = EncryptionManager(derive_key("key", generate_salt()))
    data = b"confidential financial data"
    assert manager.decrypt(manager.encrypt(data)) == data

def test_decrypt_with_wrong_key_fails():
    data = b"secret"
    token = EncryptionManager(derive_key("right", b"0" *
16)).encrypt(data)
    wrong = EncryptionManager(derive_key("wrong", b"0" * 16))
    with pytest.raises(Exception):
        wrong.decrypt(token)

def test_invalid_key_length_rejected():
    with pytest.raises(ValueError):
        EncryptionManager(b"too-short")

```

Листинг Г.14 — tests/test_transaction_edit.py

```

"""Тесты редактирования транзакций с пересчётом баланса."""

from __future__ import annotations

from decimal import Decimal

from wallet.models import Transaction, TransactionType

def test_edit_amount_recalculates_balance(context, account):
    tx = context.transaction_service.add(
        Transaction(amount=Decimal("1000"), type=TransactionType.INCOME,
            account_id=account.id)
    )
    context.transaction_service.edit(tx.id, amount=Decimal("600"))
    assert context.accounts.get(account.id).balance == Decimal("600")

def test_edit_can_change_category(context, account):
    from wallet.models import Category, CategoryKind

    food = context.categories.add(Category(name="Еда",
kind=CategoryKind.EXPENSE))
    taxi = context.categories.add(Category(name="Такси",
kind=CategoryKind.EXPENSE))
    tx = context.transaction_service.add(
        Transaction(amount=Decimal("100"), type=TransactionType.EXPENSE,
            account_id=account.id, category_id=food.id)
    )
    # смена суммы и категории — одним вызовом сервиса

```

```

    context.transaction_service.edit(tx.id, amount=Decimal("150"),
category_id=taxi.id)

    result = context.transaction_service.get(tx.id)
    assert result.category_id == taxi.id
    assert result.amount == Decimal("150")

def test_edit_can_clear_category(context, account):
    from wallet.models import Category, CategoryKind

    food = context.categories.add(Category(name="Еда",
kind=CategoryKind.EXPENSE))
    tx = context.transaction_service.add(
        Transaction(amount=Decimal("100"), type=TransactionType.EXPENSE,
            account_id=account.id, category_id=food.id)
    )
    context.transaction_service.edit(tx.id, category_id=None) # снять
категорию
    assert context.transaction_service.get(tx.id).category_id is None

def test_edit_without_category_keeps_it(context, account):
    from wallet.models import Category, CategoryKind

    food = context.categories.add(Category(name="Еда",
kind=CategoryKind.EXPENSE))
    tx = context.transaction_service.add(
        Transaction(amount=Decimal("100"), type=TransactionType.EXPENSE,
            account_id=account.id, category_id=food.id)
    )
    context.transaction_service.edit(tx.id, note="изменено") # категорию
не трогаем
    assert context.transaction_service.get(tx.id).category_id == food.id

def test_edit_type_recalculates_balance(context, account):
    tx = context.transaction_service.add(
        Transaction(amount=Decimal("200"), type=TransactionType.INCOME,
            account_id=account.id)
    )
    # было +200 (доход); меняем на расход -> итог -200
    context.transaction_service.edit(tx.id,
tx_type=TransactionType.EXPENSE)
    assert context.accounts.get(account.id).balance == Decimal("-200")

```

Листинг Г.15 — tests/test_transaction_service.py

"""Тесты сервиса учёта доходов и расходов."""

```

from __future__ import annotations

```



```

from decimal import Decimal

import pytest

from wallet.models import Transaction, TransactionType


def test_income_increases_balance(context, account):
    context.transaction_service.add(
        Transaction(amount=Decimal("1000"), type=TransactionType.INCOME,
                    account_id=account.id, note="Зарплата")
    )
    assert context.accounts.get(account.id).balance == Decimal("1000")


def test_expense_decreases_balance(context, account):
    context.transaction_service.add(
        Transaction(amount=Decimal("1000"), type=TransactionType.INCOME,
                    account_id=account.id)
    )
    context.transaction_service.add(
        Transaction(amount=Decimal("300"), type=TransactionType.EXPENSE,
                    account_id=account.id, note="Продукты")
    )
    assert context.accounts.get(account.id).balance == Decimal("700")


def test_negative_amount_rejected(context, account):
    with pytest.raises(ValueError):
        context.transaction_service.add(
            Transaction(amount=Decimal("-5"),
                        type=TransactionType.EXPENSE,
                        account_id=account.id)
        )


def test_delete_reverts_balance(context, account):
    tx = context.transaction_service.add(
        Transaction(amount=Decimal("500"), type=TransactionType.INCOME,
                    account_id=account.id)
    )
    context.transaction_service.delete(tx.id)
    assert context.accounts.get(account.id).balance == Decimal("0")
    assert context.transaction_service.list(account_id=account.id) == []


def test_filter_by_type(context, account):
    context.transaction_service.add(
        Transaction(amount=Decimal("100"), type=TransactionType.INCOME,
                    account_id=account.id)
    )

```

```

    )
    context.transaction_service.add(
        Transaction(amount=Decimal("40"), type=TransactionType.EXPENSE,
                    account_id=account.id)
    )
    expenses =
context.transaction_service.list(tx_type=TransactionType.EXPENSE)
    assert len(expenses) == 1
    assert expenses[0].amount == Decimal("40")

```

Листинг Г.16 — tests/test_vault.py

"""Тесты хранилища с защитой паролем."""

```

from __future__ import annotations

from decimal import Decimal

import pytest

from wallet.app_context import AppContext
from wallet.core import vault
from wallet.models import Account


def test_init_creates_files(tmp_path):
    vault.init_vault(tmp_path, "pass").close()
    assert (tmp_path / vault.SALT_FILE).exists()
    assert (tmp_path / vault.DB_FILE).exists()
    assert vault.vault_exists(tmp_path)


def test_data_persists_with_correct_password(tmp_path):
    ctx = AppContext.init_vault(tmp_path, "pass")
    ctx.account_service.create("Капта", initial_balance=Decimal("500"))
    ctx.save()
    ctx.close()

    reopened = AppContext.open_vault(tmp_path, "pass")
    accounts = reopened.account_service.list()
    assert [a.name for a in accounts] == ["Капта"]
    assert accounts[0].balance == Decimal("500")


def test_wrong_password_raises(tmp_path):
    vault.init_vault(tmp_path, "right").close()
    with pytest.raises(vault.InvalidPasswordError):
        vault.open_vault(tmp_path, "wrong")


def test_open_missing_vault_raises(tmp_path):

```

```

with pytest.raises(FileNotFoundError):
    vault.open_vault(tmp_path / "nope", "pass")

def test_change_password(tmp_path):
    ctx = AppContext.init_vault(tmp_path, "old")
    ctx.account_service.create("Карта", initial_balance=Decimal("100"))
    ctx.change_password("new")
    ctx.close()

    # старый пароль больше не подходит
    with pytest.raises(vault.InvalidPasswordError):
        vault.open_vault(tmp_path, "old")

    # новый пароль открывает данные
    reopened = AppContext.open_vault(tmp_path, "new")
    assert reopened.account_service.list()[0].balance == Decimal("100")

```